

spintent

SPANISH PARSE INTENTS

v0.99 2026-08-27^{*}

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1 Descripción	1	3.3 El comando <code>\spmoney</code>	6
1.1 Carga del paquete	1	3.3.1 Llaves para <code>\spmoney</code>	7
1.2 El paquete <code>babel</code>	2	3.3.2 Monedas y alias para <code>\spmoney</code>	7
1.3 Fuentes tipográficas	2	3.4 El comando <code>\spshort</code>	7
1.4 El paquete <code>hyperref</code>	2	3.4.1 Llaves para <code>\spshort</code>	9
2 El comando <code>\setspintent</code>	3	3.4.2 Argumentos para <code>\spshort</code>	9
3 The big four	3	4 Utilidades generales	11
3.1 El comando <code>\spnum</code>	3	5 Utilidades para símbolos	11
3.1.1 Llaves para <code>\spnum</code>	3	6 Utilidades para números	13
3.2 El comando <code>\spunit</code>	4	7 Utilidades para geometría plana	17
3.2.1 Llaves para <code>\spunit</code>	4	8 Referencias	24
3.2.2 Argumentos para <code>\spunit</code>	4	9 Change history	25
3.2.3 El comando <code>\spnewunit</code>	6	10 Índice de la Documentación	26
3.2.4 El comando <code>\spunitalias</code>	6	11 Implementation	28
		12 Índice de la Implementación	111

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/`MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/`MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

^{*}Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-08-27.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/lualatex/spintent/
spintent.lua  ⇒ TDS:tex/lualatex/spintent/
spintent.pdf  ⇒ TDS:doc/lualatex/spintent/
README.md    ⇒ TDS:doc/lualatex/spintent/
spintent.dtx  ⇒ TDS:source/lualatex/spintent/
spintent.ins  ⇒ TDS:source/lualatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como *⟨valor prohibido⟩* no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo *⟨valor prohibido⟩* se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*²»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

• También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

<code>\spnum</code>	<code>\spnum{⟨número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número;número;número⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨+número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número.número;número⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨-número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número;número⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨número;número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número e exponente⟩}</code>
	<code>\spnum[⟨key = val⟩]{⟨número.número⟩}</code>	<code>\spnum[⟨key = val⟩]{⟨número E exponente⟩}</code>

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {*⟨número⟩*}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma ';'.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {*⟨número⟩*} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

• La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {*⟨argumento⟩*}. El {*⟨argumento⟩*} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}`

default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '10 00 00' en la salida PDF.

`read-sign = {⟨terse | clear⟩}`

default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en *terse*, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}`

default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘,’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{\argumento}` una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘,’ periódica.

`notation-sym = {\times | \cdot}`

default: `\cdot`

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘x’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en `cdot` imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit</code>	<code>\spunit{\langle unidad \rangle}</code>	<code>\spunit[\langle key = val \rangle]{\langle número unidad * unidad \rangle}</code>
	<code>\spunit[\langle key = val \rangle]{\langle unidad \rangle}</code>	<code>\spunit[\langle key = val \rangle]{\langle número unidad * unidad / unidad \rangle}</code>
	<code>\spunit[\langle key = val \rangle]{\langle número unidad \rangle}</code>	<code>\spunit[\langle key = val \rangle]{\langle número unidad * unidad / unidad * unidad \rangle}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{\langle unidad \rangle}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{\langle argumento \rangle}` seguido de una `{\langle unidad \rangle}` o un `{\langle alias \rangle}` valido descritos en §3.2.2 o una `{\langle unidad \rangle}` creada por el comando `\spnewunit` (§3.2.3) o un `{\langle alias \rangle}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]` y se ejecuta `\spunit{23 m}` se imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{\langle unidades \rangle}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{\langle exponente \rangle}`’. Por ejemplo si se ejecuta `\spunit{23 m * s^2}` se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{\langle unidades \rangle}` utilizando una «*única barra*» ‘/’ para la separación entre las `{\langle unidades \rangle}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta `\spunit{23 m/s^2}` se imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{\langle argumento \rangle}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra*» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {\true | false}`

default: `false`

Establece si se *imprime* `\cdot` o un *espacio fino* ‘\,’ para la multiplicación de `{\langle unidades \rangle}`. Si se establece en `true`, se imprimirá ‘.’ entre las `{\langle unidades \rangle}` en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las `{\langle unidades \rangle}` en la salida PDF.

`read-cdot = {\true | false}`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{\langle unidades \rangle}`. Si se establece en `true`, NVDA/MathCAT verbalizará «*por*» entre las `{\langle unidades \rangle}`; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las `{\langle unidades \rangle}`.

`read-slash = {\por | partido-por}`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra*» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «*por*»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{\langle unidades \rangle}` y `{\langle alias \rangle}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\langle nueva unidad \rangle}{\langle verbalización NVDA \rangle}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\langle \text{nueva unidad} \rangle$ que establece lo que se imprimirá en la salida PDF y $\langle \text{verbalización NVDA} \rangle$ que establece como NVDA/MathCAT verbalizará la $\langle \text{nueva unidad} \rangle$.

La $\{\langle \textit{nueva unidad} \rangle\}$ no debe estar registrada como $\{\langle \textit{unidad} \rangle\}$ o $\{\langle \textit{alias} \rangle\}$ para `\spunit` descritos en §3.2.2. Si la $\{\langle \textit{nueva unidad} \rangle\}$ ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\langle\textit{nueva unidad}\rangle$ es «global» y se almacena en el módulo `spintent.lua` como una $\langle\textit{unidad}\rangle$ válida para pasar como $\langle\textit{argumento}\rangle$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\langle\textit{nueva unidad}\rangle$ ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias` `\spunitalias{nuevo alias}{expresión de unidades}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\{\text{\textcolor{teal}{nuevo alias}}\}\}$ que establece es el nuevo $\{\{\text{\textcolor{teal}{alias}}\}\}$ de $\{\{\text{\textcolor{teal}{argumento}}\}\}$ para `\spunit` y $\{\{\text{\textcolor{teal}{expresión de unidades}}\}\}$ que puede contener multiplicación o división de $\{\{\text{\textcolor{teal}{unidades}}\}\}$ ya registradas que se imprimirán en la salida PDF.

El `{\langle nuevo alias \rangle}` no debe estar registrado como `{\langle unidad \rangle}` o `{\langle alias \rangle}` para `\spunit` descritos en §3.2.2 o estar definido como `{\langle unidad \rangle}` por medio del comando `\spnewunit`. Si el `{\langle nuevo alias \rangle}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `\{nuevo alias\}` es «global» y se almacena en el módulo `spintent.lua` como una `\{alias\}` valido para pasar como `\{argumento\}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el `\{alias\}` ‘caudal’ y cuando se utilice `$\spunit{50 caudal}$` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<code>\spmoney</code>	<code>\spmoney{<i>cifra</i>}</code>
	<code>\spmoney{<i>cifra moneda</i>}</code>
	<code>\spmoney[<i>key = val</i>]{<i>cifra moneda</i>}</code>

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{«cifra»}` que contiene la «*cifra numérica*» positiva o negativa la se cual procesa internamente por el comando `\spnum` al inicio del `{«argumento»}` acompañada de un «*símbolo de moneda*» o «*alias de moneda*» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `$\spmoney{500}$` se imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `\$ \spmoney{500} dolares` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «*quinientos dolares*» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estándar ISO. Por ejemplo: `$\spmoney{500 CLP}` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos».

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en **NVDA/MathCAT** agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 clp}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo de moneda*» como '\$', '€', '£' o un «*alias de moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo de moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo de moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo de moneda*» poco habitual que esté soportada por `spintent`. El `{\argumento}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {\moneda | alias}` default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{\argumento}` al comando `\spmoney`. El «*símbolo de moneda*» o el «*alias de moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {\true | false}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

`dolar-math = {\true | false}` default: true

Establece si el *símbolo* '\$' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los `{\símbolos de moneda}` y `{\alias de moneda}` soportados como `{\argumento}` para `\spmoney`.

3.4 El comando \spshort

`\spshort` `\spshort{\abreviaturas | acrónimos | números ordinales}`
`\spshort[key = val]{\abreviaturas | acrónimos | números ordinales}`
`\spshort[read-arg={\verbalización del argumento para NVDA}]{\comandos LATEX}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{\abreviaturas}`, `{\acrónimos}` o `{\números ordinales}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr . a}` se imprimirá 'Dr.^a' en la salida PDF y NVDA/MathCAT verbalizará «*doctora*»; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará «*organización de las naciones unidas*» y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará «*ministerio de educación*».

Para la escritura de `{\números ordinales}` se puede ejecutar `\spshort{1 . a}` que imprimirá '1.^a' en la salida PDF y NVDA/MathCAT verbalizará «*primera*»; si se ejecuta `\spshort{1 . o}` imprimirá '1.^o' en la salida PDF y NVDA/MathCAT verbalizará «*primero*».

Para crear una nueva `{\abreviatura}`, `{\acrónimo}` o `{\número ordinal}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{\comandos LATEX}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{\números ordinales}` se debe tener cuidado y no confundir la «*letra o voladita*» 'º' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «*letra voladita*», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1 . o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₴	grivna, grivnas	grivna	₴	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = { <verbalización NVDA> }` default: no usado

Establece la verbalización del argumento { <comandos $\LaTeX$ > } de la nueva { <abreviatura> }, { <acrónimo> } o { <número ordinal> } para NVDA. El valor de esta llave siempre debe pasarse entre ‘{ }’. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva { <abreviatura> } que imprimirá ‘8.^{avo}’ en la salida PDF y NVDA verbalizará «octavo».

`small-caps = { <true | false> }` default: false

Establece si se usarán versalitas para imprimir { <acrónimos> } y para la «letra volada» en { <abreviaturas> } y { <números ordinales> } respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán versalitas. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^a’ en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete **spintent** provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete **spintent** que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete **tagpdf** en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el argumento `{<siglo>}`. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/ MM/YYYY>} \spdate{<DD- MM-YYYY>}
\spdate{<YYYY/ MM/DD>} \spdate{<YYYY- MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el argumento `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete **spintent** provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por **spintent** (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «forma» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spusep`

```
\spusep <separador>
\spusep[<key = val>]{<separador>}
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<separador>}`, el cual debe ser uno de los «delimitadores» soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’. Imprime el *delimitador* indicado con el tamaño establecido por la llave `size` y controla si NVDA/MathCAT lo verbaliza o no mediante la llave `silent`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\$<separador>$` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «paréntesis izquierdo».

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

`size = {<none> | <big> | <Big> | <bigg> | <Bigg>}` default: none

Establece el *tamaño* del `{<delimitador>}` que se imprimirá en la salida PDF. Los valores corresponden a los cuatro tamaños fijos provistos por L^AT_EX: si se establece en `none`, el `{<delimitador>}` se imprime en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr`; si se establece en `Big`, se usa `\Bigl/\Bigr`; si se establece en `bigg`, se usa `\biggl/\biggr`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr`. El tamaño se aplica automáticamente en el lado correcto (izquierdo o derecho) según el `{<delimitador>}` pasado.

`silent = {<true> | <false>}` default: false

Establece si NVDA/MathCAT verbalizará el `{<delimitador>}` impreso por `\spusep`. Si se establece en `true`, el `{<delimitador>}` se marcará con el *intent* `:silent` y NVDA/MathCAT no lo verbalizará; si se establece en `false`, NVDA/MathCAT verbalizará el `{<delimitador>}` de forma normal.

El comando `\spdsym`

```
\spdsym
\spdsym[<key = val>]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\$<spdsym>$` imprimirá ‘÷’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para `\spdsym`

`prefix = {<prefijo>}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`suffix = {<sufijo>}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`set-sym = {<old-style> | <two-dots> | <slash>}` default: two-dots

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘⋮’ y si se establece `slash`, imprimirá ‘/’.

`read-sym = {<dividido> | <dividido-por> | <dividido-entre>}` default: dividido

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «dividido»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «dividido por» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «dividido entre».

El comando `\spmsym`

```
\spmsym
\spmsym[<key = val>]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\$<spmsym>$` imprimirá ‘⋅’ en la salida PDF y NVDA/MathCAT verbalizará «por».

Llaves para `\spmsym`

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix = {⟨sufijo⟩}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el {⟨valor⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym = {⟨cross | dot | asterisk⟩}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá '×'; si se establece en `dot`, imprimirá '·' y si se establece en `asterisk`, imprimirá '*'.

`read-sym = {⟨por | multiplicado-por | veces⟩}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {⟨true | false⟩}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

`\spdiv {⟨dividendo⟩} {⟨divisor⟩}`
`\spdiv [⟨key = val⟩] {⟨dividendo⟩} {⟨divisor⟩}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨dividendo⟩} y {⟨divisor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spdiv{20}{26}$` imprimirá '20 : 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

`wrap-parens = {⟨true | false⟩}` default: false

Establece si se *imprimen* paréntesis redondos '(...)' alrededor de los *argumentos* {⟨dividendo⟩} y {⟨divisor⟩} pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos '(...)' alrededor de los *argumentos* {⟨dividendo⟩} y {⟨divisor⟩} pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul {⟨factor⟩} {⟨factor⟩}`
`\spmul [⟨key = val⟩] {⟨factor⟩} {⟨factor⟩}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨factor⟩} y {⟨factor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], `$\spmul{20}{26}$` imprimirá '20 · 26' en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens = {⟨true | false⟩}` default: false

Establece si se *imprimen* paréntesis redondos '(...)' alrededor de los *argumentos* {⟨factor⟩} y {⟨factor⟩} pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor de los *argumentos* $\{ \langle \text{factor} \rangle \}$ y $\{ \langle \text{factor} \rangle \}$ pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD \spnD
\spnD(\<número natural>)
\spnD[key = val](\<número natural>)
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \text{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \text{argumento} \rangle)$, `\spnD$` imprimirá ‘D(n)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de n».

Si se usa con $(\langle \text{argumento} \rangle)$, por ejemplo, `\spnD(6)$`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del $(\langle \text{número natural} \rangle)$ pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)$` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del $(\langle \text{argumento} \rangle)$ pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)$` imprimirá ‘D(m)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de m»; si se establece en `false`, el $(\langle \text{argumento} \rangle)$ debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { \<true | false> }` default: false

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(\<argumento>)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin $(\langle \text{argumento} \rangle)$. Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(\<número natural>)
\spnM[key = val](\<número natural>)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \text{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \text{argumento} \rangle)$, `\spnM$` imprimirá ‘M(n)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de n».

Si se usa con $(\langle \text{argumento} \rangle)$, por ejemplo, `\spnM(6)$`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del $(\langle \text{número natural} \rangle)$ pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)$` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (*argumento*) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = {\true | false}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc(\números separados por coma)
\spmc[key = val](\números separados por coma)
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = {\true | false}` default: `false`

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true`, se imprimirá ‘MCD’ en la salida PDF; si se establece en `false`, se imprimirá ‘mcd’ en la salida PDF.

`read-cmd = {\terse | clear}` default: `clear`

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c d*»; si se establece en `clear`, NVDA/MathCAT verbalizará «máximo común divisor».

`prefix = {\prefijo}` default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {\true | false}` default: `false`

Establece si se *imprime* el resultado de calcular el «máximo común divisor» del (*argumento*) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcd(2, 3, 6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis es igual a uno»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «máximo común divisor».

`sample-arg = {\true | false}` default: `false`

Desactiva la *validación* interna del (*argumento*) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcd(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre *m* y *n*»; si se establece en `false` el (*argumento*) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = {\true | false}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc(\números separados por coma)
\spmc[key = val](\números separados por coma)
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo» o «*m c m*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcm(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis» o «*m c m* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case` = { `<true>` | `<false>` } default: `false`

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true` se imprimirá ‘MCM’ en la salida PDF; si se establece en `false` se imprimirá ‘mcm’ en la salida PDF.

`read-cmd` = { `<terse>` | `<clear>` } default: `clear`

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c m*»; si se establece en `clear`, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result` = { `<true>` | `<false>` } default: `false`

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del (`<argumento>`) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «*mínimo común múltiplo*».

`sample-arg` = { `<true>` | `<false>` } default: `false`

Desactiva la *validación* interna del (`<argumento>`) pasado a `\spmc`, permitiendo usar ejemplos «*no numéricos*» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcm(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre m y n*»; si se establece en `false` el (`<argumento>`) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd` = { `<true>` | `<false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(<argumento>)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (`<argumento>`). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

`\spnZ` `\spnZ{<número entero>}`
`\spnZ[<key = val>]{<número entero>}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio { `<número entero>` } el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las *meta-keys* `cifras-sep` y `read-sign` pasadas al comando `\spnum` (§3.1.1).

`wrap-parens` = { `<true>` | `<false>` } default: `false`

Establece si se *imprimen* paréntesis redondos ‘()’ alrededor del argumento { `<número entero>` } pasado a `\spnZ`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor del { `<número entero>` }; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens` = { `<true>` | `<false>` } default: `false`

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘()’ alrededor del *argumento* { `<número entero>` } pasado a `\spnZ`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura del paréntesis ‘()’; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` `\spmQ{<número entero>}{<numerador>}{<denominador>}`
`\spmQ[<key = val>]{<número entero>}{<numerador>}{<denominador>}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: { `<número entero>` }, { `<numerador>` } y { `<denominador>` }, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spmQ{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de **spintent**. Por ejemplo, la entrada `\frac{5}{7}` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}`

default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en `entero`, al usar `\spmQ[join-word=entero]{1}{5}{7}` imprimirá ‘ $1\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «un entero cinco séptimos»; si se establece en `enteros`, al usar `\spmQ[join-word=enteros]{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «dos enteros cinco séptimos»; si se establece en `y`, al usar `\spmQ[join-word=y]{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF, y NVDA/MathCAT verbalizará «dos y cinco séptimos».

`use-spnZ = {⟨true | false⟩}`

default: *true*

Establece si el `{⟨numerador⟩}` y el `{⟨denominador⟩}` que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en `true`, `{⟨numerador⟩}` y `{⟨denominador⟩}` se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en `false`, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac = {⟨true | false⟩}`

default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\spmQ[print-dfrac=true]{2}{5}{7}` imprimirá la fracción en tamaño *display* ‘ $2\frac{5}{7}$ ’ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}`

default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: *true*

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría plana

El paquete `spintent` provee comandos para trabajar con los objetos y medidas de la *geometría plana* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y comparten el mismo módulo `spintent.lua` para construir la verbalización correcta.

- Todos los comandos de esta sección aceptan como puntos solo *letras mayúsculas* como base, con modificadores opcionales de subíndice ‘`_{\dots}`’ o prima ‘`’`’. La única excepción es `\sprecta` cuando el *argumento* es de la forma `{⟨letra⟩}`, que acepta *letras minúsculas*. Los subíndices siempre deben escribirse de la forma correcta usando `_{\dots}` como `A_{\dots}` y nunca como `A_n` sin llaves que funciona solo por coincidencia.

El comando `\spPoint`

`\spPoint` `\spPoint{⟨punto⟩}`
`\spPoint[⟨key = val⟩]{⟨punto⟩}`

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨punto⟩}` el cual es una *única letra mayúscula* con un modificador opcional de subíndice ‘`_{\dots}`’ o prima ‘`’`’.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spPoint{A}` imprimirá ‘A’ en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `\spPoint{A_{1}}` imprimirá ‘A₁’ y verbalizará «punto A sub uno».

- El comando `\spPoint` solo acepta *un punto*. Para pares o listas de puntos (rectas, rayos, lados, ángulos, arcos, polígonos), véanse los comandos correspondientes de esta sección.

Llaves para `\spPoint`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}` sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «punto A»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «punto». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «punto». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\sprecta`

```
\sprecta <punto, punto> <letra>
\sprecta[<key = val>]{<punto, punto>} <letra>
```

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{<letra>}` o `{<punto, punto>}`. Si el *argumento* es de la forma `{<punto, punto>}` se aplica `\overleftrightharpoonrightarrow` en la impresión; si el *argumento* no contiene coma y es de la forma `{<letra>}` (mayúscula o minúscula), se imprimirá solo la `{<letra>}` sin aplicar `\overleftrightharpoonrightarrow`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\sprecta{A, B}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; `\sprecta{m}` imprimirá m en la salida PDF y NVDA/MathCAT verbalizará «recta m».

Llaves para `\sprecta`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto, punto>}` y el argumento `{<letra>}` cuando está en mayúscula. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{A, B}` se verbaliza «recta A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «recta». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «recta». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\sprayo`

```
\sprayo <punto, punto>
\sprayo[<key = val>]{<punto, punto>}
```

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}`. Representa un *rayo* o *semirecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\sprayo{A, B}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para `\sprayo`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprayo{A, B}` se verbaliza «rayo A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término establecido por la llave `read-sty`. Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

```
read-sty = {<rayo | semirrecta>} default: rayo
```

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\sprayo[read-sty=rayo]{A, B}`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `\sprayo[read-sty=semirrecta]{A, B}`, NVDA/MathCAT verbalizará «semirrecta A B».

- La impresión $\overrightarrow{AB}$ en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

```
\spside <punto, punto>
\spside[<key = val>]{<punto, punto>}
```

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}`. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spside{A, B}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$(\spside{A, B})$` se verbaliza «*lado A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término establecido por la llave `read-sty`. Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$(\spside[read-sty=lado]{A, B})$`, NVDA/MathCAT verbalizará «*lado A B*»; si se utiliza `$(\spside[read-sty=segmento]{A, B})$`, NVDA/MathCAT verbalizará «*segmento A B*» y si se utiliza `$(\spside[read-sty=trazo]{A, B})$`, NVDA/MathCAT verbalizará «*trazo A B*».

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *término* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `$(\spside[read-arg={hipotenusa}]{A, B})$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*hipotenusa A B*».

El comando `\spmside`

`\spmside` `\spmside{<punto, punto>}`
 `\spmside[<key = val>]{<punto, punto>}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` }. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ' $\overline{AB}$ '; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* [`<key = val>`], `$(\spmside{A, B})$` imprimirá ' AB ' en la salida PDF y NVDA/MathCAT verbalizará «*medida del lado A B*».

Llaves para `\spmside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$(\spmside{A, B})$` se verbaliza «*medida del lado A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar «*medida del*» seguido por el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de «*medida del*» seguido por el término establecido por la llave `read-sty`. Si se establece en `true`, NVDA/MathCAT NO verbalizará «*medida del*» seguido por el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$(\spmside[read-sty=lado]{A, B})$`, NVDA/MathCAT verbalizará «*medida del lado A B*»; si se utiliza `$(\spmside[read-sty=segmento]{A, B})$`, NVDA/MathCAT verbalizará «*medida del segmento A B*» y si se utiliza `$(\spmside[read-sty=trazo]{A, B})$`, NVDA/MathCAT verbalizará «*medida del trazo A B*».

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *término* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `$(\spmside[read-arg={medida cateto}]{A, B})$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*medida cateto A B*».

`print-m` = { `<true | false>` } default: `false`

Establece si se *imprime* la notación visual 'm' antes del argumento. Si se establece en `true`, `$(\spmside[print-m=true]{A, B})$` imprimirá ' $m\overline{AB}$ ' en la salida PDF, si se establece en `false` imprimirá ' AB ' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «*medida del lado A B*».

El comando `\spangle`

<code>\spangle</code>	<code>\spangle{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
	<code>\spangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\angle` o `\rightangle` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `\spangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa» y `\spangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{A, O, B}` se verbaliza «ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «ángulo». Si se establece en *true*, NVDA/MathCAT NO verbalizará el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`recto = {⟨true | false⟩}` default: *false*

Establece si se usará `\rightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A», `\spangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

El comando `\spmangle`

<code>\spmangle</code>	<code>\spmangle{⟨punto letra griega número⟩}</code>	<code>\spmangle{⟨punto, punto, punto⟩}</code>
	<code>\spmangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spmangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa» y `\spmangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

Llaves para `\spmangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{A, O, B}` se verbaliza «medida del ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del ángulo». Si se establece en *true*, NVDA/MathCAT NO verbalizará el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`recto = {⟨true | false⟩}` default: *false*

Establece si se usará `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A», `\spmangle[recto=true]{A, O, B}`

imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto $A O B$ »; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

`print-m = {\langle true | false \rangle}` default: `false`

Establece si se *imprime* la notación visual ‘m’ antes del argumento. Si se establece en `true`, `\spmangle[print-m=true]{A}`, imprimirá ‘ $m \angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo $A O B$ ».

El comando `\spang`

```
\spang \spang{\langle grados \rangle}
\spang{\langle grados : minutos \rangle}
\spang{\langle grados : minutos : segundos \rangle}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{\langle grados \rangle}`, `{\langle grados : minutos \rangle}` o `{\langle grados : minutos : segundos \rangle}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar el «ángulo sexagesimal». Por ejemplo, `\spang{45:30:15}` imprimirá ‘ $45^{\circ}30'15''$ ’ en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

```
\sparc \sparc{\langle punto, punto \rangle} \sparc{\langle punto, punto, punto \rangle}
\sparc[\langle key = val \rangle]{\langle punto, punto \rangle} \sparc[\langle key = val \rangle]{\langle punto, punto, punto \rangle}
```

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Representa un arco determinado por dos o tres puntos aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\sparc{A, B}` imprimirá ‘ $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «arco $A B$ ».

Llaves para `\sparc`

`read-cmd = {\langle school | mathcat \rangle}` default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{A, B}` se verbaliza «arco $A B$ »; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\langle prefijo \rangle}` default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {\langle true | false \rangle}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmarc`

```
\spmarc \spmarc{\langle punto, punto \rangle} \spmarc{\langle punto, punto, punto \rangle}
\spmarc[\langle key = val \rangle]{\langle punto, punto \rangle} \spmarc[\langle key = val \rangle]{\langle punto, punto, punto \rangle}
```

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Representa la *medida* del arco determinado por dos o tres puntos. La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* ‘ $\widehat{AB}$ ’; `\spmarc` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\spmarc{A, B}` imprimirá ‘ $m \widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco $A B$ ».

Llaves para `\spmarc`

`read-cmd = {\langle school | mathcat \rangle}` default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{\langle punto, punto \rangle}` o `{\langle punto, punto, punto \rangle}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{A, B}` se verbaliza «medida del arco $A B$ »; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {\langle prefijo \rangle}` default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {\langle true | false \rangle}` default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spPoly`

```
\spPoly {<punto, punto, punto, ...>}
\spPoly[<key = val>]{<punto, punto, punto, ...>}
```

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto, punto, ...>}`, una lista de al menos tres `{<puntos>}` separados por comas. El tipo de *símbolo* en la impresión y la verbalización se determinan acorde al número de puntos en el argumento: Si son tres puntos se aplica `\triangle` y añade el término «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el término «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spPoly{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; `\spPoly{A, B, C, D}` imprimirá ' $\square ABCD$ ' y verbalizará «cuadrado A B C D»; `\spPoly{A, B, C, D, E}` imprimirá ' $ABCDE$ ' y verbalizará «A B C D E».

Llaves para `\spPoly`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto, punto, punto, ...>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{A, B, C}` se verbaliza «triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el argumento o los términos «triángulo» y «cuadrado» establecido por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

```
print-sym = {<true | false>} default: true
```

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión añadiendo el término correspondiente a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spPoly[print-sym=true]{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; si se establece en `false`, imprimirá ' ABC ' en la salida PDF y NVDA/MathCAT omitirá el término «triángulo», verbalizando solo «A B C».

```
silent-cmd = {<true | false>} default: false
```

Alias de `print-sym=false`, incluido por consistencia con el resto de los comandos de geometría plana.

El comando `\spAfig`

```
\spAfig {<punto, punto, punto, ...>}
\spAfig(<número | letra | letra_{número}>)
\spAfig[<key = val>]{<punto, punto, punto, ...>}
```

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el área de una figura poligonal en la impresión aplicando `\mathcal{A}`.

El argumento admite tres formas: una *lista de vértices* `(<punto, punto, punto, ...>)` (al menos tres); un *número o letra sueltos* `(<1>)`, `(<n>)`; o una *letra de una tabla fija* (F, P, B, L) con subíndice numérico opcional, `(<F_1>)`.

Si no se proporciona el *argumento opcional* `[<key = val>]` ni el argumento delimitado por paréntesis, `\spAfig` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área». Con vértices, `\spAfig(A, B, C)` imprimirá ' $\mathcal{A}_{\triangle ABC}$ ' y verbalizará «área del triángulo A B C». Con un número o letra sueltos, `\spAfig(1)` imprimirá ' $\mathcal{A}_1$ ' y verbalizará «área uno»; `\spAfig(n)` imprimirá ' $\mathcal{A}_n$ ' y verbalizará «área n». Con una letra de la tabla, `\spAfig(F)` imprimirá ' $\mathcal{A}_F$ ' y verbalizará «área de la figura»; `\spAfig(F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' y verbalizará «área de la figura uno».

Llaves para `\spAfig`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(A, B, C)` se verbaliza «área del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

```
print-sym = {<auto | triangle | square | none>} default: auto
```

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión. Con vértices, `auto` infiere el símbolo según el número de puntos (tres o cuatro); `none` lo omite y NVDA/MathCAT verbaliza solo los vértices sin nombrar la figura: `\spAfig[print-sym=none](A, B, C)` imprimirá ' $\mathcal{A}_{ABC}$ ' y verbalizará «área A B C».

Forzar `triangle/square` exige que coincida con el número real de vértices; en caso contrario se produce un error. Con un número o letra sueltos, `auto` y `none` se comportan igual (sin símbolo); `triangle/square` anida el argumento bajo el símbolo: `\spAfig[print-sym=triangle](1)` imprimirá ' $\mathcal{A}_{\Delta_1}$ ' y verbalizará «área del triángulo uno». Esta llave no puede combinarse con una letra de la tabla (F, P, B, L).

`read-sub = {⟨true | false⟩}` default: false

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único. Si se establece en `true`, `\spAfig[read-sub=true](1)` verbaliza «área sub uno»; si se establece en `false`, verbaliza «área uno». No tiene efecto en la notación de vértices ni cuando el argumento es una letra de la tabla sin subíndice.

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {⟨nombre⟩}` default: no usado

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapezio A B C D».

El comando `\spPfig`

`\spPfig` `\spPfig`
`\spPfig(⟨punto, punto, punto, ...⟩)`
`\spPfig(⟨número | letra | letra_{número}⟩)`
`\spPfig[⟨key = val⟩](⟨punto, punto, punto, ...⟩)`

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el *perímetro* de una figura poligonal en la impresión aplicando `\mathcal{P}`.

El argumento admite las mismas tres formas que `\spAfig` (§7): una *lista de vértices*, un *número o letra sueltos*, o una *letra de la tabla fija* (F, P, B, L) con subíndice numérico opcional.

Si no se proporciona el *argumento opcional* [`key = val`] ni el argumento delimitado por paréntesis, `\spPfig` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro». Con vértices, `\spPfig(A, B, C)` imprimirá ' $\mathcal{P}_{\Delta ABC}$ ' y verbalizará «perímetro del triángulo A B C». Con un número o letra sueltos, `\spPfig(2)` imprimirá ' $\mathcal{P}_2$ ' y verbalizará «perímetro dos». Con una letra de la tabla, `\spPfig(P_{\{2\}})` imprimirá ' $\mathcal{P}_{P_2}$ ' y verbalizará «perímetro del polígono dos».

Llaves para `\spPfig`

`read-cmd = {⟨school | mathcat⟩}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(A, B, C)` se verbaliza «perímetro del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {⟨auto | triangle | square | none⟩}` default: auto

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión, con la misma semántica que en `\spAfig` (§7).

`read-sub = {⟨true | false⟩}` default: false

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único, con la misma semántica que en `\spAfig` (§7).

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {⟨nombre⟩}` default: no usado

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spPfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapezio A B C D».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-08-27 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAfig	22
\spPfig	23
\spPoint	17
\spPoly	22
\spangle	20
\spang	21
\sparc	21
\spdate	11
\spdiv	13
\spdsym	12
\spmQ	16
\spmangle	20
\spmarc	21
\spmcd	15
\spmcm	15
\spmoney	6
\spmside	19
\spmsym	12
\spmul	13
\spnD	14
\spnM	14
\spnZ	16
\spnewunit	6
\spnum	3
\sprayo	18
\spread	11
\sprecta	18
\spshort	7
\spside	18
\spsiglo	11
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
Keys for \spdiv provide by spintent :	
read-parens	13
wrap-parens	13
Keys for \spdsym provide by spintent :	
prefix	12
read-sym	12
set-sym	12
suffix	12
Keys for \spmangle provide by spintent :	
prefix	20
print-m	21
read-cmd	20
recto	20
silent-cmd	20
Keys for \spmarc provide by spintent :	
prefix	21
read-cmd	21
silent-cmd	21
Keys for \spmcd provide by spintent :	
prefix	15
read-cmd	15
result	15
sample-arg	15
silent-cmd	15
upper-case	15
Keys for \spmcm provide by spintent :	
prefix	16
read-cmd	16
result	16
sample-arg	16
silent-cmd	16
upper-case	16
Keys for \spmoney provide by spintent :	
currency	7
dolar-math	7
sub-units	7
Keys for \spmQ provide by spintent :	
join-word	17
print-dfrac	17
read-parens	17
use-spZ	17
wrap-parens	17
Keys for \spmside provide by spintent :	
prefix	19
print-m	19
read-arg	19
read-cmd	19
read-sty	19
silent-cmd	19
Keys for \spmsym provide by spintent :	
not-print	13
prefix	13
read-sym	13
set-sym	13
suffix	13
Keys for \spmul provide by spintent :	
read-parens	13
wrap-parens	13
Keys for \spnD provide by spintent :	
prefix	14
result-num	14
K	
Keys for \spAfig provide by spintent :	
prefix	22
print-sym	22
read-arg	23
read-cmd	22
read-sub	23
silent-cmd	23
Keys for \spangle provide by spintent :	
prefix	20
read-cmd	20
recto	20
silent-cmd	20
Keys for \sparc provide by spintent :	
prefix	21
read-cmd	21
silent-cmd	21

result	14	notation-sym	13
sample-arg	14	read-period-sep	13
silent-cmd	14	read-sign	13
Keys for \spnM provide by spintent :		read-sym	13
prefix	14	set-sym	13
result-num	14	Keys for \spdsym provide by spintent :	
result	14	read-sym	12
sample-arg	14	Keys for \spmcd provide by spintent :	
silent-cmd	15	read-cmd	15
Keys for \spnum provide by spintent :		Keys for \spmcm provide by spintent :	
cifras-sep	3	read-cmd	16
notation-sym	4	Keys for \spmoney provide by spintent :	
read-period-sep	3	cifras-sep	7
read-sign	3	Keys for \spmQ provide by spintent :	
Keys for \spnZ provide by spintent :		wrap-parens	17
read-parens	16	Keys for \spmsym provide by spintent :	
wrap-parens	16	read-sym	13
Keys for \spPfig provide by spintent :		Keys for \spm\ provide by spintent :	
prefix	23	cifras-sep	13
print-sym	23	notation-sym	13
read-arg	23	read-period-sep	13
read-cmd	23	read-sign	13
read-sub	23	read-sym	13
silent-cmd	23	set-sym	13
Keys for \spPoint provide by spintent :		Keys for \spnZ provide by spintent :	
prefix	17	cifras-sep	16
read-cmd	17	read-sign	16
silent-cmd	17	Keys for \spPoly provide by spintent :	
Keys for \spPoly provide by spintent :		print-sym	22
prefix	22	Keys for \spside provide by spintent :	
print-sym	22	read-sty	19
read-cmd	22	Keys for \spunit provide by spintent :	
silent-cmd	22	cifras-sep	4
Keys for \sprayo provide by spintent :		notation-sym	4
prefix	18	read-period-sep	4
read-cmd	18	Keys for \spusep provide by spintent :	
read-sty	18	silent	12
silent-cmd	18	size	12
Keys for \sprecta provide by spintent :			
prefix	18		
read-cmd	18		
silent-cmd	18		
Keys for \spshort provide by spintent :			
read-arg	9		
small-caps	9		
Keys for \spside provide by spintent :			
prefix	19		
read-arg	19		
read-cmd	19		
read-sty	19		
silent-cmd	19		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
print-cdot	4		
read-cdot	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		
size	12		
Keys for \spd\ provide by spintent :			
cifras-sep	13		

M

Lua module provide by spintent :	
spintent.lua	6, 7, 9, 10, 17

P

Packages :	
babel-spanish	2
babel	2, 7
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 7, 11
unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 Initial set up

Start the DocStrip guards.

```
1 <(*package)
```

Identify the internal prefix (L^AT_EX DocStrip convention) for `l3doc` class.

```
2 <__spintent=spintent>
```

11.2 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8   {
9     spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10    later.~Update~your~TeX~distribution.
11  }
12 }
```

Now declare the `spintent` package and we will add the only key `mathcal` that can be passed as a package option.

```
13 \ProvidesExplPackage {spintent} {2026-08-27} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n    = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n  = true,
19   mathcal .value_required:n = true,
20 }
21 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_{TeX} must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
22 \IfDocumentMetadataTF
23 {
24   \sys_if_engine luatex:TF
25   {
26     \msg_new:nnn { spintent } { package-not-load }
27     {
28       The~'#1'~package~will~be~loaded~as~a~dependency.
29     }
30     \IfPackageLoadedF { unicode-math }
31     {
32       \IfPackageLoadedF { lua-unicode-math }
33       {
34         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
35         \RequirePackage{unicode-math}
36       }
37     }
38     \msg_new:nnn { spintent } { env-success }
39     { Everything~looks~correct,~spintent~will~make~the~intent! }
```



```

40     \msg_info:nn { spintent } { env-success }
41   }
42   {
43     \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
44     { The~spintent~package~requires~LuaLaTeX. }
45     { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
46     \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
47   }
48 }
49 {
50   \msg_new:nnnn { spintent } { fatal-metadata-missing }
51   { \token_to_str:N \DocumentMetadata \c_space_tl is~missing. }
52   {
53     The~spintent~package~requires~tagged~PDF~active.\\
54     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
55   }
56   \msg_fatal:nn { spintent } { fatal-metadata-missing }
57 }

```

11.4 Some kernel variants

```

\str_if_eq:nVTF \str_if_eq:nVF \str_uppercase:V \str_lowercase:V \seq_set_from_clist:Ne
58 \cs_generate_variant:Nn \str_if_eq:nnTF { V }
59 \cs_generate_variant:Nn \str_if_eq:nnF { V }
60 \cs_generate_variant:Nn \str_uppercase:n { V }
61 \cs_generate_variant:Nn \str_lowercase:n { V }
62 \cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

Non-standard kernel variants.

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal` and `\rightanglesqr`

First, we declare the variables `\l__spintent_luaset_cal_fam_ok_str` and `\l__spintent_luaset_sym_fam_ok_str` before loading the module `spintent.lua`. This is necessary because of the implementation of the functions `\__spintent_luafun_classic_mathcal:nn` and `\__spintent_luafun_right_angle_sqr:` within the Lua module; otherwise, `expl3` will complain about duplicate variable declarations.

```

63 \str_new:N \l__spintent_luaset_cal_fam_ok_str
64 \str_new:N \l__spintent_luaset_sym_fam_ok_str

```

(End of definition for `\l__spintent_luaset_cal_fam_ok_str` and `\l__spintent_luaset_sym_fam_ok_str`.)

11.6 Loading the Lua module and functions

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit` used by `\spunit` (§11.15), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.17), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.30) and `\__spintent_luafun_clean_split_and_set:V` used by `\spnum` (§11.14) defined in it.

```

65 \lua_load_module:n { spintent.lua }
66 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
67 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
68 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { VV, Vn }
69 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdfTeX for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

```

mathcal-fallback
right-angle-sqr-fallback

```

First, we'll define two error messages in case our fallback fails.

```

70 \msg_new:nnn { spintent } { mathcal-fallback }
71 {
72   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~(font~not~found~or~
73   no~free~math~family~available).~\msg_line_context:
74 }
75 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
76 {
77   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~for~the~right-angle~
78   glyph.~\msg_line_context:
79 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\@@_mathcal:n` Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

80 \cs_new_protected:Nn \__spintent_mathcal:n
81 {
82   \bool_if:NTF \__spintent_mathcal_bool
83   {
84     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
85     { \__spintent_luafun_classic_mathcal:n {#1} }
86     {
87       \msg_warning:nn { spintent } { mathcal-fallback }
88       \mathcal {#1}
89     }
90   }
91   { \mathcal {#1} }
92 }

```

(End of definition for `\@@_mathcal:n`.)

`\@@_right_angle_sqr:` The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check if the command `\rightanglesqr` is defined using `\cs_if_exist:NTF`; if it is true (the user has a font loaded that provides the command) we do not load our fallback; if it is false, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr::`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

93 \cs_new_protected:Nn \__spintent_right_angle_sqr:
94 {
95   \cs_if_exist:NTF \rightanglesqr
96   { \rightanglesqr }
97   {
98     \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
99     { \__spintent__luafun_right_angle_sqr: }
100    {
101      \msg_warning:nn { spintent } { right-angle-sqr-fallback }
102      \rightangle
103    }
104  }
105 }

```

(End of definition for `\@@_right_angle_sqr:`.)

11.8 Internal copy of the unicode invisible separator

`\@@_invisible_sep:` The function `\__spintent_invisible_sep:` is an internal copy of the Unicode invisible separator U-2063 that uses `\luamml_annotate:en` from the package `luamml`[2] and let `<mo>...</mo>` in the MathML structure embedded in the PDF output.

```

106 \cs_new_protected:Nn \__spintent_invisible_sep:
107 {
108   \luamml_annotate:en
109   {
110     core = {[0] = 'mo', intent=':silent', "^^^2063"}
111   }
112   {
113     \latelua{}
114   }
115 }

```

The token list constant `\c__spintent_invisible_sep_tl` is an internal copy of the Unicode invisible separator U-2063 using the primitive `\Umathchardef` from LuaTeX which leaves `<mi>...</mi>` in the structure MathML embedded in the output PDF.

```

116 \hook_gput_code:nnn { begindocument } { spintent }
117 {
118   \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
119 }

```

- Both copies are used for different purposes, the function `\__spintent_invisible_sep:` will be used to force the start of a `<mrow>` in the structure MathML, the token list constant `\c__spintent_invisible_sep_tl` will be used to inject text into the structure MathML when we use `\MathMLintent{<text>}`.

(End of definition for `\@@_invisible_sep:` and `\c__spintent_invisible_sep_tl`.)

11.9 Normalize argument and affixes

We normalize the $\{\langle argument \rangle\}$ to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\@@_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

120 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
121 {
122   \tl_if_in:NnT #1 { -- }
123   {
124     \tl_replace_all:Nnn #1 { -- } { - }
125     \__spintent_collapse_hyphens:N #1
126   }
127 }
```

(End of definition for `\@@_collapse_hyphens:N`.)

`\@@_sanitize_affix:Nn` The function `\__spintent_sanitize_affix:Nn` checks if the $\{\langle argument \rangle\}$ passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘ ’ to hyphens ‘-’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\l__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

128 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
129 {
130   \tl_if_blank:nTF {#2}
131   { \tl_clear:N #1 }
132   {
133     \tl_set:Ne #1 {#2}
134     \tl_trim_spaces:N #1
135     \tl_replace_all:Nnn #1 { ~ } { - }
136     \__spintent_collapse_hyphens:N #1
137     \tl_put_left:Nn #1 { \q_mark }
138     \tl_put_right:Nn #1 { \q_stop }
139     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
140     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
141     \tl_remove_all:Nn #1 { \q_mark }
142     \tl_remove_all:Nn #1 { \q_stop }
143   }
144 }
145 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(End of definition for `\@@_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure $\langle keys \rangle$ “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

`\l__@setspintent_input_arg_cmd_tl` The variable `\l__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” $\{\langle argument \rangle\}$ ‘#1’ of the command `\setspintent`.

```
146 \tl_new:N \l__spintent_setspintent_input_arg_cmd_tl
```

Error message if the $\{\langle argument \rangle\}$ ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

147 \msg_new:nnnn { spintent } { unknown-command-setup }
148 {
149   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
150 }
151 {
152   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
153   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
154 }
```

(End of definition for `\l__@setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```
\@@_setspintent_set_keys:nn
\@@_setspintent_set_keys:en
```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as `{⟨argument⟩}` exists in ‘#1’ and set the `⟨keys⟩` passed as `{⟨argument⟩}` in ‘#2’ to the correct path; otherwise, it will return an error.

```
155 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
156 {
157   \cs_if_exist:cTF { #1 }
158   {
159     \keys_set:nn { spintent / #1 } { #2 }
160   }
161   {
162     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
163   }
164 }
165 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }
```

(End of definition for `\@@_setspintent_set_keys:nn`.)

```
\@@_setspintent_parse_args:nn
\@@_setspintent_parse_args:Vn
```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```
166 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
167 {
168   \tl_if_single:nTF { #1 }
169   {
170     \token_if_cs:NTF #1
171     {
172       \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
173     }
174     {
175       \__spintent_setspintent_set_keys:nn { #1 } { #2 }
176     }
177   }
178   {
179     \__spintent_setspintent_set_keys:nn { #1 } { #2 }
180   }
181 }
182 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }
```

(End of definition for `\@@_setspintent_parse_args:nn`.)

`\setspintent`

Now we define the user command `\setspintent`.

```
183 \NewDocumentCommand \setspintent { m m }
184 {
185   \tl_if_blank:nTF { #1 }
186   {
187     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
188   }
189   {
190     \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
191     \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
192     \__spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
193   }
194 }
```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.11 Handling unknown keys

```
\l_@@_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown
```

We define the variable `\l__spintent_command_name_str` which will *store* the “name” of each command within the implementation along with the messages `unknown-choice` used by `⟨keys⟩` that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the `⟨key⟩` `unknown` in commands that use `[⟨key = val⟩]`.

```
195 \str_new:N \l__spintent_command_name_str
196 \msg_new:nnn { spintent } { unknown-choice }
197 {
198   The~value~'!#3'~for~'!#1'~key~is~invalid~use~('!#2').
199 }
200 \msg_new:nnnn { spintent } { cmd-key-unknown }
201 {
202   The~key~'!#1'~is~unknown~by~command~
```

```

203     \c_backslash_str \l__spintent_command_name_str \c_space_tl
204     and~is~being~ignored~\msg_line_context:.
205 }
206 {
207     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
208     does~not~have~a~key~called~'#1'.\\
209     Check~that~you~have~spelled~the~key~name~correctly.
210 }
211 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
212 {
213     The~key~'#1=#2'~is~unknown~by~command~
214     \c_backslash_str \l__spintent_command_name_str \c_space_tl
215     and~is~being~ignored~\msg_line_context:.
216 }
217 {
218     The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
219     does~not~have~a~key~called~'#1'.\\
220     Check~that~you~have~spelled~the~key~name~correctly.
221 }

```

(End of definition for \l__@_command_name_str and others.)

\@@_unknown_keys:n The internal functions __spintent_unknown_keys:n and __spintent_unknown_keys:nn used by the
 \@@_unknown_keys:nn *<key>* unknown.

```

222 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
223 {
224     \exp_args:NV \__spintent_unknown_keys_cmd:nn \l_keys_key_str {#1}
225 }
226 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
227 {
228     \tl_if_blank:nTF {#2}
229     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
230     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
231 }

```

(End of definition for \@@_unknown_keys:n and \@@_unknown_keys:nn.)

11.12 Definition of core base variables

\l__luafun_clean_split_and_set:n The function __spintent_luafun_clean_split_and_set:n defined in the Lua module `spintent.lua`,
 \l__luafun_clean_split_and_set:V is common to several commands in this implementation. It analyzes, processes, cleans and splits the *<argument>* passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for \l__luafun_clean_split_and_set:n.)

\l__@_luaset_sign_tl The variable \l__spintent_luaset_sign_tl stores the explicit ‘+’ or ‘-’ sign if present. The variable
 \l__@_luaset_part_int_tl \l__spintent_luaset_part_int_tl stores the raw digit sequence of the “integer part”. The variable
 \l__@_luaset_dec_sep_tl \l__spintent_luaset_dec_sep_tl stores the matched ‘,’ or ‘.’ used as the decimal separator. The
 \l__@_luaset_part_dec_tl variable \l__spintent_luaset_part_dec_tl stores the raw digits of the “finite decimal part” that follow
 \l__@_luaset_part_period_tl the separator, and the variable \l__spintent_luaset_part_period_tl stores the digits of the “infinite
 \l__@_luaset_has_integer_str periodic decimal part” that follow a semicolon ‘;’.

The variable \l__spintent_luaset_has_integer_str store the state `true` when it is *only* an “integer”.
 The variable \l__spintent_luaset_has_natural_str store the state `true` when it is *only* an “natural number”.

```

232 \tl_new:N \l__spintent_luaset_sign_tl
233 \tl_new:N \l__spintent_luaset_part_int_tl
234 \tl_new:N \l__spintent_luaset_dec_sep_tl
235 \tl_new:N \l__spintent_luaset_part_dec_tl
236 \tl_new:N \l__spintent_luaset_part_period_tl
237 \str_new:N \l__spintent_luaset_has_integer_str
238 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for \l__@_luaset_sign_tl and others.)


```

\l_@@_luaset_only_part_int_str
\l_@@_luaset_only_part_dec_str
\l_@@_luaset_only_part_period_str
\l_@@_luaset_format_part_int_str
\l_@@_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are *stored* in `\l__spintent_luaset_only_part_int_str` for the “*integer part*”, in `\l__spintent_luaset_only_part_dec_str` for the “*finite decimal part*”, and in `\l__spintent_luaset_only_part_period_str` for the “*infinite periodic decimal part*”.

Formatted versions (`cifras-sep=true`) are *stored* in `\l__spintent_luaset_format_part_int_str` for the “*integer part*” and `\l__spintent_luaset_format_part_dec_str` for the “*finite decimal part*”.

```

239 \str_new:N \l__spintent_luaset_only_part_int_str
240 \str_new:N \l__spintent_luaset_only_part_dec_str
241 \str_new:N \l__spintent_luaset_only_part_period_str
242 \str_new:N \l__spintent_luaset_format_part_int_str
243 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l_@@_luaset_only_part_int_str` and others.)

```

\l_@@_luaset_millions_str
\l_@@_luaset_decimal_period_str
\l_@@_luaset_not_decimal_period_str
\l_@@_luaset_not_decimal_not_period_str
\l_@@_luaset_dec_is_all_zeros_str
\l_@@_luaset_has_exponent_str
\l_@@_luaset_exponent_tl

```

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “*finite decimal part*” is completely zero (useful for “*pure periodic decimals*”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “*scientific notation*” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

244 \str_new:N \l__spintent_luaset_millions_str
245 \str_new:N \l__spintent_luaset_decimal_period_str
246 \str_new:N \l__spintent_luaset_not_decimal_period_str
247 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
248 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
249 \str_new:N \l__spintent_luaset_has_exponent_str
250 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\l_@@_luaset_millions_str` and others.)

```

\l_@@_luaset_has_units_str
\l_@@_luaset_status_str
\l_@@_luaset_arg_above_tl
\l_@@_luaset_arg_below_tl
\l_@@_luaset_denom_has_number_str

```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “*units*”. It sets the flag variable `\l__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\l__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\l__spintent_luaset_arg_above_tl` and `\l__spintent_luaset_arg_below_tl`. Finally, it analyzes `\l__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```

251 \str_new:N \l__spintent_luaset_has_units_str
252 \str_new:N \l__spintent_luaset_status_str
253 \tl_new:N \l__spintent_luaset_arg_above_tl
254 \tl_new:N \l__spintent_luaset_arg_below_tl
255 \str_new:N \l__spintent_luaset_denom_has_number_str

```

(End of definition for `\l_@@_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```

256 \msg_new:nnnn { spintent } { need-math-mode }
257 {
258   Math~mode~required~for~'#1'~\msg_line_context:.
259 }
260 {
261   Use~inside~$...$~or~in~math~environment.
262 }
263 \msg_new:nnnn { spintent } { need-text-mode }
264 {
265   Text~mode~required~for~'#1'~\msg_line_context:.
266 }
267 {
268   Use~outside~of~$...$~or~math~environment.
269 }

```

11.14 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for `\spnum`

```
\l_@@_spnum_read_terse_bool
\l_@@_spnum_read_period_sep_str
\l_@@_spnum_notation_sym_tl
\l_@@_spnum_intent_read_sign_str
\l_@@_spnum_intent_read_dec_sep_str
```

The variable `\l_@@_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘-’. If set to `clear`, it will say “*positivo*” or “*negativo*” after reading the number; if set to `terse`, it will say “*más*” or “*menos*” before reading the number.

The variable `\l_@@_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘.’ (periodic part) is printed and spoken when the {*argument*} is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “*coma*” and print ‘,’; if set to `punto` it will say “*punto*” and print ‘.’.

The variable `\l_@@_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘x’ will be printed when set to `times` or ‘·’ when set to `cdot`.

The variable `\l_@@_spnum_intent_read_sign_str` will store the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘-’ according to the value passed to the key `read-sign`.

The variable `\l_@@_spnum_intent_read_dec_sep_str` stores the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “*coma*” or a “*punto*” according to the value found in the variable `\l__spintent_luaseta_dec_sep_tl`.

```
276 \bool_new:N \l__spintent_spnum_read_terse_bool
277 \str_new:N \l__spintent_spnum_read_period_sep_str
278 \tl_new:N \l__spintent_spnum_notation_sym_tl
279 \str_new:N \l__spintent_spnum_intent_read_sign_str
280 \str_new:N \l__spintent_spnum_intent_read_dec_sep_str
```

(End of definition for `\l_@@_spnum_read_terse_bool` and others.)

```
cifras-sep
read-sign
read-period-sep
notation-sym
```

Now we add the keys `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym`.

```
275 \keys_define:nn { spintent/spnum }
276 {
277   cifras-sep .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
278   cifras-sep .initial:n = true,
279   cifras-sep .value_required:n = true,
280   read-sign .choices:nn =
281     { terse , clear }
282     {
283       \int_case:nn { \l_keys_choice_int }
284       {
285         {1} { \bool_set_true:N \l__spintent_spnum_read_terse_bool }
286         {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
287       }
288     },
289   read-sign .initial:n = terse,
290   read-sign .value_required:n = true,
291   read-sign / unknown .code:n =
292     \msg_error:nneee { spintent } { unknown-choice }
293     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
294   read-period-sep .choices:nn =
295     { coma , punto }
296     {
297       \int_case:nn { \l_keys_choice_int }
298       {
299         {1} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { coma } }
300         {2} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { punto } }
301       }
302     },
303   read-period-sep .initial:n = coma,
304   read-period-sep .value_required:n = true,
305   read-period-sep / unknown .code:n =
306     \msg_error:nneee { spintent } { unknown-choice }
307     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
308   notation-sym .choices:nn =
309     { times , cdot }
310     {
```

```

311     \int_case:n { \l_keys_choice_int }
312     {
313         {1} { \tl_set:Nn \l__spintrent_snum_notation_sym_tl { \times } }
314         {2} { \tl_set:Nn \l__spintrent_snum_notation_sym_tl { \cdot } }
315     }
316 },
317 notation-sym .initial:n = cdot,
318 notation-sym .value_required:n = true,
319 notation-sym / unknown .code:n =
320     \msg_error:nneee { spintrent } { unknown-choice }
321     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
322 unknown .code:n =
323     {
324         \str_set:Nn \l__spintrent_command_name_str { snum }
325         \__spintrent_unknown_keys_cmd:n {#1}
326     },
327 }

```

(End of definition for *cifras-sep* and others.)

11.14.2 Internal functions for \snum

`\@@_snum_render_part:n` The function `\__spintrent_snum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintrent_snum_render_int:` and `\__spintrent_snum_render_dec:` to define the *intent function* `:number` for the integer and decimal part passed to `\MathMLIntent`. It evaluates whether the key *cifras-sep* is active or not.

```

328 \cs_new_protected:Nn \__spintrent_snum_render_part:n
329 {
330     \MathMLIntent { \str_use:c { l__spintrent_luaset_only_part_#1_str } :number }
331     {
332         \bool_if:NTF \l__spintrent_snum_cifras_sep_bool
333         { \str_use:c { l__spintrent_luaset_format_part_#1_str } }
334         { \tl_use:c { l__spintrent_luaset_part_#1_tl } }
335     }
336 }
337 \cs_new_protected:Nn \__spintrent_snum_render_int:
338 {
339     \__spintrent_snum_render_part:n { int }
340 }
341 \cs_new_protected:Nn \__spintrent_snum_render_dec:
342 {
343     \__spintrent_snum_render_part:n { dec }
344 }

```

(End of definition for `\@@_snum_render_part:n`, `\@@_snum_render_int:`, and `\@@_snum_render_dec:`.)

`\@@_snum_render_only_period:` The function `\__spintrent_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* `:number` passed to `\MathMLIntent` which will be used by `\MathMLarg` in the function `\__spintrent_snum_print_period_bar:`.

```

345 \cs_new_protected:Nn \__spintrent_snum_render_only_period:
346 {
347     \MathMLIntent { \l__spintrent_luaset_only_part_period_str :number }
348     {
349         \l__spintrent_luaset_part_period_tl
350     }
351 }

```

The function `\__spintrent_snum_print_period_bar:` inserts the function `\__spintrent_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLIntent`, generating the correct spoken for the “infinite periodic decimal part”.

```

352 \cs_new_protected:Nn \__spintrent_snum_print_period_bar:
353 {
354     \MathMLIntent { _periódico:postfix($dp) }
355     {
356         {
357             \overline
358             {
359                 \MathMLarg { dp } { \__spintrent_snum_render_only_period: }
360             }
361         }
362     }
363 }

```

The function `\__spintent_spnum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “*pure periodic decimal*” is passed.

```

364 \cs_new_protected:Nn \__spintent_spnum_render_period_sep:
365 {
366   \str_if_eq:VnTF \__spintent_spnum_read_period_sep_str { coma }
367   {
368     \MathMLintent { _coma:prefix($pp) }
369     {
370       \mathord{,}
371       \MathMLarg { pp } { \__spintent_spnum_print_period_bar: }
372     }
373   }
374   {
375     \MathMLintent { _punto:prefix($pp) }
376     {
377       \mathord{.}
378       \MathMLarg { pp } { \__spintent_spnum_print_period_bar: }
379     }
380   }
381 }

```

(End of definition for `\@@_spnum_render_only_period:`, `\@@_spnum_print_period_bar:`, and `\@@_spnum_render_period_sep:`.)

`\@@_spnum_render_decimal_sep:`

The function `\__spintent_spnum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_dec_sep_tl` and sets the variable `\l_@@_spnum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “*coma*” or `_punto:prefix($dp)` for reading a “*punto*” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_spnum_render_dec:` to print the finite decimal part.

```

382 \cs_new_protected:Nn \__spintent_spnum_render_decimal_sep:
383 {
384   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
385   { \str_set:Nn \l__spintent_spnum_intent_read_dec_sep_str { _coma } }
386   { \str_set:Nn \l__spintent_spnum_intent_read_dec_sep_str { _punto } }
387   \MathMLintent { \l__spintent_spnum_intent_read_dec_sep_str :prefix($dp) }
388   {
389     \mathord { \l__spintent_luaset_dec_sep_tl }
390     \MathMLarg { dp }
391     {
392       \__spintent_invisible_sep:
393       \__spintent_spnum_render_dec:
394     }
395   }
396   % period part
397   \str_if_eq:VnTF \l__spintent_luaset_decimal_period_str { true }
398   {
399     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
400     {
401       \__spintent_invisible_sep:
402       \__spintent_spnum_print_period_bar:
403     }
404     {
405       \MathMLintent { _con:prefix($pnum) }
406       {
407         \__spintent_invisible_sep:
408         \MathMLarg { pnum }
409         {
410           \__spintent_spnum_print_period_bar:
411         }
412       }
413     }
414   }
415 }

```

(End of definition for `\@@_spnum_render_decimal_sep:`.)

`\@@_spnum_print_part_dec:`

The function `\__spintent_spnum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

`\@@_spnum_print_part_period:`

The function `\__spinttent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

416 \cs_new_protected:Nn \__spinttent_snum_print_part_dec:
417 {
418   \tl_if_empty:NF \__spinttent_luaset_part_dec_tl
419   {
420     \__spinttent_snum_render_decimal_sep:
421   }
422 }
423 \cs_new_protected:Nn \__spinttent_snum_print_part_period:
424 {
425   \tl_if_empty:NF \__spinttent_luaset_part_period_tl
426   {
427     \str_if_eq:VnTF \__spinttent_luaset_not_decimal_period_str { true }
428     { \__spinttent_snum_render_period_sep: }
429     {
430       \tl_if_empty:NT \__spinttent_luaset_part_dec_tl
431       {
432         \__spinttent_snum_render_period_sep:
433       }
434     }
435   }
436 }

```

(End of definition for `\@@_snum_print_part_dec:` and `\@@_snum_print_part_period:`.)

`\@@_snum_print_integer:`
`\@@_snum_print_num_start:`
`\@@_snum_print_number_core:`

The function `\__spinttent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spinttent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spinttent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

 Note for `\__spinttent_invisible_sep:`.

```

437 \cs_new_protected:Nn \__spinttent_snum_print_integer:
438 {
439   \str_if_eq:VnTF \__spinttent_luaset_not_decimal_not_period_str { true }
440   { \__spinttent_snum_render_int: }
441   {
442     \MathMLintent { \__spinttent_luaset_only_part_int_str :number:prefix($next) }
443     {
444       \bool_if:NTF \__spinttent_snum_cifras_sep_bool
445       { \__spinttent_luaset_format_part_int_str }
446       { \__spinttent_luaset_part_int_tl }
447       \MathMLarg { next }
448       {
449         \tl_if_empty:NTF \__spinttent_luaset_part_dec_tl
450         { \__spinttent_snum_print_part_period: }
451         { \__spinttent_snum_print_part_dec: }
452       }
453     }
454   }
455 }
456 \cs_new_protected:Nn \__spinttent_snum_print_num_start:
457 {
458   \tl_if_empty:NTF \__spinttent_luaset_sign_tl
459   {
460     \__spinttent_invisible_sep: % need
461     \__spinttent_snum_print_integer:
462   }
463   {
464     \bool_if:NTF \__spinttent_snum_read_terse_bool
465     {
466       \str_if_eq:VnTF \__spinttent_luaset_sign_tl { + }
467       { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _más:prefix($n) } }
468       { \str_set:Nn \__spinttent_snum_intent_read_sign_str { _menos:prefix($n) } }

```

```

469     }
470     {
471         \str_if_eq:VnTF \l__spintenc_luaset_sign_tl { + }
472         { \str_set:Nn \l__spintenc_spnum_intent_read_sign_str { _positivo:postfix($n) } }
473         { \str_set:Nn \l__spintenc_spnum_intent_read_sign_str { _negativo:postfix($n) } }
474     }
475     \l__spintenc_invisible_sep: % need
476     \MathMLintent { \l__spintenc_spnum_intent_read_sign_str }
477     {
478         \l__spintenc_luaset_sign_tl
479         \MathMLarg { n } { \l__spintenc_spnum_print_integer: }
480     }
481 }
482 }
483 \cs_new_protected:Nn \l__spintenc_spnum_print_number_core:
484 {
485     \l__spintenc_spnum_print_num_start:
486     \str_if_eq:VnTF \l__spintenc_luaset_has_exponent_str { true }
487     {
488         \MathMLintent { _por } { \l__spintenc_spnum_notation_sym_tl }
489         10^{\l__spintenc_luaset_exponent_tl }
490     }
491 }

```

(End of definition for `\@@_spnum_print_integer:`, `\@@_spnum_print_num_start:`, and `\@@_spnum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle\textit{argument}\rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle\textit{argument}\rangle$ contains no numerical digits.

```

492 \msg_new:nnnn { spintenc } { spnum-has-units }
493 {
494     The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~(''#1'').
495 }
496 {
497     Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
498 }
499 \msg_new:nnnn { spintenc } { spnum-no-digits }
500 {
501     The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
502 }
503 {
504     You~provided~''#1',~which~contains~no~digits.
505 }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.14.4 Implementation of the `\spnum` command

`\spnum` The public user command `\spnum` accepts an optional $[\langle\textit{key} = \textit{val}\rangle]$ argument in ‘#1’ and a mandatory $\langle\textit{rational number}\rangle$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\l__spintenc_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\l__spintenc_spnum_print_number_core:` to render the final number, and finally closes the group.

```

506 \NewDocumentCommand \spnum { O{} m }
507 {
508     \mode_if_math:TF
509     {
510         \group_begin:
511         \keys_set:nn { spintenc/spnum } { #1 }
512         \l__spintenc_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
513         \str_if_eq:VnTF \l__spintenc_luaset_has_units_str { true }
514         { \msg_error:nnn { spintenc } { spnum-has-units } { #2 } }
515         {
516             \tl_if_empty:NTF \l__spintenc_luaset_part_int_tl
517             { \msg_error:nnn { spintenc } { spnum-no-digits } { #2 } }
518             { \l__spintenc_spnum_print_number_core: }
519         }
520         \group_end:
521     }

```



```

522     { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
523   }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\langle argument \rangle$ to Lua module `spintent.lua` via `__spintent_luafun_clean_split_and_set:n` to separate and clean the $\langle argument \rangle$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

524 \str_new:N \l__spintent_spunit_luaset_canonical_str
525 \str_new:N \l__spintent_spunit_luaset_lookup_status_str
526 \str_new:N \l__spintent_spunit_luaset_read_str
527 \str_new:N \l__spintent_spunit_luaset_is_compact_str
528 \str_new:N \l__spintent_spunit_luaset_compact_exp_str
529 \str_new:N \l__spintent_spunit_luaset_is_sexagesimal_str
530 \str_new:N \l__spintent_spunit_luaset_status_str

```

(End of definition for `\l__spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\l__spintent_spunit_mult_seq` and `\l__spintent_spunit_exp_seq` split compound “units” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

531 \tl_new:N \l__spintent_spunit_exponent_tl
532 \tl_new:N \l__spintent_spunit_unit_name_tl
533 \str_new:N \l__spintent_spunit_read_slash_str
534 \seq_new:N \l__spintent_spunit_exp_seq
535 \seq_new:N \l__spintent_spunit_mult_seq
536 \bool_new:N \l__spintent_spunit_is_mult_bool
537 \bool_new:N \l__spintent_spunit_read_cdod_bool

```

(End of definition for `\l__spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\langle argument \rangle$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

538 \msg_new:nnnn { spintent } { spunit-invalid-unit }
539 {
540   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
541 }
542 {
543   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
544 }
545 \msg_new:nnnn { spintent } { spunit-no-units }
546 {
547   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
548 }
549 {
550   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
551 }
552 \msg_new:nnnn { spintent } { inline-numeric-denominator }
553 {
554   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
555 }
556 {
557   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
558   does~not~allow~numbers~after~the~slash~(/).
559 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.15.3 Definition of keys for \spunit

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

560 \keys_define:nn { spintent/spunit }
561 {
562   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
563   print-cdot      .initial:n   = false,
564   print-cdot      .value_required:n = true,
565   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
566   read-cdot       .initial:n   = false,
567   read-cdot       .value_required:n = true,
568   read-slash      .choices:nn =
569     { por , partido-por }
570   {
571     \int_case:nn { \l_keys_choice_int }
572     {
573       {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
574       {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
575     }
576   },
577   read-slash      .initial:n   = por,
578   read-slash      .value_required:n = true,
579   read-slash / unknown .code:n =
580     \msg_error:nneee { spintent } { unknown-choice }
581     { read-slash } { por, ~partido-por } { \exp_not:n {#1} },
582   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
583   cifras-sep      .value_required:n = true,
584   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
585   read-period-sep .value_required:n = true,
586   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
587   notation-sym    .value_required:n = true,
588   unknown         .code:n =
589     {
590       \str_set:Nn \l__spintent_command_name_str { spunit }
591       \__spintent_unknown_keys_cmd:n {#1}
592     },
593 }

```

(End of definition for `read-cdot` and others.)

11.15.4 Internal functions for \spunit

`\@@_spunit_format_canonical:` The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘*l*’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

The function `\__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent. It first looks up the unit ‘*#1*’ in the dictionary. If the “unit” is not found, it flags it as invalid. If found, it handles the spacing and multiplication dot ‘`\cdot`’ intents, and then renders the base unit and its exponent (either using a pre-defined compact exponent from Lua or by manually splitting the string at the ‘`^`’ symbol).

```

594 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
595 {
596   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
597   {
598     { l } { \ell }
599   }
600   { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
601 }
602 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
603 {
604   \tl_clear:N \l__spintent_spunit_exponent_tl
605   \__spintent_luafun_spunit_lookup_alias:n { #1 }
606   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
607   {
608     \str_set:Nn \l__spintent_luaset_status_str { invalid }
609   }
610   {
611     \bool_if:NF \l__spintent_spunit_is_mult_bool
612     {
613       \bool_if:NTF \l__spintent_spunit_print_cdot_bool
614       {
615         \bool_if:NTF \l__spintent_spunit_read_cdot_bool

```

```

616         { \MathMLintent { por } { \cdot } }
617         { \MathMLintent { :silent } { \cdot } }
618     }
619     {
620         \bool_if:NTF \l__spintent_spunit_read_cdott_bool
621         { \MathMLintent { por } { \, } }
622         { \, }
623     }
624 }
625 \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
626 {
627     {
628         \MathMLintent { \l__spintent_spunit_luaset_read_str }
629         {
630             \__spintent_spunit_format_canonical:
631         }
632     }
633     ^ { \l__spintent_spunit_luaset_compact_exp_str }
634 }
635 {
636     \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
637     \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
638     \seq_if_empty:NF \l__spintent_spunit_exp_seq
639     {
640         \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
641     }
642     \MathMLintent { \l__spintent_spunit_luaset_read_str }
643     {
644         \__spintent_spunit_format_canonical:
645     }
646     \tl_if_empty:NF \l__spintent_spunit_exponent_tl
647     {
648         ^ { \l__spintent_spunit_exponent_tl }
649     }
650 }
651 \bool_set_false:N \l__spintent_spunit_is_mult_bool
652 }
653 }

```

(End of definition for \@@_spunit_format_canonical: and \@@_spunit_process_base_exp:n.)

\@@_spunit_process_mult:n
\@@_spunit_process_mult:V

The function `\__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the `{\argument}` at each “star symbol” and applies `\__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

654 \cs_new_protected:Nn \__spintent_spunit_process_mult:n
655 {
656     \bool_set_true:N \l__spintent_spunit_is_mult_bool
657     \seq_set_split:Nnn \l__spintent_spunit_mult_seq { * } { #1 }
658     \seq_map_function:NN \l__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
659 }
660 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for \@@_spunit_process_mult:n.)

\@@_print_spunit_hierarchy:

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

661 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
662 {
663     \tl_if_empty:NF \l__spintent_luaset_arg_above_tl
664     {
665         \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_above_tl
666         \tl_if_empty:NF \l__spintent_luaset_arg_below_tl
667         {
668             \MathMLintent { \l__spintent_spunit_read_slash_str } { / }
669             \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_below_tl
670         }
671     }
672 }

```

(End of definition for \@@_print_spunit_hierarchy:.)

`\@@_spunit_safe_exec:n` The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the `{\langle argument \rangle}` has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “*without any space in between*”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

673 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
674 {
675   \str_if_eq:VnTF \l__spintent_luaset_has_units_str { false }
676   { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
677   {
678     \str_if_eq:VnTF \l__spintent_luaset_status_str { valid }
679     {
680       \str_if_eq:VnTF
681       \l__spintent_luaset_denom_has_number_str { true }
682       {
683         \msg_error:nnn { spintent }
684         { inline-numeric-denominator } { #1 }
685       }
686       {
687         \__spintent_luafun_spunit_lookup_alias:V \l__spintent_luaset_arg_above_tl
688         \str_if_eq:VnTF \l__spintent_spunit_luaset_is_sexagesimal_str { true }
689         {
690           \tl_if_empty:NF \l__spintent_luaset_part_int_tl
691           {
692             \__spintent_spnum_print_number_core:
693           }
694           \MathMLintent { \l__spintent_spunit_luaset_read_str }
695           {
696             \mathord
697             {
698               \str_case:Vn \l__spintent_spunit_luaset_canonical_str
699               {
700                 { " } { \prime \! \prime }
701                 { ' } { \prime }
702                 { ° } { ° }
703               }
704             }
705           }
706         }
707         {
708           \tl_if_empty:NF \l__spintent_luaset_part_int_tl
709           {
710             \__spintent_spnum_print_number_core: \,
711           }
712           \__spintent_print_spunit_hierarchy:
713         }
714       }
715     }
716     { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
717   }
718 }

```

(End of definition for `\@@_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional `[\langle key = val \rangle]` argument in ‘#1’ and a mandatory `{\langle unit expression \rangle}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{\langle argument \rangle}` to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

719 \NewDocumentCommand \spunit { O{} m }
720 {
721   \mode_if_math:TF
722   {
723     \group_begin:
724     \keys_set:nn { spintent/spunit } { #1 }
725     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }

```

```

726         \__spintent_spunit_safe_exec:n { #2 }
727     \group_end:
728 }
729 { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
730 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit:nn`. During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintent_spunit_luaset_status_str` variable. If a collision is detected or the $\langle argument \rangle$ is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

731 \msg_new:nnnn { spintent } { spnewunit-duplicate }
732 {
733     The~unit~symbol~or~alias~'#1'~is~already~defined!
734 }
735 {
736     You~cannot~redefine~with~\token_to_str:N \spnewunit.
737 }
738 \msg_new:nnnn { spintent } { spunitalias-duplicate }
739 {
740     The~alias~'#1'~is~already~defined~or~reserved!
741 }
742 {
743     You~cannot~redefine~an~existing~unit~or~another~alias.
744 }
745 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
746 {
747     The~expression~'#2'~for~alias~'#1'~is~invalid.
748 }
749 {
750     It~must~contain~only~defined~units,~exponents~and~one~'/'~.
751 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

752 \NewDocumentCommand \spnewunit { m m }
753 {
754     \mode_if_math:TF
755     {
756         \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
757     }
758     {
759         \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
760         \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
761         {
762             \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
763         }
764     }
765 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias`

The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

766 \NewDocumentCommand \spunitalias { m m }

```

```

767 {
768   \mode_if_math:TF
769   {
770     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
771   }
772   {
773     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
774     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
775     {
776       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
777     }
778     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
779     {
780       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
781     }
782   }
783 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

784 \str_new:N \l__spintent_spmoney_luaset_symbol_str
785 \str_new:N \l__spintent_spmoney_luaset_position_str
786 \str_new:N \l__spintent_spmoney_luaset_sing_str
787 \str_new:N \l__spintent_spmoney_luaset_plur_str
788 \str_new:N \l__spintent_spmoney_luaset_conde_str
789 \str_new:N \l__spintent_spmoney_luaset_subsing_str
790 \str_new:N \l__spintent_spmoney_luaset_subplur_str
791 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
792 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
793 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

794 \bool_new:N \l__spintent_spmoney_sub_units_bool
795 \str_new:N \l__spintent_spmoney_intent_money_str
796 \str_new:N \l__spintent_spmoney_intent_sign_str
797 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
798 \str_new:N \l__spintent_spmoney_res_main_voice_str
799 \int_new:N \l__spintent_spmoney_subnum_int
800 \str_new:N \l__spintent_spmoney_extracted_curr_str
801 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

802 \msg_new:nnnn { spintent } { currency-no-subunits }
803 {
804   The~currency~'#1'~does~not~support~subunits.~
805   The~option~'sub-units=true'~is~ignored.
806 }

```



```

807 {
808     Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
809     do~not~have~fractional~subunits~registered~in~Lua~module.~
810 }
811 \msg_new:nnnn { spintent } { invalid-currency }
812 {
813     Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
814 }
815 {
816     The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
817     the~Lua~module.
818 }

```

(End of definition for currency-no-subunits and invalid-currency.)

11.17.2 Auxiliary functions for \spmoney

\@@_spmoney_set_currency:n The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

819 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
820 {
821     \__spintent_spmoney_normalize_key:n {#1}
822     \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
823     {
824         \msg_error:nnn { spintent } { invalid-currency } {#1}
825     }
826     { \str_set:Nn \__spintent_spmoney_luaset_currency_arg_str {#1} }
827 }
828 \cs_new_protected:Nn \__spintent_spmoney_normalize_key:n
829 {
830     \__spintent_luafun_spmoney_normalize_key:n { #1 }
831 }
832 \cs_generate_variant:Nn \__spintent_spmoney_normalize_key:n { V }

```

(End of definition for \@@_spmoney_set_currency:n and \@@_spmoney_normalize_key:n.)

11.17.3 Definition of keys for \spmoney

currency Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

833 \keys_define:nn { spintent / spmoney }
834 {
835     currency .code:n = { \__spintent_spmoney_set_currency:n {#1} },
836     currency .initial:n = { pesos },
837     currency .value_required:n = true,
838     sub-units .bool_set:N = \__spintent_spmoney_sub_units_bool,
839     sub-units .value_required:n = true,
840     dolar-math .bool_set:N = \__spintent_spmoney_dolar_math_bool,
841     dolar-math .initial:n = { true },
842     dolar-math .value_required:n = true,
843     cifras-sep .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
844     cifras-sep .value_required:n = true,
845     unknown .code:n =
846     {
847         \str_set:Nn \__spintent_command_name_str { spmoney }
848         \__spintent_unknown_keys_cmd:n {#1}
849     },
850 }

```

(End of definition for currency and others.)

11.17.4 Internal functions for \spmoney

\@@_spmoney_validate_money: The function `\__spintent_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\__spintent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\__spintent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\__spintent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintent.lua` via `\__spintent_luafun_spmoney_lookup_data:V`.

```

851 \cs_new_protected:Nn \__spintent_spmoney_validate_money:
852 {
853     \str_if_eq:VnT \__spintent_luaset_has_units_str { true }
854     {

```

```

855     \__spintent_spmoney_normalize_key:V \__spintent_luaset_arg_above_tl
856     \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
857     { \msg_error:nne { spintent } { invalid-currency } { \__spintent_luaset_arg_above_tl } }
858     {
859         \str_set:NV
860         \__spintent_spmoney_luaset_currency_arg_str
861         \__spintent_luaset_arg_above_tl
862     }
863 }
864 \str_set:Nc \__spintent_spmoney_luaset_status_str { found }
865 \__spintent_luafun_spmoney_lookup_data:V \__spintent_spmoney_luaset_currency_arg_str
866 }

```

(End of definition for \@@_spmoney_validate_money:.)

\@@_spmoney_print_symbol:

The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\__spintent_spmoney_luaset_print_iso_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\__spintent_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

867 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
868 {
869     \str_if_eq:VnTF \__spintent_spmoney_luaset_print_iso_str { true }
870     { \mathrm { \__spintent_spmoney_luaset_currency_arg_str } }
871     {
872         \str_if_eq:VnTF \__spintent_spmoney_luaset_symbol_str { $ }
873         {
874             \bool_if:NTF \__spintent_spmoney_dolar_math_bool
875             { \$ } { \text { \$ } }
876         }
877         { \__spintent_spmoney_luaset_symbol_str }
878     }
879 }

```

(End of definition for \@@_spmoney_print_symbol:.)

\@@_spmoney_render_layout:

The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

880 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
881 {
882     \str_if_eq:VnTF \__spintent_spmoney_luaset_position_str { pre }
883     {
884         \__spintent_spmoney_print_symbol:
885         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
886     }
887     {
888         \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
889         \__spintent_spmoney_print_symbol:
890     }
891 }

```

(End of definition for \@@_spmoney_render_layout:.)

\@@_spmoney_render_number_node:

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

\@@_spmoney_render_only_integer_node:

```

892 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
893 {
894     \__spintent_spmoney_render_int:
895     \__spintent_spmoney_print_part_dec:
896 }
897 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
898 {
899     \__spintent_spmoney_render_int:
900 }

```

(End of definition for \@@_spmoney_render_number_node: and \@@_spmoney_render_only_integer_node:.)

`\@@_spmoney_print_int:` The function `\__spintrent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintrent_luaset`
`\@@_spmoney_intent_int:` If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent

layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

The function `\__spintrent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintrent_spmoney_render_layout:`.

```

901 \cs_new_protected:Nn \__spintrent_spmoney_print_int:
902 {
903   \tl_if_empty:NTF \l__spintrent_luaset_sign_tl
904   {
905     \bool_if:NTF \l__spintrent_spmoney_sub_units_bool
906     { \__spintrent_spmoney_set_and_print_sub_units: }
907     { \__spintrent_spmoney_intent_int: }
908   }
909   {
910     \str_if_eq:VnTF \l__spintrent_luaset_sign_tl { + }
911     {
912       \str_set:Nn \l__spintrent_spmoney_intent_sign_str { _más:prefix ($n) }
913     }
914     {
915       \str_set:Nn \l__spintrent_spmoney_intent_sign_str { _menos:prefix ($n) }
916     }
917     \__spintrent_invisible_sep:
918     \MathMLintent { \l__spintrent_spmoney_intent_sign_str }
919     {
920       \l__spintrent_luaset_sign_tl
921       \MathMLarg { n }
922       {
923         \bool_if:NTF \l__spintrent_spmoney_sub_units_bool
924         { \__spintrent_spmoney_set_and_print_sub_units: }
925         { \__spintrent_spmoney_intent_int: }
926       }
927     }
928   }
929 }
930 \cs_new_protected:Nn \__spintrent_spmoney_intent_int:
931 {
932   \tl_if_empty:NTF \l__spintrent_luaset_dec_sep_tl
933   {
934     \str_if_eq:VnTF \l__spintrent_luaset_only_part_int_str { 1 }
935     {
936       \str_set:NV \l__spintrent_spmoney_intent_money_str \l__spintrent_spmoney_luaset_sing_str
937     }
938     {
939       \str_set:NV \l__spintrent_spmoney_intent_money_str \l__spintrent_spmoney_luaset_plur_str
940     }
941     \str_if_eq:VnT \l__spintrent_luaset_millions_str { true }
942     {
943       \str_set:NV \l__spintrent_spmoney_intent_money_str \l__spintrent_spmoney_luaset_conde_s
944     }
945   }
946   {
947     \str_set:NV \l__spintrent_spmoney_intent_money_str \l__spintrent_spmoney_luaset_plur_str
948     \str_if_eq:VnT \l__spintrent_luaset_millions_str { true }
949     {
950       \str_if_eq:VnTF \l__spintrent_luaset_dec_is_all_zeros_str { true }
951       {
952         \str_set:NV \l__spintrent_spmoney_intent_money_str \l__spintrent_spmoney_luaset_conde
953       }
954       {
955         \str_set:NV \l__spintrent_spmoney_intent_money_str \l__spintrent_spmoney_luaset_plur
956       }
957     }
958   }
959   \__spintrent_invisible_sep: % need for MathCAT
960   \MathMLintent { \l__spintrent_spmoney_intent_money_str :postfix($n) }
961   {
962     \__spintrent_spmoney_render_layout:

```

```

963     }
964 }

```

(End of definition for \@@_spmoney_print_int: and \@@_spmoney_intent_int:.)

\@@_spmoney_check_sub_units: The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

965 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
966 {
967   \str_if_empty:NT \__spintent_spmoney_luaset_subsing_str
968   {
969     \msg_warning:nne { spintent } { currency-no-subunits }
970     { \__spintent_spmoney_luaset_currency_arg_str }
971     \bool_set_false:N \__spintent_spmoney_sub_units_bool
972   }
973 }
974 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
975 {
976   \int_compare:nTF { \str_count:N \__spintent_luaset_only_part_dec_str == 1 }
977   {
978     \int_set:Nn \__spintent_spmoney_subnum_int
979     { \__spintent_luaset_only_part_dec_str * 10 }
980     \str_set:Ne \__spintent_luaset_only_part_dec_str
981     { \__spintent_luaset_only_part_dec_str 0 }
982   }
983   {
984     \int_set:Nn \__spintent_spmoney_subnum_int
985     { \__spintent_luaset_only_part_dec_str }
986   }
987   \int_compare:nTF { \__spintent_spmoney_subnum_int == 1 }
988   {
989     \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
990     { \__spintent_spmoney_luaset_subsing_str :postfix($sub) }
991   }
992   {
993     \str_set:Ne \__spintent_spmoney_intent_sub_unit_str
994     { \__spintent_spmoney_luaset_subplur_str :postfix($sub) }
995   }
996   \str_if_eq:VnTF \__spintent_luaset_only_part_int_str { 1 }
997   {
998     \str_set:NV \__spintent_spmoney_res_main_voice_str
999     \__spintent_spmoney_luaset_sing_str
1000   }
1001   {
1002     \str_set:NV \__spintent_spmoney_res_main_voice_str
1003     \__spintent_spmoney_luaset_plur_str
1004   }
1005   \str_if_eq:VnT \__spintent_luaset_millions_str { true }
1006   {
1007     \str_set:NV \__spintent_spmoney_res_main_voice_str
1008     \__spintent_spmoney_luaset_conde_str
1009   }
1010   \__spintent_spmoney_build_tree:
1011 }

```

(End of definition for \@@_spmoney_check_sub_units: and \@@_spmoney_set_and_print_sub_units:.)

\@@_spmoney_build_tree: The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

The function `\__spintent_spmoney_print_integer_continue:` isolates the rendering of the integer node with its associated main “*currency*” intent. Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```

1012 \cs_new_protected:Nn \__spintent_spmoney_build_tree:

```

```

1013 {
1014   \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
1015   {
1016     \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1017     {
1018       \__spintent_spmoney_print_symbol:
1019       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1020     }
1021     \__spintent_spmoney_print_sep_con:
1022     \__spintent_spmoney_print_dec_sub_unit:
1023   }
1024   {
1025     \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1026     {
1027       \__spintent_invisible_sep:
1028       \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1029     }
1030     \__spintent_spmoney_print_sep_con:
1031     \__spintent_spmoney_print_dec_sub_unit:
1032     \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1033   }
1034 }
1035 \cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
1036 {
1037   \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1038   {
1039     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1040   }
1041 }
1042 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1043 {
1044   \MathMLintent { con }
1045   {
1046     \mathord { \l__spintent_luaset_dec_sep_tl }
1047   }
1048 }
1049 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1050 {
1051   \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
1052   {
1053     \__spintent_invisible_sep:
1054     \MathMLarg { sub }
1055     {
1056       \__spintent_spmoney_render_dec:
1057     }
1058   }
1059 }

```

(End of definition for \@@_spmoney_build_tree: and others.)

\@@_spmoney_parse_and_route:n

The internal function `\__spintent_spmoney_parse_and_route:n` processes the `{(argument)}` passed in ‘#1’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1060 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1061 {
1062   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1063   \__spintent_luafun_clean_split_and_set:V \l__spintent_spmoney_extracted_num_tl
1064   \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
1065   {
1066     \str_set:Nc \l__spintent_luaset_has_units_str { true }
1067     \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
1068   }
1069   \__spintent_spmoney_validate_money:
1070   \bool_if:NT \l__spintent_spmoney_sub_units_bool
1071   { \__spintent_spmoney_check_sub_units: }
1072   \__spintent_spmoney_print_int:
1073 }

```

(End of definition for \@@_spmoney_parse_and_route:n)

`\spmoney` The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘#1’ for local key-value configurations and a mandatory argument ‘#2’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1074 \NewDocumentCommand \spmoney { 0{ } m }
1075 {
1076   \mode_if_math:TF
1077   {
1078     \group_begin:
1079     \keys_set:nn { spintent / spmoney } { #1 }
1080     \__spintent_spmoney_parse_and_route:n { #2 }
1081     \group_end:
1082   }
1083   { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1084 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1085 \str_new:N \__spintent_spshort_luaset_status_str
1086 \str_new:N \__spintent_spshort_luaset_layout_str
1087 \str_new:N \__spintent_spshort_luaset_expanded_str
1088 \str_new:N \__spintent_spshort_luaset_output_str
1089 \str_new:N \__spintent_spshort_luaset_base_str
1090 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for `\__spintent_spshort_luaset_status_str` and others.)

11.18.1 Definition of error messages for `\spshort`

unknown-abbreviation The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{<argument>}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1091 \msg_new:nnnn { spintent } { unknown-abbreviation }
1092 { The~abbreviation~or~ordinal~'~#1~'~is~invalid~or~not~recognized. }
1093 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.18.2 Accessibility interface

`\@@_spshort_tag_span:nn` The internal function `\__spintent_spshort_tag_span:nn` processes the `{<argument>}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{<argument>}` passed in ‘#1’ for proper reading with NVDA.

```

1094 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1095 {
1096   \tag_mc_end_push:
1097   \tag_struct_begin:n { tag=Span, E={ #1 } }
1098   \tag_mc_begin:n { tag=Span }
1099   \tag_suspend:n { \spshort }
1100   #2
1101   \tag_resume:n { \spshort }
1102   \tag_mc_end:
1103   \tag_struct_end:
1104   \tag_mc_begin_pop:n {}
1105 }
1106 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\@@_spshort_tag_span:nn`.)

11.18.3 Definition of keys for \spshort

small-caps Now we add the keys `small-caps` and `read-arg`.

```
read-arg 1107 \keys_define:nn { spintent / spshort }
1108 {
1109     small-caps .bool_set:N = \l__spintent_spshort_smallcaps_bool,
1110     small-caps .initial:n = false,
1111     small-caps .value_required:n = true,
1112     read-arg .str_set:N = \l__spintent_spshort_read_arg_str,
1113     read-arg .initial:n = {},
1114     read-arg .value_required:n = true,
1115     unknown .code:n =
1116     {
1117         \str_set:Nn \l__spintent_command_name_str { spshort }
1118         \__spintent_unknown_keys_cmd:n {#1}
1119     },
1120 }
```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for \spshort

\@@_spshort_print:nn The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\l__spintent_spshort_read_arg_str` and evaluates the optional `<keys>` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{<argument>}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```
1121 \cs_new_protected:Nn \__spintent_spshort_print:nn
1122 {
1123     \str_clear:N \l__spintent_spshort_read_arg_str
1124     \keys_set:nn { spintent / spshort } {#1}
1125     \str_if_empty:NTF \l__spintent_spshort_read_arg_str
1126     {
1127         \__spintent_spshort_process_and_render:n {#2}
1128     }
1129     {
1130         \__spintent_spshort_render_bypass:n {#2}
1131     }
1132 }
```

(End of definition for `\@@_spshort_print:nn`.)

\@@_spshort_process_and_render:n The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{<argument>}` passed in ‘#1’.

It subsequently evaluates the execution state stored within the string variable `\l__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```
1133 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1134 {
1135     \__spintent_luafun_spshort_process:n {#1}
1136     \str_case:NnF \l__spintent_spshort_luaset_status_str
1137     {
1138         { success } { \__spintent_spshort_render_success: }
1139         { fallback } { \__spintent_spshort_render_fallback: }
1140         { error } { \__spintent_spshort_render_error:n {#1} }
1141     }
1142     { \__spintent_spshort_render_error:n {#1} }
1143 }
```

(End of definition for `\@@_spshort_process_and_render:n`.)

\@@_spshort_render_success: The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

\@@_spshort_render_fallback: Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “speech” to the `{<argument>}`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `{<argument>}` to prevent fatal compilation.

```
1144 \cs_new_protected:Nn \__spintent_spshort_render_success:
1145 {
```

```

1146     \__spintent_spshort_tag_span:en
1147     { \l__spintent_spshort_luaset_expanded_str }
1148     { \__spintent_spshort_execute_layout: }
1149   }
1150 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1151 {
1152   \__spintent_spshort_tag_span:en
1153   { \l__spintent_spshort_luaset_expanded_str }
1154   {
1155     \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1156     { \textsc{ \l__spintent_spshort_luaset_output_str } }
1157     { \l__spintent_spshort_luaset_output_str }
1158   }
1159 }
1160 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1161 {
1162   \__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
1163 }
1164 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1165 {
1166   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1167 }

```

(End of definition for `\@@_spshort_render_success:` and others.)

`\@@_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\TeX$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1168 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1169 {
1170   \str_case:VnF \l__spintent_spshort_luaset_layout_str
1171   {
1172     { linear_regular }
1173     {
1174       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1175       { \textsc{ \l__spintent_spshort_luaset_output_str } }
1176       { \l__spintent_spshort_luaset_output_str }
1177     }
1178     { linear_caps }
1179     { \l__spintent_spshort_luaset_output_str }
1180     { linear_mixed }
1181     { \l__spintent_spshort_luaset_output_str }
1182     { superscript }
1183     {
1184       \l__spintent_spshort_luaset_base_str
1185       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1186       {
1187         \textsuperscript{
1188           \textsc{
1189             \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
1190           }
1191         }
1192       }
1193       {
1194         \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1195       }
1196     }
1197     { small_caps_pure }
1198     {
1199       \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1200       {
1201         \textsc{
1202           \str_lowercase:V \l__spintent_spshort_luaset_output_str
1203         }
1204       }
1205       { \l__spintent_spshort_luaset_output_str }
1206     }
1207     { acronym_long }
1208     { \l__spintent_spshort_luaset_output_str }

```

```

1209     }
1210     { \l__spintent_spsshort_lua_set_output_str }
1211 }

```

(End of definition for \l__spsshort_execute_layout:.)

\spsshort The public document command `\spsshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\__spintent_spsshort_print:nn` for processing and printing.

```

1212 \NewDocumentCommand \spsshort { 0{} m }
1213 {
1214     \mode_if_math:TF
1215     {
1216         \msg_error:nnn { spintent } { need-text-mode } { \spsshort }
1217     }
1218     {
1219         \mode_leave_vertical:
1220         \group_begin:
1221         \__spintent_spsshort_print:nn {#1} {#2}
1222         \group_end:
1223     }
1224 }

```

(End of definition for \spsshort. This function is documented on page 7.)

11.19 The command \spsiglo

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle argument \rangle$, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible **MathML** intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```

\l__spsiglo_lua_set_error_str
\l__spsiglo_lua_set_arabic_str
\l__spsiglo_lua_set_roman_min_str
\l__spsiglo_print_era_str

```

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

1225 \str_new:N \l__spintent_spsiglo_lua_set_error_str
1226 \str_new:N \l__spintent_spsiglo_lua_set_arabic_str
1227 \str_new:N \l__spintent_spsiglo_lua_set_roman_min_str
1228 \str_new:N \l__spintent_spsiglo_print_era_str

```

(End of definition for \l__spsiglo_lua_set_error_str and others.)

11.19.1 Definition of error messages for \spsiglo

invalid-century-format

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle argument \rangle$ to an integer within the valid range of 1 to 4000, preventing incorrect speech synthesis.

```

1229 \msg_new:nnnn { spintent } { invalid-century-format }
1230 {
1231     Invalid~century~format~in~'#1'.
1232 }
1233 {
1234     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1235     Ensure~the~value~is~between~1~and~4000.
1236 }

```

(End of definition for invalid-century-format.)

11.19.2 Definition of keys for \spsiglo

print-era

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1237 \keys_define:nn { spintent / spsiglo }
1238 {
1239     print-era .choices:nn =
1240     { none , ac , dc }
1241     {
1242         \int_case:nn { \l_keys_choice_int }
1243         {

```

```

1244         {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
1245         {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
1246         {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
1247     }
1248 },
1249 print-era .initial:n = none,
1250 print-era .value_required:n = true,
1251 print-era / unknown .code:n =
1252     \msg_error:nneee { spintent } { unknown-choice }
1253     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1254 unknown .code:n =
1255     {
1256         \str_set:Nn \l__spintent_command_name_str { spsiglo }
1257         \__spintent_unknown_keys_cmd:n {#1}
1258     },
1259 }

```

(End of definition for `print-era`.)

11.19.3 Internal functions for \spsiglo

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1260 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1261 {
1262     \MathMLintent { #1:number } { \text { \textsc {#2} } } }
1263 }
1264 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1265 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1266 {
1267     \str_if_eq:nnTF { #1 } { ac }
1268     {
1269         \MathMLintent { antes-de-cristo:postfix($z) }
1270         {
1271             \__spintent_invisible_sep:
1272             \MathMLarg { z }
1273             {
1274                 \MathMLintent { #2:number }
1275                 {
1276                     \text { \textsc {#3}~a.~C. }
1277                 }
1278             }
1279         }
1280     }
1281     {
1282         \MathMLintent { después-de-cristo:postfix($z) }
1283         {
1284             \__spintent_invisible_sep:
1285             \MathMLarg { z }
1286             {
1287                 \MathMLintent { #2:number }
1288                 {
1289                     \text { \textsc {#3}~d.~C. }
1290                 }
1291             }
1292         }
1293     }
1294 }
1295 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (#1) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1296 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1297 {
1298     \tag_mc_end_push:

```

```

1299     \tag_struct_begin:n { tag=Span, alt={ #1 } }
1300     \tag_mc_begin:n { tag=Span }
1301     \tag_suspend:n { \spsiglo }
1302     #2
1303     \tag_resume:n { \spsiglo }
1304     \tag_mc_end:
1305     \tag_struct_end:
1306     \tag_mc_begin_pop:n {}
1307   }
1308   \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\@@_spsiglo_tag_span:nn`.)

```

\@@_spsiglo_print_math:n
\@@_spsiglo_print_text:n

```

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1309   \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1310   {
1311     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1312     {
1313       \msg_error:nnnn { spintent } { invalid-century-format }
1314       { \spsiglo } { #1 }
1315     }
1316     {
1317       \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1318       {
1319         \__spintent_spsiglo_render_simple:VV
1320         \l__spintent_spsiglo_luaset_arabic_str
1321         \l__spintent_spsiglo_luaset_roman_min_str
1322       }
1323       {
1324         \__spintent_spsiglo_render_with_era:VVV
1325         \l__spintent_spsiglo_print_era_str
1326         \l__spintent_spsiglo_luaset_arabic_str
1327         \l__spintent_spsiglo_luaset_roman_min_str
1328       }
1329     }
1330   }
1331   \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1332   {
1333     \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1334     {
1335       \msg_error:nnnn { spintent } { invalid-century-format }
1336       { \spsiglo } { #1 }
1337     }
1338     {
1339       \str_case:Vn \l__spintent_spsiglo_print_era_str
1340       {
1341         { none }
1342         {
1343           \__spintent_spsiglo_tag_span:en
1344           { \l__spintent_spsiglo_luaset_arabic_str }
1345           { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1346         }
1347         { ac }
1348         {
1349           \__spintent_spsiglo_tag_span:en
1350           { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1351           { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1352         }
1353         { dc }
1354         {
1355           \__spintent_spsiglo_tag_span:en
1356           { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1357           { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1358         }

```

```

1359     }
1360   }
1361 }

```

(End of definition for `\@@_spsiglo_print_math:n` and `\@@_spsiglo_print_text:n`.)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional *(keys)* passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1362 \NewDocumentCommand \spsiglo { 0{} m }
1363 {
1364   \group_begin:
1365     \keys_set:nn { spintent / spsiglo } { #1 }
1366     \__spintent_luafun_spsiglo_parse:n { #2 }
1367     \mode_if_math:TF
1368       { \__spintent_spsiglo_print_math:n { #2 } }
1369       {
1370         \mode_leave_vertical:
1371         \__spintent_spsiglo_print_text:n { #2 }
1372       }
1373   \group_end:
1374 }

```

(End of definition for `\spsiglo`. This function is documented on page 11.)

11.20 The command `\sptime`

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1375 \str_new:N \__spintent_sptime_luaset_seconds_str
1376 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1377 \str_new:N \__spintent_sptime_luaset_error_str
1378 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1379 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for `\__spintent_luaset_seconds_str` and others.)

11.20.1 Definition of error messages

`invalid-time-format`

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1380 \msg_new:nnnn { spintent } { invalid-time-format }
1381 {
1382   Invalid~time~format~in~#1. \\\
1383   The~argument~'~#2'~does~not~match~any~supported~time~format.
1384 }
1385 {
1386   Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1387   Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1388 }

```

(End of definition for `invalid-time-format`.)

11.20.2 Internal functions for `\sptime`

`\@@_sptime_build_seconds:nn`

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1389 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1390 {
1391   \MathMLintent { #1:prefix($z) }
1392   {
1393     \text { : }
1394     \MathMLarg { z }
1395     {
1396       \MathMLintent { segundos:postfix($m) }
1397       {

```



```

1398         \__spintent_invisible_sep:
1399         \MathMLLarg { m }
1400         {
1401             \text { #2 }
1402         }
1403     }
1404 }
1405 }
1406 }

```

(End of definition for \@@_sptime_build_seconds:nn.)

\@@_sptime_build_tree: The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:` with the appropriate prefix ('con' or 'minutos-con'), or simply appends the 'minutos' intent tag if no seconds are present.

```

1407 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1408 {
1409     \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1410     {
1411         \MathMLintent { :time }
1412         {
1413             \text { \l__spintent_sptime_luaset_base_str }
1414         }
1415         \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1416         {
1417             \__spintent_sptime_build_seconds:nn { con }
1418             { \l__spintent_sptime_luaset_seconds_str }
1419         }
1420         {
1421             \__spintent_sptime_build_seconds:nn { minutos-con }
1422             { \l__spintent_sptime_luaset_seconds_str }
1423         }
1424     }
1425     {
1426         \MathMLintent { :time }
1427         {
1428             \text { \l__spintent_sptime_luaset_base_str }
1429         }
1430         \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1431         {
1432             \MathMLintent { minutos }
1433             {
1434                 \__spintent_invisible_sep:
1435             }
1436         }
1437     }
1438 }

```

(End of definition for \@@_sptime_build_tree:.)

`\sptime` The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\@@_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\@@_sptime_build_tree:` to assemble the MathML output.

```

1439 \NewDocumentCommand \sptime { m }
1440 {
1441     \mode_if_math:TF
1442     {
1443         \group_begin:
1444         \__spintent_luafun_sptime_parse:n { #1 }
1445         \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1446         {
1447             \msg_error:nnnn { spintent } { invalid-time-format }
1448             { \sptime } { #1 }
1449         }
1450         {
1451             \__spintent_sptime_build_tree:

```

```

1452     }
1453     \group_end:
1454 }
1455 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1456 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.21 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

```

\l_@@_spdate_luaaset_output_str
\l_@@_spdate_luaaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1457 \str_new:N \l__spintent_spdate_luaaset_output_str
1458 \str_new:N \l__spintent_spdate_luaaset_error_str

```

(End of definition for `\l_@@_spdate_luaaset_output_str` and `\l_@@_spdate_luaaset_error_str`.)

11.21.1 Definition of error messages

```
invalid-date-format
```

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1459 \msg_new:nnnn { spintent } { invalid-date-format }
1460 {
1461   Invalid~date~or~unsupported~format~in~#1. \\\
1462   The~argument~'#2'~does~not~match~a~supported~format.
1463 }
1464 {
1465   Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1466   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1467 }

```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

```
\@@_spdate_process:n
```

The internal function `\__spintent_spdate_process:n` parses the input using `\@@_luafun_spdate_parse:n` and checks the error status in `\l__spintent_spdate_luaaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the MathML `intent` structure using the formatted date stored in `\l__spintent_spdate_luaaset_output_str`.

```

1468 \cs_new_protected:Nn \__spintent_spdate_process:n
1469 {
1470   \__spintent_luafun_spdate_parse:n { #1 }
1471   \str_if_eq:VnTF \l__spintent_spdate_luaaset_error_str { true }
1472   {
1473     \msg_error:nnnn { spintent } { invalid-date-format }
1474     { \spdate } { #1 }
1475   }
1476   {
1477     \MathMLintent { :date }
1478     {
1479       \text { \l__spintent_spdate_luaaset_output_str }
1480     }
1481   }
1482 }

```

(End of definition for `\@@_spdate_process:n`.)

```
\spdate
```

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\@@_spdate_process:n` to process the input.

```

1483 \NewDocumentCommand \spdate { m }
1484 {
1485   \mode_if_math:TF
1486   {
1487     \group_begin:
1488     \__spintent_spdate_process:n { #1 }
1489     \group_end:
1490   }
1491   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1492 }

```

(End of definition for `\spdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two `<keys>` `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank `{<argument>}`, the second this is when an input is invalid.

```
1493 \msg_new:nnnn { spintent } { spusep-empty-arg }
1494 { Empty~argument~in~\token_to_str:N \spusep. }
1495 { \token_to_str:N \spusep~requires~a~non-empty~argument. }
1496 \msg_new:nnn { spintent } { spusep-invalid-arg }
1497 {
1498   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1499 }
```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\l_@@_spusep_arg_tl`
`\l_@@_spusep_left_tl`
`\l_@@_spusep_right_tl`

The variable `\l_@@_spusep_arg_tl` stores the `{<argument>}`, the variable `\l_@@_spusep_left_tl` and `\l_@@_spusep_right_tl` stores the parentheses.

```
1500 \tl_new:N \l__spintent_spusep_arg_tl
1501 \tl_new:N \l__spintent_spusep_left_tl
1502 \tl_new:N \l__spintent_spusep_right_tl
```

(End of definition for `\l_@@_spusep_arg_tl`, `\l_@@_spusep_left_tl`, and `\l_@@_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the `<keys>` `size` and `silent`.

```
1503 \keys_define:nn { spintent / spusep }
1504 {
1505   size .choices:nn =
1506     { none , big , Big, bigg, Bigg }
1507     {
1508       \int_case:nn { \l_keys_choice_int }
1509       {
1510         { 1 }
1511         {
1512           \tl_clear:N \l__spintent_spusep_left_tl
1513           \tl_clear:N \l__spintent_spusep_right_tl
1514         }
1515         { 2 }
1516         {
1517           \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1518           \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1519         }
1520         { 3 }
1521         {
1522           \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1523           \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1524         }
1525         { 4 }
1526         {
1527           \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1528           \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1529         }
1530         { 5 }
1531         {
1532           \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1533           \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1534         }
1535       }
1536     },
1537   size .initial:n = none,
1538   size .value_required:n = true,
1539   size / unknown .code:n =
```

```

1540     \msg_error:nneee { spintent } { unknown-choice }
1541     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1542     silent      .bool_set:N = \l__spintent_spusep_silent_bool,
1543     silent      .initial:n = false,
1544     silent      .value_required:n = true,
1545     unknown     .code:n =
1546     {
1547         \str_set:Nn \l__spintent_command_name_str { spusep }
1548         \__spintent_unknown_keys_cmd:n {#1}
1549     },
1550 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\l__spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\l__spusep_left_tl` and `\l__spusep_right_tl` and then check the state of the variable `\l__spusep_silent_bool` handled by the key `silent` to print.

```

1551 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1552 {
1553     \tl_put_right:cn { l__spintent_spusep_ #1 _tl } { #2 }
1554     \bool_if:NTF \l__spintent_spusep_silent_bool
1555     {
1556         \MathMLintent { :silent } { \tl_use:c { l__spintent_spusep_ #1 _tl } }
1557     }
1558     {
1559         \tl_use:c { l__spintent_spusep_ #1 _tl }
1560     }
1561 }

```

(End of definition for `\l__spusep_set_parens:nn`.)

`\l__spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\l__spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1562 \cs_new_protected:Nn \__spintent_spusep_convert:n
1563 {
1564     \tl_set:Ne \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1565     \tl_if_empty:NT \l__spintent_spusep_arg_tl
1566     {
1567         \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1568     }
1569     \str_case:VnF \l__spintent_spusep_arg_tl
1570     {
1571         { ( }
1572         {
1573             \__spintent_spusep_set_parens:nn { left } { \lparen }
1574         }
1575         { ) }
1576         {
1577             \__spintent_spusep_set_parens:nn { right } { \rparen }
1578         }
1579         { [ }
1580         {
1581             \__spintent_spusep_set_parens:nn { left } { \lbrack }
1582         }
1583         { ] }
1584         {
1585             \__spintent_spusep_set_parens:nn { right } { \rbrack }
1586         }
1587         { \{ }
1588         {
1589             \__spintent_spusep_set_parens:nn { left } { \lbrace }
1590         }
1591         { \} }
1592         {
1593             \__spintent_spusep_set_parens:nn { right } { \rbrace }
1594         }
1595     }
1596     {
1597         \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}

```

```

1598     }
1599 }

```

(End of definition for `\@@_spusep_convert:n`.)

\spusep The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1600 \NewDocumentCommand \spusep { O{} m }
1601 {
1602   \mode_if_math:TF
1603   {
1604     \group_begin:
1605       \keys_set:nn { spintent / spusep } {#1}
1606       \__spintent_spusep_convert:n {#2}
1607     \group_end:
1608   }
1609   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1610 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

spread-empty-args The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1611 \msg_new:nnnn { spintent } { spread-empty-args }
1612 { Empty~argument~in~\token_to_str:N \spread. }
1613 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for `spread-empty-args`.)

\l_@@_spread_intent_tl The variable `\l__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle argument \rangle$ for symbol to be injected into the `\MathMLintent`.

```

1614 \tl_new:N \l__spintent_spread_intent_tl

```

(End of definition for `\l_@@_spread_intent_tl`.)

\@@_spread_exec:nn The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle argument \rangle$ is empty or blank. If valid, it evaluates the reading $\langle argument \rangle$ ‘#1’ using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math* symbol ‘#2’.

```

1615 \cs_new_protected:Nn \__spintent_spread_exec:nn
1616 {
1617   \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1618   { \msg_error:nn { spintent } { spread-empty-args } }
1619   {
1620     \__spintent_sanitize_affix:Nn \l__spintent_spread_intent_tl { #1 }
1621     \MathMLintent { \l__spintent_spread_intent_tl } { #2 }
1622   }
1623 }

```

(End of definition for `\@@_spread_exec:nn`.)

\spread The command `\spread` assigns a custom semantic *reading* ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```

1624 \NewDocumentCommand \spread { m m }
1625 {
1626   \mode_if_math:TF
1627   {
1628     \group_begin:
1629       \__spintent_spread_exec:nn { #1 } { #2 }
1630     \group_end:
1631   }
1632   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1633 }

```

(End of definition for `\spread`. This function is documented on page 11.)

11.23.2 The command \spdsym

\l_@@_spdsym_read_sym_tl Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for \spdsym.

```
\l_@@_spdsym_symbol_tl
\l_@@_spdsym_prefix_tl
\l_@@_spdsym_suffix_tl
```

(End of definition for \l_@@_spdsym_read_sym_tl and others.)

```
prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
```

```
1638 \keys_define:nn { spintent / spdsym }
1639 {
1640   prefix .code:n =
1641     { \__spintent_sanitize_affix:Nn \__spintent_spdsym_prefix_tl {#1} },
1642   prefix .initial:n = ,
1643   prefix .value_required:n = true,
1644   suffix .code:n =
1645     { \__spintent_sanitize_affix:Nn \__spintent_spdsym_suffix_tl {#1} },
1646   suffix .initial:n = ,
1647   suffix .value_required:n = true,
1648   set-sym .choices:nn =
1649     { old-style , two-dots , slash }
1650   {
1651     \int_case:nn { \l_keys_choice_int }
1652     {
1653       {1} { \tl_set:Nn \__spintent_spdsym_symbol_tl { \div } }
1654       {2} { \tl_set:Nn \__spintent_spdsym_symbol_tl { : } }
1655       {3} { \tl_set:Nn \__spintent_spdsym_symbol_tl { / } }
1656     }
1657   },
1658   set-sym .initial:n = two-dots,
1659   set-sym .value_required:n = true,
1660   set-sym / unknown .code:n =
1661     \msg_error:nneee { spintent } { unknown-choice }
1662     { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1663   read-sym .choices:nn =
1664     { dividido , dividido-por , dividido-entre }
1665   {
1666     \int_case:nn { \l_keys_choice_int }
1667     {
1668       {1} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido } }
1669       {2} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-por } }
1670       {3} { \tl_set:Nn \__spintent_spdsym_read_sym_tl { dividido-entre } }
1671     }
1672   },
1673   read-sym .initial:n = dividido,
1674   read-sym .value_required:n = true,
1675   read-sym / unknown .code:n =
1676     \msg_error:nneee { spintent } { unknown-choice }
1677     { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1678     { \exp_not:n {#1} },
1679   unknown .code:n =
1680   {
1681     \str_set:Nn \__spintent_command_name_str { spdsym }
1682     \__spintent_unknown_keys_cmd:n {#1}
1683   },
1684 }
```

(End of definition for prefix and others.)

\spdsym Defines the \spdsym operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```
1685 \NewDocumentCommand \spdsym { 0{ } }
1686 {
1687   \mode_if_math:TF
1688   {
1689     \group_begin:
1690     \keys_set:nn { spintent / spdsym } { #1 }
1691     \MathMLintent
1692     {
```



```

1693         \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1694         { \l__spintent_spdsym_prefix_tl - }
1695         \l__spintent_spdsym_read_sym_tl
1696         \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1697         { - \l__spintent_spdsym_suffix_tl }
1698     }
1699     { \l__spintent_spdsym_symbol_tl }
1700 \group_end:
1701 }
1702 { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1703 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l_@@_spmsym_read_sym_tl
\l_@@_spmsym_symbol_tl
\l_@@_spmsym_prefix_tl
\l_@@_spmsym_suffix_tl
1704 \tl_new:N \l__spintent_spmsym_read_sym_tl
1705 \tl_new:N \l__spintent_spmsym_symbol_tl
1706 \tl_new:N \l__spintent_spmsym_prefix_tl
1707 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l_@@_spmsym_read_sym_tl` and others.)

```

prefix    We define the (keys) prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym   1708 \keys_define:nn { spintent / spmsym }
           {
read-sym  1710     prefix                .code:n =
           { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
not-print 1711     prefix                .initial:n = ,
           1712     prefix                .value_required:n = true,
           1713     suffix                .code:n =
           { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
           1714     suffix                .initial:n = ,
           1715     suffix                .value_required:n = true,
           1716     set-sym                .choices:nn =
           { cross , dot , asterisk }
           1717     {
           1718         \int_case:nn { \l_keys_choice_int }
           1719         {
           1720             {
           1721                 {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
           1722                 {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
           1723                 {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
           1724             }
           1725         },
           1726     },
           1727     set-sym                .initial:n = dot,
           1728     set-sym                .value_required:n = true,
           1729     set-sym / unknown      .code:n =
           { \msg_error:nneee { spintent } { unknown-choice }
           { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} } },
           1730     read-sym                .choices:nn =
           { por , multiplicado-por , veces }
           1731     {
           1732         \int_case:nn { \l_keys_choice_int }
           1733         {
           1734             {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
           1735             {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
           1736             {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
           1737         },
           1738     },
           1739     read-sym                .initial:n = por,
           1740     read-sym                .value_required:n = true,
           1741     read-sym / unknown      .code:n =
           { \msg_error:nneee { spintent } { unknown-choice }
           { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} } },
           1742     not-print                .bool_set:N = \l__spintent_spmsym_not_print_bool,
           1743     not-print                .initial:n = false,
           1744     not-print                .value_required:n = true,
           1745     unknown                .code:n =
           {
           1746         \str_set:Nn \l__spintent_command_name_str { spmsym }
           1747         \__spintent_unknown_keys_cmd:n {#1}

```

```

1755     },
1756 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the **\spmsym** operator. Similar to **\spdsym**, it outputs the correct mathematical glyph based on the local *⟨keys⟩*, safely attaching prefixes and suffixes if present.

```

1757 \NewDocumentCommand \spmsym { 0{} }
1758 {
1759   \mode_if_math:TF
1760   {
1761     \group_begin:
1762     \keys_set:nn { spintent / spmsym } { #1 }
1763     \MathMLintent
1764     {
1765       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1766       { \l__spintent_spmsym_prefix_tl - }
1767       \l__spintent_spmsym_read_sym_tl
1768       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1769       { - \l__spintent_spmsym_suffix_tl }
1770     }
1771     {
1772       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1773       {
1774         \invisibletimes
1775       }
1776       { \l__spintent_spmsym_symbol_tl }
1777     }
1778     \group_end:
1779   }
1780   { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1781 }

```

(End of definition for *\spmsym*. This function is documented on page 12.)

11.24 The command \spdiv

The user-level command **\spdiv** formats division representations. It takes two mandatory arguments: the *⟨dividend⟩* and the *⟨divisor⟩*. Both *⟨arguments⟩* are internally processed by the **\spnum** command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the *⟨dividend⟩* nor the *⟨divisor⟩* is *blank*.

11.24.1 Definition of keys for \spdiv

We define the *⟨keys⟩* **wrap-parens** and **read-parens** for **\spdiv** command. Number-related *⟨keys⟩* **cifras-sep**, **read-sign**, **read-period-sep** and **notation-sym** are forwarded to **\spnum** command using **.meta:nn**, while semantic operator *⟨keys⟩* **set-sym** and **read-sym** are forwarded to the **\spdsym** command using **.meta:nn**.

```

1782 \keys_define:nn { spintent / spdiv }
1783 {
1784   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1785   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1786   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1787   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1788   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1789   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1790   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1791   wrap-parens     .initial:n = false,
1792   wrap-parens     .value_required:n = true,
1793   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1794   read-parens     .initial:n = true,
1795   read-parens     .value_required:n = true,
1796   unknown         .code:n =
1797   {
1798     \str_set:Nn \l__spintent_command_name_str { spdiv }
1799     \l__spintent_unknown_keys_cmd:n {#1}
1800   },
1801 }

```

(End of definition for *cifras-sep* and others.)

11.24.2 Definition of error messages

spdiv-empty-argument Raised when either the $\langle\textit{dividend}\rangle$ or the $\langle\textit{divisor}\rangle$ is missing or *blank*.

```

1802 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1803 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1804 {
1805   You~must~provide~both~a~dividend~and~a~divisor.\\
1806 }

```

(End of definition for spdiv-empty-argument.)

11.24.3 Internal functions for \spdiv

@@_spdiv_render_op:nn The function `\__spintent_spdiv_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1807 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1808 {
1809   \spnum {#1}
1810   \MathMLintent { \__spintent_spdsym_read_sym_tl }
1811   {
1812     \__spintent_spdsym_symbol_tl
1813   }
1814   \spnum {#2}
1815 }

```

(End of definition for @@_spdiv_render_op:nn.)

@@_spdiv_render_parens:nn The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1816 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1817 {
1818   \bool_if:NTF \__spintent_spdiv_read_parens_bool
1819   {
1820     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1821   }
1822   {
1823     \MathMLintent { :silent } { ( }
1824     \__spintent_spdiv_render_op:nn {#1} {#2}
1825     \MathMLintent { :silent } { ) }
1826   }
1827 }

```

(End of definition for @@_spdiv_render_parens:nn.)

@@_spdiv_process:nnn The function `\__spintent_spdiv_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1828 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1829 {
1830   \keys_set:nn { spintent / spdiv } { #1 }
1831   \tl_if_blank:nTF { #2 }
1832   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1833   {
1834     \tl_if_blank:nTF { #3 }
1835     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1836     {
1837       \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1838       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1839       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1840     }
1841   }
1842 }

```

(End of definition for @@_spdiv_process:nnn.)

\spdiv Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1843 \NewDocumentCommand \spdiv { 0{ } m m }
1844 {
1845   \mode_if_math:TF
1846   {
1847     \group_begin:
1848     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1849     \group_end:
1850   }
1851   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1852 }

```

(End of definition for `\spd\div`. This function is documented on page 13.)

11.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the `{\langle first factor \rangle}` and the `{\langle second factor \rangle}`. Both `{\langle arguments \rangle}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{\langle first factor \rangle}` nor the `{\langle second factor \rangle}` is *blank*.

11.25.1 Definition of keys for `\spmul`

We define the `\langle keys \rangle` `wrap-parens` and `read-parens` for `\spmul` command. Number-related `\langle keys \rangle` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `\langle keys \rangle` `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1853 \keys_define:nn { spintrent / spmul }
1854 {
1855   cifras-sep      .meta:nn = { spintrent / spnum } { cifras-sep      = {#1} },
1856   read-sign       .meta:nn = { spintrent / spnum } { read-sign       = {#1} },
1857   read-period-sep .meta:nn = { spintrent / spnum } { read-period-sep = {#1} },
1858   notation-sym    .meta:nn = { spintrent / spnum } { notation-sym    = {#1} },
1859   set-sym         .meta:nn = { spintrent / spmsym } { set-sym         = {#1} },
1860   read-sym        .meta:nn = { spintrent / spmsym } { read-sym        = {#1} },
1861   wrap-parens     .bool_set:N = \l__spintrent_spmul_wrap_parens_bool,
1862   wrap-parens     .initial:n = false,
1863   wrap-parens     .value_required:n = true,
1864   read-parens     .bool_set:N = \l__spintrent_spmul_read_parens_bool,
1865   read-parens     .initial:n = true,
1866   read-parens     .value_required:n = true,
1867   unknown         .code:n =
1868     {
1869       \str_set:Nn \l__spintrent_command_name_str { spmul }
1870       \__spintrent_unknown_keys_cmd:n {#1}
1871     },
1872 }

```

(End of definition for `cifras-sep` and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the `{\langle first factor \rangle}` or the `{\langle second factor \rangle}` is missing or *blank*.

```

1873 \msg_new:nnnn { spintrent } { spmul-empty-argument }
1874 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1875 {
1876   You~must~provide~both~first~factor~and~second~factor.\\
1877 }

```

(End of definition for `spmul-empty-argument`.)

11.25.3 Internal functions for `\spmul`

`\@@_spmul_render_op:nn` The function `\__spintrent_spmul_render_op:nn` parses both `{\langle arguments \rangle}` through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1878 \cs_new_protected:Nn \__spintrent_spmul_render_op:nn
1879 {
1880   \spnum {#1}
1881   \MathMLintent { \l__spintrent_spmsym_read_sym_tl }
1882   {
1883     \l__spintrent_spmsym_symbol_tl
1884   }
1885   \spnum {#2}
1886 }

```

(End of definition for `\@@_spmul_render_op:nn`.)

`\@@_spmul_render_parens:nn` The function `\__spintrent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1887 \cs_new_protected:Nn \__spintrent_spmul_render_parens:nn
1888 {
1889   \bool_if:NTF \l__spintrent_spmul_read_parens_bool
1890     {
1891       ( \__spintrent_spmul_render_op:nn {#1} {#2} )
1892     }
1893     {
1894       \MathMLintent { :silent } { ( }

```

```

1895     \__spintent_spmul_render_op:nn {#1} {#2}
1896     \MathMLintent { :silent } { ) }
1897   }
1898 }

```

(End of definition for \@_spmull_render_parens:nn.)

\@@_spmull_process:nnn

The function `\__spintent_spmull_process:nnn` sets `\keys`, validates `\{arguments\}`, and decides on parenthesis wrapping.

```

1899 \cs_new_protected:Nn \__spintent_spmull_process:nnn
1900 {
1901   \keys_set:nn { spintent / spmul } { #1 }
1902   \tl_if_blank:nTF { #2 }
1903   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1904   {
1905     \tl_if_blank:nTF { #3 }
1906     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1907     {
1908       \bool_if:NTF \__spintent_spmull_wrap_parens_bool
1909       { \__spintent_spmull_render_parens:nn {#2} {#3} }
1910       { \__spintent_spmull_render_op:nn {#2} {#3} }
1911     }
1912   }
1913 }

```

(End of definition for \@_spmull_process:nnn.)

\spmul

Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1914 \NewDocumentCommand \spmul { O{} m m }
1915 {
1916   \mode_if_math:TF
1917   {
1918     \group_begin:
1919     \__spintent_spmull_process:nnn { #1 } { #2 } { #3 }
1920     \group_end:
1921   }
1922   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1923 }

```

(End of definition for `\spmul`. This function is documented on page 13.)

11.26 The commands `\spmcdd` and `\spmcm`

The user-level commands `\spmcdd` and `\spmcm` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for `\spmcdd` and `\spmcm`

\l_@@_spmcm_spmcdd_luaset_error_str

\l_@@_mc_args_seq

\l_@@_spmcdd_print_result_bool

\l_@@_spmcm_print_result_bool

\l_@@_spmcdd_read_terse_bool

\l_@@_spmcm_read_terse_bool

\l_@@_spmcdd_print_upper_bool

\l_@@_spmcm_print_upper_bool

\l_@@_spmcdd_prefix_tl

\l_@@_spmcm_prefix_tl

\l_@@_spmcdd_suffix_tl

\l_@@_spmcm_suffix_tl

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```

1924 \str_new:N \l__spintent_spmcm_spmcdd_luaset_error_str
1925 \seq_new:N \l__spintent_mc_args_seq
1926 \clist_map_inline:nn { mcd , mcm }
1927 {
1928   \tl_new:c { \l__spintent_sp #1 _luaset_ #1 _value_tl }
1929   \bool_new:c { \l__spintent_sp #1 _print_result_bool }
1930   \bool_new:c { \l__spintent_sp #1 _read_terse_bool }
1931   \bool_new:c { \l__spintent_sp #1 _print_upper_bool }
1932   \tl_new:c { \l__spintent_sp #1 _prefix_tl }
1933   \tl_new:c { \l__spintent_sp #1 _suffix_tl }
1934 }

```

(End of definition for `\l_@@_spmcm_spmcdd_luaset_error_str` and others.)

11.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

1935 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1936 {
1937   Invalid~arguments~provided~to~#1.
1938 }
1939 {
1940   The~arguments~must~be~a~comma-separated~list. \l
1941 }

```

(End of definition for `mc-invalid-arguments`.)

11.26.3 Keys for `\spmc`d and `\spmc`m

We define the shared *(keys)* `result`, `prefix`, `read-cmd`, `upper-case`, `sample-arg` and `silent-cmd`.

```

1942 \clist_map_inline:nn { mcd , mcm }
1943 {
1944   \keys_define:nn { spintent / sp #1 }
1945   {
1946     result      .bool_set:c = { l__spintent_sp #1 _print_result_bool },
1947     result      .initial:n = false,
1948     result      .value_required:n = true,
1949     prefix      .code:n =
1950       { \__spintent_sanitizize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
1951     prefix      .initial:n = ,
1952     prefix      .value_required:n = true,
1953     read-cmd    .choices:nn =
1954       { terse , clear }
1955     {
1956       \int_case:nn { \_keys_choice_int }
1957       {
1958         {1} { \bool_set_true:c { l__spintent_sp #1 _read_terse_bool } }
1959         {2} { \bool_set_false:c { l__spintent_sp #1 _read_terse_bool } }
1960       }
1961     },
1962     read-cmd    .initial:n = clear,
1963     read-cmd    .value_required:n = true,
1964     read-cmd / unknown .code:n =
1965       \msg_error:nneee { spintent } { unknown-choice }
1966       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1967     upper-case  .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
1968     upper-case  .initial:n = false,
1969     upper-case  .value_required:n = true,
1970     sample-arg  .bool_set:c = { l__spintent_sp #1 _sample_arg_bool },
1971     sample-arg  .initial:n = false,
1972     sample-arg  .value_required:n = true,
1973     silent-cmd  .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
1974     silent-cmd  .initial:n = false,
1975     silent-cmd  .value_required:n = true,
1976     unknown     .code:n =
1977       {
1978         \str_set:Nn \l__spintent_command_name_str { sp #1 }
1979         \__spintent_unknown_keys_cmd:n {##1}
1980       },
1981   }
1982 }

```

(End of definition for `result` and others.)

11.26.4 Internal functions for `\spmc`d and `\spmc`m

`\@@_mc_process_args:nn` The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its arguments ‘#2’, injecting MathML intents.

```

1983 \cs_new_protected:Nn \__spintent_mc_process_args:nn
1984 {
1985   \seq_set_from_clist:Nn \l__spintent_mc_args_seq {#2}
1986   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
1987   {
1988     \seq_set_map:Nnn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
1989     { \MathMLintent { :silent } { ##1 } }
1990     \MathMLintent { entre:silent } { ( }
1991     \seq_use:Nnnn \l__spintent_mc_args_seq
1992     { \MathMLintent { _y:silent } { , } }
1993     { \MathMLintent { :silent } { , } }
1994     { \MathMLintent { _y:silent } { , } }
1995   }
1996   {
1997     \MathMLintent { entre } { ( }
1998     \seq_use:Nnnn \l__spintent_mc_args_seq
1999     { \MathMLintent { _y } { , } }
2000     { \MathMLintent { :silent } { , } }
2001     { \MathMLintent { _y } { , } }
2002   }

```



```

2003   \MathMLintent { :silent } { } }
2004 }

```

(End of definition for \@@_mc_process_args:nn.)

\@@_spmc_process:nnnn

The internal function `\__spintent_spmc_process:nnnn` serves as a shared central handler for both the `\spmc`d and `\spmc`m commands. It takes the command identifier ‘#1’, its full reading string ‘#2’, the local *keys* ‘#3’, and the optional values ‘#4’. It dynamically resolves variables using the identifier to build the `MathMLintent` string (including any prefixes or suffixes) and prints the math operator. If values are provided in ‘#4’, it calculates the result using the Lua backend. If an error occurs, it throws the `mc-invalid-arguments` error; otherwise, it processes the arguments and conditionally prints the calculated result.

```

2005 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2006 {
2007   \keys_set:nn { spintent / sp #1 } { #3 }
2008   \MathMLintent
2009   {
2010     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2011     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
2012     \bool_if:cTF { l__spintent_sp #1 _read_terse_bool }
2013     { #1 } { #2 }
2014     \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
2015     {
2016       \tl_if_no_value:nTF {#4}
2017       { :silent }
2018       { \bool_if:cT { l__spintent_sp #1 _sample_arg_bool } { :silent } }
2019     }
2020   }
2021   {
2022     \operatorname
2023     {
2024       \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
2025       { \text_uppercase:n {#1} } { #1 }
2026     }
2027   }
2028   \tl_if_no_value:nF { #4 }
2029   {
2030     \bool_if:cTF { l__spintent_sp #1 _sample_arg_bool }
2031     {
2032       \__spintent_mc_process_args:nn {#1} {#4}
2033     }
2034     {
2035       \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2036       \str_if_eq:VnTF \l__spintent_spmcm_spmcd_luaset_error_str { true }
2037       {
2038         \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2039       }
2040       {
2041         \__spintent_mc_process_args:nn {#1} {#4}
2042         \bool_if:cT { l__spintent_sp #1 _print_result_bool }
2043         {
2044           \MathMLintent { es-igual-a } { = }
2045           \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
2046         }
2047       }
2048     }
2049   }
2050 }

```

(End of definition for \@@_spmc_process:nnnn.)

\spmc

\spmc

The commands `\spmc`d and `\spmc`m act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2051 \NewDocumentCommand \spmc { 0{ } d() }
2052 {
2053   \mode_if_math:TF
2054   {
2055     \group_begin:
2056     \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2057     \group_end:

```

```

2058     }
2059     { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2060   }
2061   \NewDocumentCommand \spmc { 0{} d() }
2062   {
2063     \mode_if_math:TF
2064     {
2065       \group_begin:
2066         \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2067       \group_end:
2068     }
2069     { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2070   }

```

(End of definition for `\spmc` and `\spmc`. These functions are documented on page 15.)

11.27 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\l_@@_md_out_seq
\l_@@_spnM_luaset_result_tl
\l_@@_spnD_luaset_result_tl
\l_@@_spnM_luaset_is_truncated_str
\l_@@_spnD_luaset_is_truncated_str
\l_@@_spnM_print_result_bool
\l_@@_spnD_print_result_bool
\l_@@_spnM_silent_cmd_bool
\l_@@_spnD_silent_cmd_bool
\l_@@_spnM_prefix_tl
\l_@@_spnD_prefix_tl
\l_@@_spnM_print_result_num_tl
\l_@@_spnD_print_result_num_tl
2071 \seq_new:N \__spintent_md_out_seq
2072 \clist_map_inline:nn { nM , nD }
2073 {
2074   \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2075   \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2076   \bool_new:c { \__spintent_sp #1 _print_result_bool }
2077   \bool_new:c { \__spintent_sp #1 _silent_cmd_bool }
2078   \tl_new:c { \__spintent_sp #1 _prefix_tl }
2079   \tl_new:c { \__spintent_sp #1 _print_result_num_tl }
2080 }

```

(End of definition for `\l_@@_md_out_seq` and others.)

11.27.2 Keys for `\spnM` and `\spnD`

We define the shared (*keys*) `result`, `silent-cmd`, `result-num` and `prefix`.

```

result
silent-cmd
result-num
prefix
sample-arg
2081 \clist_map_inline:nn { nM , nD }
2082 {
2083   \keys_define:nn { spintent / sp #1 }
2084   {
2085     result .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2086     result .initial:n = false,
2087     result .value_required:n = true,
2088     silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
2089     silent-cmd .initial:n = false,
2090     silent-cmd .value_required:n = true,
2091     result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2092     result-num .initial:n = ,
2093     prefix .code:n =
2094       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2095     prefix .initial:n = ,
2096     prefix .value_required:n = true,
2097     sample-arg .bool_set:c = { \__spintent_sp #1 _sample_arg_bool },
2098     sample-arg .initial:n = false,
2099     sample-arg .value_required:n = true,
2100     unknown .code:n =
2101       {
2102         \str_set:Nn \__spintent_command_name_str { sp #1 }
2103         \__spintent_unknown_keys_cmd:n {##1}
2104       },
2105   }
2106 }

```

(End of definition for `result` and others.)

11.27.3 Definition of error messages for \spnM and \spnD

md-invalid-argument Exception triggered when the argument for multiples or divisors is invalid.

```
2107 \msg_new:nnnn { spintenc } { md-invalid-argument }
2108 { Invalid~argument~provided~to~#1. }
2109 {
2110   The~argument~must~be~a~single~positive~integer. \
2111 }
```

(End of definition for md-invalid-argument.)

11.27.4 Internal functions for \spnM and \spnD

\@@_spnM_spnD_process:nnnnn

The internal function `\__spintenc_spnM_spnD_process:nnnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintenc_spnM_and_D_exec:nnnn`.

```
2112 \cs_new_protected:Nn \__spintenc_spnM_spnD_process:nnnnn
2113 {
2114   \keys_set:nn { spintenc / sp #1 } { #4 }
2115   \__spintenc_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2116 }
```

(End of definition for \@@_spnM_spnD_process:nnnnn.)

\@@_spnM_spnD_intent:nn

The internal function `\__spintenc_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined and handles specific string normalizations, such as ensuring the correct accentuation for “múltiplos”.

```
2117 \cs_new:Nn \__spintenc_spnM_spnD_intent:nn
2118 {
2119   \tl_if_empty:cF { l__spintenc_sp #1 _prefix_tl }
2120   { \tl_use:c { l__spintenc_sp #1 _prefix_tl } - }
2121   \str_case:nnF { #2 }
2122   { { múltiplos } { múltiplos } }
2123   { #2 }
2124 }
```

(End of definition for \@@_spnM_spnD_intent:nn.)

\@@_spnM_spnD_silent:nn

The internal function `\__spintenc_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like “de”.

```
2125 \cs_new_protected:Nn \__spintenc_spnM_spnD_silent:nn
2126 {
2127   \MathMLintent { :silent } { \operatorname { #1 } }
2128   \MathMLintent { :silent } { ( }
2129   \MathMLintent { :silent } { \__spintenc_invisible_sep: }
2130   \MathMLintent { :silent } { #2 }
2131   \MathMLintent { :silent } { ) }
2132 }
```

(End of definition for \@@_spnM_spnD_silent:nn.)

\@@_spnM_spnD_result:n

The internal function `\__spintenc_spnM_and_D_result:n` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
2133 \cs_new_protected:Nn \__spintenc_spnM_spnD_result:n
2134 {
2135   \use:c { __spintenc_luafun_calcular_ #1 :nn }
2136   { \l__spintenc_luaseta_part_int_tl }
2137   { \tl_use:c { l__spintenc_sp #1 _print_result_num_tl } }
2138   \MathMLintent { son } { = \{ }
2139   \seq_set_from_clist:Ne
2140   \l__spintenc_md_out_seq { \tl_use:c { l__spintenc_sp #1 _luaseta_result_tl } }
2141   \seq_use:Nnnn \l__spintenc_md_out_seq
2142   { \MathMLintent { _y } { , } }
2143   { \MathMLintent { :silent } { , } }
2144   { \MathMLintent { _y } { , } }
```

```

2145 \str_if_eq:vnT
2146 { \__spintenc_sp #1 \luaset_is_truncated_str } { true }
2147 {
2148   \MathMLintent { :silent } { , } ~ \dots
2149 }
2150 \MathMLintent { :silent } { \} }
2151 }

```

(End of definition for \@@_spnM_spnD_result:n.)

\@@_spnM_spnD_exec:nnnn

The function `\__spintenc_spnM_and_D_exec:nnnn` implements the multiples and divisors commands. It takes the `{\langle argument \rangle}` ‘#1’ (the command name), the `{\langle argument \rangle}` ‘#2’ (the mathematical symbol), the base reading `{\langle argument \rangle}` ‘#3’, and the optional `{\langle argument \rangle}` ‘#4’. If no `{\langle argument \rangle}` is provided, it evaluates the key `silent-cmd` using the default token ‘n’. If an `{\langle argument \rangle}` is given, it evaluates the key `sample-arg`. If true, it processes the `{\langle argument \rangle}` directly and evaluates the key `silent-cmd`. If false, it bypasses the *silent mode*, validates the `{\langle argument \rangle}` as a natural number, and evaluates the key `result` optionally calculate and append the resulting set.

```

2152 \cs_new_protected:Nn \__spintenc_spnM_spnD_exec:nnnn
2153 {
2154   \tl_if_novalue:nTF { #4 }
2155   {
2156     \bool_if:cTF { \__spintenc_sp #1 _silent_cmd_bool }
2157     { \__spintenc_spnM_spnD_silent:nn { #2 } { n } }
2158     {
2159       \MathMLintent
2160       { \__spintenc_spnM_spnD_intent:nn { #1 } { #3 } }
2161       { \operatorname { #2 } ( n ) }
2162     }
2163   }
2164   {
2165     \bool_if:cTF { \__spintenc_sp #1 _sample_arg_bool }
2166     {
2167       \bool_if:cTF { \__spintenc_sp #1 _silent_cmd_bool }
2168       { \__spintenc_spnM_spnD_silent:nn { #2 } { #4 } }
2169       {
2170         \MathMLintent
2171         { \__spintenc_spnM_spnD_intent:nn { #1 } { #3 } }
2172         { \operatorname { #2 } } ( #4 )
2173       }
2174     }
2175     {
2176       \__spintenc_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2177       \str_if_eq:VnTF \__spintenc_luaset_has_natural_str { true }
2178       {
2179         \MathMLintent
2180         { \__spintenc_spnM_spnD_intent:nn { #1 } { #3 } }
2181         { \operatorname { #2 } } ( #4 )
2182         \bool_if:cT { \__spintenc_sp #1 _print_result_bool }
2183         { \__spintenc_spnM_spnD_result:n { #1 } }
2184       }
2185       {
2186         \msg_error:nne { spintenc } { md-invalid-argument } { \exp_not:c { sp #1 } }
2187       }
2188     }
2189   }
2190 }

```

(End of definition for \@@_spnM_spnD_exec:nnnn.)

`\spnM` The commands `\spnM` and `\spnD` act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function `\__spintenc_spnM_spnD_process:nnnnn`, passing along the optional `\keys` ‘#1’ and the numeric `{\langle argument \rangle}` ‘#2’.

```

2191 \NewDocumentCommand \spnM { 0{} d() }
2192 {
2193   \mode_if_math:TF
2194   {
2195     \group_begin:
2196     \__spintenc_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2197     \group_end:
2198   }

```

```

2199     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2200   }
2201   \NewDocumentCommand \spnD { 0{} d() }
2202   {
2203     \mode_if_math:TF
2204     {
2205       \group_begin:
2206       \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } {#1} {#2}
2207       \group_end:
2208     }
2209     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2210   }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.28 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a MathML `intent` tree with the correct postfixes for NVDA/MathCAT.

```

\l_@@_spang_luaset_grado_str
\l_@@_spang_luaset_minuto_str
\l_@@_spang_luaset_segundo_str
\l_@@_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2211 \str_new:N \l__spintent_spang_luaset_grado_str
2212 \str_new:N \l__spintent_spang_luaset_minuto_str
2213 \str_new:N \l__spintent_spang_luaset_segundo_str
2214 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for `\l_@@_spang_luaset_grado_str` and others.)

11.28.1 Definition of error messages for `\spang`

invalid-angle-format

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2215 \msg_new:nnnn { spintent } { invalid-angle-format }
2216 { Invalid~angle~format~in~#1. }
2217 {
2218   The~argument~'~#2'~could~not~be~parsed. \\\
2219 }

```

(End of definition for `invalid-angle-format`.)

11.28.2 Internal functions for `\spang`

\l_@@_spang_process:n

The internal function `\__spintent_spang_process:n` parses the angle input using `\l_@@_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaset_grado_str`, `\l__spintent_spang_luaset_minuto_str`, and `\l__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2220 \cs_new_protected:Nn \__spintent_spang_process:n
2221 {
2222   \str_set:Nn \l__spintent_spang_luaset_error_str { false }
2223   \__spintent_luafun_spang_parse:n { #1 }
2224   \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
2225   {
2226     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2227   }
2228   {
2229     \str_if_empty:NF \l__spintent_spang_luaset_grado_str
2230     {
2231       \spunit{ \l__spintent_spang_luaset_grado_str °}
2232     }
2233     \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
2234     {
2235       \MathMLintent { minutos : postfix($m) }
2236       {
2237         \MathMLarg { m }
2238         {
2239           \spunit{ \l__spintent_spang_luaset_minuto_str ' }

```

```

2240         }
2241     }
2242 }
2243 \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
2244 {
2245     \MathMLintent { segundos : postfix($s) }
2246     {
2247         \MathMLarg { s }
2248         {
2249             \spunit { \l__spintent_spang_luaset_segundo_str " }
2250         }
2251     }
2252 }
2253 }
2254 }

```

(End of definition for `\@@_spang_process:n`.)

\spang The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2255 \NewDocumentCommand \spang { m }
2256 {
2257     \mode_if_math:TF
2258     {
2259         \group_begin:
2260         \__spintent_spang_process:n { #1 }
2261         \group_end:
2262     }
2263     { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2264 }

```

(End of definition for `\spang`. This function is documented on page 21.)

11.29 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.29.1 Definition of keys for `\spnZ`

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2265 \keys_define:nn { spintent / spnZ }
2266 {
2267     cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2268     read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2269     wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2270     wrap-parens .initial:n = false,
2271     wrap-parens .value_required:n = true,
2272     read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2273     read-parens .initial:n = true,
2274     read-parens .value_required:n = true,
2275     unknown .code:n =
2276     {
2277         \str_set:Nn \l__spintent_command_name_str { spnZ }
2278         \__spintent_unknown_keys_cmd:n {#1}
2279     },
2280 }

```

(End of definition for `cifras-sep` and others.)

11.29.2 Definition of error messages

spnz-not-integer Raised when the $\langle argument \rangle$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2281 \msg_new:nnnn { spintent } { spnz-not-integer }
2282 { The~value~'~#1'~is~not~a~valid~integer. }
2283 {

```



```

2284 Command~\token_to_str:N \spnZ~only~accepts~integers.\{
2285 }

```

(End of definition for `spnz-not-integer`.)

11.29.3 Internal functions for `\spnZ`

`\@@_spnz_render_parens:` The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\l__spintent_spnz` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2286 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2287 {
2288   \bool_if:NTF \l__spintent_spnz_read_parens_bool
2289   {
2290     ( \__spintent_spnum_print_num_start: )
2291   }
2292   {
2293     \MathMLintent { :silent } { ( }
2294     \__spintent_spnum_print_num_start:
2295     \MathMLintent { :silent } { ) }
2296   }
2297 }

```

(End of definition for `\@@_spnz_render_parens:`.)

`\@@_spnz_process:nn` The function `\__spintent_spnz_process:nn` first processes the `<keys>` passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\l__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\l__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\l__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnum_print_num_start:`.

```

2298 \cs_new_protected:Nn \__spintent_spnz_process:nn
2299 {
2300   \keys_set:nn { spintent / spnZ } { #1 }
2301   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
2302   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2303   {
2304     \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
2305     { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
2306     {
2307       \bool_if:NTF \l__spintent_spnz_wrap_parens_bool
2308       { \__spintent_spnz_render_parens: }
2309       { \__spintent_spnum_print_num_start: }
2310     }
2311   }
2312   { \msg_error:nnn { spintent } { spnz-not-integer } { #2 } }
2313 }

```

(End of definition for `\@@_spnz_process:nn`.)

`\spnZ` Defines the user command `\spnZ`, which is executed within a *group* and strictly enforces *math mode*.

```

2314 \NewDocumentCommand \spnZ { O{} m }
2315 {
2316   \mode_if_math:TF
2317   {
2318     \group_begin:
2319     \__spintent_spnz_process:nn { #1 } { #2 }
2320     \group_end:
2321   }
2322   { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2323 }

```

(End of definition for `\spnZ`. This function is documented on page 16.)

11.30 The command `\spmQ`

The command `\spmQ` typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module `spintent.lua` via `\__spintent_luafun_spmQ_scale_int:Vn`, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.30.1 Definition of variables for \spmQ

```
\l_@@_spmQ_join_word_str
\l_@@_spmQ_math_style_tl
\l_@@_spmQ_intent_read_sign_str
\l_@@_spmQ_wrap_parens_l_tl
\l_@@_spmQ_wrap_parens_r_tl
\l_@@_saved_luamml_flag_int
```

The variable `\l__spintrent_spmQ_join_word_str` stores the value set by the key `join-word` which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable `\l__spintrent_spmQ_math_style_tl` stores the current mathematical style used by the function `\__spintrent_spmQ_set_wrap_parens_size:`.

The variable `\l__spintrent_spmQ_intent_read_sign_str` stores the `:intent` passed to `\MathMLintent` when the signs ‘+’ or ‘-’ are present in the integer part, used by the function `\__spintrent_spmQ_print_sacle_int:n`.

The variables `\l__spintrent_spmQ_wrap_parens_l_tl` and `\l__spintrent_spmQ_wrap_parens_r_tl` store the sized parentheses set by the function `\__spintrent_spmQ_set_wrap_parens_size:` and used by the key `wrap-parens`.

The integer variable `\l__spintrent_saved_luamml_flag_int` temporarily stores the current value of `\l__luamml_flag_int` before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents `luamml` from generating a spurious `<math>` element around the sub_box processed in isolation, before `\frac` has expanded.

```
2324 \str_new:N \l__spintrent_spmQ_join_word_str
2325 \tl_new:N \l__spintrent_spmQ_math_style_tl
2326 \str_new:N \l__spintrent_spmQ_intent_read_sign_str
2327 \tl_new:N \l__spintrent_spmQ_wrap_parens_l_tl
2328 \tl_new:N \l__spintrent_spmQ_wrap_parens_r_tl
2329 \int_new:N \l__spintrent_saved_luamml_flag_int
```

(End of definition for `\l_@@_spmQ_join_word_str` and others.)

11.30.2 Definition of error messages for \spmQ

```
spmQ-zero-denominator
spmQ-not-integer
```

The error message `spmQ-zero-denominator` is raised when a zero denominator is passed to `\spmQ`. The second error `spmQ-not-integer` is returned when the integer part passed in `{\langle argument \rangle}` is not actually an integer.

```
2330 \msg_new:nnnn { spintrent } { spmQ-zero-denominator }
2331 { Zero~denominator~in~\token_to_str:N \spmQ. }
2332 {
2333   The~denominator~of~a~fraction~cannot~be~zero.\l
2334 }
2335 \msg_new:nnnn { spintrent } { spmQ-not-integer }
2336 { The~value~'#1'~is~not~a~valid~integer. }
2337 {
2338   Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\l
2339 }
```

(End of definition for `spmQ-zero-denominator` and `spmQ-not-integer`.)

11.30.3 Definition of keys for \spmQ

```
join-word
use-spnZ
wrap-parens
read-parens
print-dfrac
unknown
```

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```
2340 \keys_define:nn { spintrent / spmQ }
2341 {
2342   join-word .choices:nn =
2343     { entero , enteros , y }
2344     {
2345       \int_case:nn { \l_keys_choice_int }
2346       {
2347         {1} { \str_set:Nn \l__spintrent_spmQ_join_word_str { entero } }
2348         {2} { \str_set:Nn \l__spintrent_spmQ_join_word_str { enteros } }
2349         {3} { \str_set:Nn \l__spintrent_spmQ_join_word_str { y } }
2350       }
2351     },
2352   join-word .initial:n = y,
2353   join-word .value_required:n = true,
2354   join-word/unknown .code:n =
2355     \msg_error:nneee { spintrent } { unknown-choice }
2356     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2357   use-spnZ .bool_set:N = \l__spintrent_spmQ_use_spnZ_bool,
2358   use-spnZ .initial:n = true,
2359   use-spnZ .value_required:n = true,
2360   print-dfrac .bool_set:N = \l__spintrent_spmQ_print_dfrac_bool,
2361   print-dfrac .initial:n = false,
2362   print-dfrac .value_required:n = true,
2363   wrap-parens .bool_set:N = \l__spintrent_spmQ_wrap_parens_bool,
2364   wrap-parens .initial:n = false,
2365   wrap-parens .value_required:n = true,
```

```

2366     read-parens      .bool_set:N = \l__spintent_spnQ_read_parens_bool,
2367     read-parens      .initial:n = true,
2368     read-parens      .value_required:n = true,
2369     unknown          .code:n =
2370     {
2371         \str_set:Nn \l__spintent_command_name_str { spmQ }
2372         \__spintent_unknown_keys_cmd:n {#1}
2373     },
2374 }

```

(End of definition for join-word and others.)

11.30.4 Internal functions for \spmQ

\l__spmQ_set_wrap_parens_size:

The function `\__spintent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LuaTeX and sets the variables `\l__spmQ_wrap_parens_l_tl` and `\l__spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2375 \cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
2376 {
2377     \int_case:nnF { \mathstyle }
2378     {
2379         { 0 }
2380         {
2381             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2382             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2383         }
2384         { 1 }
2385         {
2386             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2387             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2388         }
2389         { 2 }
2390         {
2391             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2392             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2393         }
2394         { 3 }
2395         {
2396             \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2397             \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2398         }
2399     }
2400     {
2401         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2402         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2403     }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spmQ_wrap_parens_l_tl` and `\l__spmQ_wrap_parens_r_tl` to `\Bigl\lparen`/`\Bigr\rparen` unconditionally.

```

2404     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2405     {
2406         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2407         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2408     }
2409 }

```

(End of definition for \l__spmQ_set_wrap_parens_size:.)

\l__spmQ_start_wrap_parens:

\l__spmQ_stop_wrap_parens:

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spmQ_wrap_parens_l_tl` or `\l__spmQ_wrap_parens_r_tl`.

If the variable `\l__spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2410 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2411 {

```

```

2412     \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2413     {
2414         \__spinttent_spmQ_set_wrap_parens_size:
2415         \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2416         { \l__spinttent_spmQ_wrap_parens_l_tl }
2417         { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_l_tl } }
2418     }
2419 }
2420 \cs_new_protected:Nn \__spinttent_spmQ_stop_wrap_parens:
2421 {
2422     \bool_if:NT \l__spinttent_spmQ_wrap_parens_bool
2423     {
2424         \__spinttent_spmQ_set_wrap_parens_size:
2425         \bool_if:NTF \l__spinttent_spmQ_read_parens_bool
2426         { \l__spinttent_spmQ_wrap_parens_r_tl }
2427         { \MathMLintent { :silent } { \l__spinttent_spmQ_wrap_parens_r_tl } }
2428     }
2429 }

```

(End of definition for \@@_spmQ_start_wrap_parens: and \@@_spmQ_stop_wrap_parens:.)

\@@_spmQ_set_join_word: The function `\__spinttent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l_@@_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c_@@_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2430 \cs_new_protected:Nn \__spinttent_spmQ_set_join_word:
2431 {
2432     \str_case:Nn \l__spinttent_spmQ_join_word_str
2433     {
2434         { entero } { \MathMLintent { _entero } { \c__spinttent_invisible_sep_tl } }
2435         { enteros } { \MathMLintent { _enteros } { \c__spinttent_invisible_sep_tl } }
2436         { y } { \MathMLintent { _y- } { \c__spinttent_invisible_sep_tl } }
2437     }
2438 }

```

(End of definition for \@@_spmQ_set_join_word:.)

\@@_spmQ_print_scale_int: The function `\__spinttent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spinttent_luafun_spmQ_scale_int:` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l_@@_saved_luamml_flag_int` and setting it to `0` (skip completely). This prevents `luamml` from generating a spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l_@@_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l_@@_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2439 \cs_new_protected:Nn \__spinttent_spmQ_print_scale_int:
2440 {
2441     \int_set_eq:Nc \l__spinttent_saved_luamml_flag_int { l__luamml_flag_int }
2442     \int_set:Nn \l__luamml_flag_int { 0 }
2443     \bool_if:NTF \l__spinttent_spmQ_print_dfrac_bool
2444     { \__spinttent_luafun_spmQ_scale_int:Nn \l__spinttent_luaset_part_int_tl { dfrac } }
2445     { \__spinttent_luafun_spmQ_scale_int:Nn \l__spinttent_luaset_part_int_tl { frac } }
2446     \int_set_eq:cN { l__luamml_flag_int } \l__spinttent_saved_luamml_flag_int
2447 }

```

(End of definition for \@@_spmQ_print_scale_int:.)

\@@_spmQ_print_sacle_int:n The function `\__spinttent_spmQ_print_sacle_int:n` takes the integer part of the “mixed number” passed in `#1`. It first calls `\__spinttent_luafun_clean_split_and_set:n` on `#1` to parse and populate the shared Lua variables, then checks the status of `\l_@@_luaset_has_integer_str`.

If the status is `true`, it inspects `\l_@@_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spinttent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spinttent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An

`\__spintent_invisible_sep`: call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2448 \cs_new_protected:Nn \__spintent_spmQ_print_sacle_int:n
2449 {
2450   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2451   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2452   {
2453     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
2454     {
2455       \__spintent_spmQ_print_scale_int:
2456     }
2457     {
2458       \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
2459       {
2460         \str_set:Nn
2461           \l__spintent_spmQ_intent_read_sign_str { _más:prefix($e) }
2462       }
2463       {
2464         \str_set:Nn
2465           \l__spintent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2466       }
2467       \MathMLintent { \l__spintent_spmQ_intent_read_sign_str }
2468       {
2469         \l__spintent_luaset_sign_tl \__spintent_invisible_sep:
2470         \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
2471       }
2472     }
2473   }
2474   { \msg_error:nn { spintent } { spmQ-not-integer } }
2475 }

```

(End of definition for `\@@_spmQ_print_sacle_int:n`.)

`\@@_spmQ_print_spnZ_dfrac:nn`

The function `\__spintent_spmQ_print_spnZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spintent_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\l_@@_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l_@@_spmQ_use_spnZ_bool` and `\l_@@_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spnZ` and `print-dfrac`. When `use-spnZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both branches, the fraction command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2476 \cs_new_protected:Nn \__spintent_spmQ_print_spnZ_dfrac:nn
2477 {
2478   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2479   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { false }
2480   { \msg_error:nn { spintent } { spmQ-not-integer } }
2481   \bool_if:NTF \l__spintent_spmQ_use_spnZ_bool
2482   {
2483     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2484     { \spnZ { #1 } } { \spnZ { #2 } }
2485   }
2486   {
2487     \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
2488     { #1 } { #2 }
2489   }
2490 }

```

(End of definition for `\@@_spmQ_print_spnZ_dfrac:nn`.)

`\@@_spmQ_print:nnn`

The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_sacle_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```

2491 \cs_new_protected:Nn \__spintent_spmQ_print:nnn

```

```

2492 {
2493   \__spintent_spmQ_start_wrap_parens:
2494   \__spintent_spmQ_print_sacle_int:n { #1 }
2495   \__spintent_spmQ_set_join_word:
2496   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2497   \__spintent_spmQ_stop_wrap_parens:
2498 }

```

(End of definition for \@@_spmQ_print:nnn.)

\@@_spmQ_exec:nnnn

The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\l_@@_luaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2499 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2500 {
2501   \keys_set:nn { spintent / spmQ } { #1 }
2502   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2503   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2504   {
2505     \int_compare:nNnTF { \l__spintent_luaset_part_int_tl } = { 0 }
2506     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2507     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2508   }
2509   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2510 }

```

(End of definition for \@@_spmQ_exec:nnnn.)

\spmQ

The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional *(keys)* and the *{(arguments)}*.

```

2511 \NewDocumentCommand \spmQ { O{} m m m }
2512 {
2513   \mode_if_math:TF
2514   {
2515     \group_begin:
2516     \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2517     \group_end:
2518   }
2519   { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2520 }

```

(End of definition for \spmQ. This function is documented on page 16.)

11.31 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\l_@@_geo_luaset_error_str`, `\l_@@_geo_luaset_intent_str` and `\l_@@_geo_luaset_print_tl`, constructing a valid MathCAT intent.

11.31.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

2521 \str_new:N \l__spintent_geo_luaset_error_str
2522 \str_new:N \l__spintent_geo_luaset_intent_str
2523 \tl_new:N \l__spintent_geo_luaset_print_tl
2524 \str_new:N \l__spintent_geo_luaset_is_name_str
2525 \str_new:N \l__spintent_geo_luaset_poly_sym_str
2526 \str_new:N \l__spintent_geo_luaset_intent_pts_str
2527 \str_new:N \l__spintent_geo_luaset_angulo_case_str
2528 \tl_new:N \l__spintent_geo_side_cmd_tl
2529 \tl_new:N \l__spintent_geo_fig_op_tl
2530 \str_new:N \l__spintent_geo_luaset_concept_str
2531 \str_new:N \l__spintent_geo_luaset_letra_word_str
2532 \str_new:N \l__spintent_geo_luaset_sub_spoken_str

```

(End of definition for \l_@@_geo_luaset_error_str and others.)

11.32 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.32.1 Definition of variables for `\spPoint`

`\l_@@_spPoint_mode_str`
`\l_@@_spPoint_prefix_tl`

Internal variables holding the values set by `read-cmd` and `prefix`.

```
2533 \str_new:N \__spintent_spPoint_mode_str
2534 \tl_new:N \__spintent_spPoint_prefix_tl
```

(End of definition for `\l_@@_spPoint_mode_str` and `\l_@@_spPoint_prefix_tl`.)

11.32.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```
2535 \msg_new:nnnn { spintent } { spPoint-invalid-point }
2536 {
2537   Invalid~point~format~in~\token_to_str:N \spPoint:~'#1'.
2538 }
2539 {
2540   The~argument~must~be~a~single~uppercase~letter~with~an~optional~
2541   subscript~(\_{...})~or~prime~(').\\
2542 }
```

(End of definition for `spPoint-invalid-point`.)

11.32.3 Definition of keys for `\spPoint`

`read-cmd`
`prefix`
`silent-cmd`
`unknown`

We define the *(keys)* `read-cmd`, `prefix` and `silent-cmd`.

```
2543 \keys_define:nn { spintent / spPoint }
2544 {
2545   read-cmd .choices:nn = { school , mathcat }
2546   {
2547     \int_case:nn { \l_keys_choice_int }
2548     {
2549       {1} { \str_set:Nn \__spintent_spPoint_mode_str { school } }
2550       {2} { \str_set:Nn \__spintent_spPoint_mode_str { mathcat } }
2551     }
2552   },
2553   read-cmd .initial:n = school,
2554   read-cmd .value_required:n = true,
2555   read-cmd / unknown .code:n =
2556     \msg_error:nnee { spintent } { unknown-choice }
2557     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2558   prefix .code:n =
2559     { \__spintent_sanitize_affix:Nn \__spintent_spPoint_prefix_tl {#1} },
2560   prefix .initial:n = ,
2561   prefix .value_required:n = true,
2562   silent-cmd .bool_set:N = \__spintent_spPoint_silent_cmd_bool,
2563   silent-cmd .initial:n = false,
2564   silent-cmd .value_required:n = true,
2565   unknown .code:n =
2566     {
2567       \str_set:Nn \__spintent_command_name_str { spPoint }
2568       \__spintent_unknown_keys_cmd:n {#1}
2569     },
2570 }
```

(End of definition for `read-cmd` and others.)

11.32.4 Internal functions for `\spPoint`

`\@@_spPoint_print:n`

The internal function `\__spintent_spPoint_print:n` calls `\__spintent_luafun_geo_point_parse_and_set:` which no longer receives the concept name: it always returns the bare body (e.g. A or A-sub-uno) in `\__spintent_geo_luaset_intent_str`. Only *after* the call does `expl3` assemble the final literal intent — prepending `punto-`, or wrapping the body in `_...` — when `silent-cmd` is active — since neither depends on anything Lua computes.

```
2571 \cs_new_protected:Nn \__spintent_spPoint_print:n
2572 {
2573   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
2574   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
2575   {
2576     \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
```

```

2577     }
2578     \str_if_eq:VnTF \l__spinttent_spPoint_mode_str { school }
2579     {
2580         \bool_if:NTF \l__spinttent_spPoint_silent_cmd_bool
2581         {
2582             \str_set:Ne \l__spinttent_geo_luaset_intent_str { _ \l__spinttent_geo_luaset_intent_str
2583         }
2584         {
2585             \str_set:Ne \l__spinttent_geo_luaset_intent_str { punto- \l__spinttent_geo_luaset_inten
2586         }
2587     }
2588     {
2589         \str_set:Ne \l__spinttent_geo_luaset_intent_str { _ punto :prefix($pt) }
2590         \bool_if:NT \l__spinttent_spPoint_silent_cmd_bool
2591         {
2592             \str_put_right:Nn \l__spinttent_geo_luaset_intent_str { :silent }
2593         }
2594     }
2595     \__spinttent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
2596     \str_if_eq:VnTF \l__spinttent_spPoint_mode_str { school }
2597     {
2598         \MathMLintent
2599         {
2600             \tl_if_empty:NF \l__spinttent_spPoint_prefix_tl
2601             {
2602                 \tl_use:N \l__spinttent_spPoint_prefix_tl -
2603             }
2604             \l__spinttent_geo_luaset_intent_str
2605         }
2606         { \c__spinttent_invisible_sep_tl \l__spinttent_geo_luaset_print_tl }
2607     }
2608     {
2609         \MathMLintent
2610         {
2611             \tl_if_empty:NF \l__spinttent_spPoint_prefix_tl
2612             {
2613                 \tl_use:N \l__spinttent_spPoint_prefix_tl -
2614             }
2615             \l__spinttent_geo_luaset_intent_str
2616         }
2617         {
2618             \c__spinttent_invisible_sep_tl
2619             \MathMLarg { pt } { \l__spinttent_geo_luaset_print_tl }
2620         }
2621     }
2622 }

```

(End of definition for \@@_spPoint_print:n.)

\@@_spPoint_exec:nn The function __spinttent_spPoint_exec:nn receives the optional $\langle keys \rangle$ in ‘#1’ and the point in ‘#2’. It sets the keys via \keys_set:nn, then delegates rendering to __spinttent_spPoint_print:n.

```

2623 \cs_new_protected:Nn \__spinttent_spPoint_exec:nn
2624 {
2625     \keys_set:nn { spinttent / spPoint } { #1 }
2626     \__spinttent_spPoint_print:n { #2 }
2627 }

```

(End of definition for \@@_spPoint_exec:nn.)

\spPoint The public command \spPoint is the interface for typesetting a single geometric point. It enforces math mode, opens a local group, and delegates execution to __spinttent_spPoint_exec:nn.

```

2628 \NewDocumentCommand \spPoint { 0{} m }
2629 {
2630     \mode_if_math:TF
2631     {
2632         \group_begin:
2633         \__spinttent_spPoint_exec:nn { #1 } { #2 }
2634         \group_end:
2635     }
2636     { \msg_error:nnn { spinttent } { need-math-mode } { \spPoint } }
2637 }

```

(End of definition for \spPoint. This function is documented on page 17.)

11.33 The command \sprecta

The user command `\sprecta` supports two notations, detected automatically by the Lua parser: when the argument contains a comma, it is interpreted as two points and rendered with `\overleftarrow`; when it contains no comma, it is interpreted as a proper name and typeset without any visual decorator. The shared variable `\l_@@_geo_luaset_is_name_str` carries the detection result from Lua to the rendering function.

11.33.1 Definition of variables for \sprecta

`\l_@@_sprecta_mode_str`
`\l_@@_sprecta_prefix_tl`

These variables follow the same semantics as `\l_@@_spPoint_mode_str` and `\l_@@_spPoint_prefix_tl` (§11.32).

```
2638 \str_new:N \l__spinttent_sprecta_mode_str
2639 \tl_new:N \l__spinttent_sprecta_prefix_tl
```

(End of definition for `\l_@@_sprecta_mode_str` and `\l_@@_sprecta_prefix_tl`.)

11.33.2 Definition of error messages for \sprecta

`sprecta-invalid-points`

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```
2640 \msg_new:nnnn { spinttent } { sprecta-invalid-points }
2641 { Invalid~format~in~\token_to_str:N \sprecta:~'#1'. }
2642 {
2643   The~argument~must~be~either~a~comma-separated~pair~of~uppercase~
2644   letters~(two-point-notation)~or~a~single~letter~(uppercase~or~
2645   lowercase)~with~optional~subscript~or~prime~(name~notation).\\
2646 }
```

(End of definition for `sprecta-invalid-points`.)

11.33.3 Definition of keys for \sprecta

`read-cmd`
`prefix`
`silent-cmd`
`unknown`

We define the keys `read-cmd`, `prefix` and `silent-cmd`, following the same pattern as `\spPoint` (§11.32).

```
2647 \keys_define:nn { spinttent / sprecta }
2648 {
2649   read-cmd .choices:nn = { school , mathcat }
2650   {
2651     \int_case:nn { \l_keys_choice_int }
2652     {
2653       {1} { \str_set:Nn \l__spinttent_sprecta_mode_str { school } }
2654       {2} { \str_set:Nn \l__spinttent_sprecta_mode_str { mathcat } }
2655     }
2656   },
2657   read-cmd .initial:n = school,
2658   read-cmd .value_required:n = true,
2659   read-cmd / unknown .code:n =
2660     \msg_error:nneee { spinttent } { unknown-choice }
2661     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2662   prefix .code:n =
2663     { \__spinttent_sanitizet_affix:Nn \l__spinttent_sprecta_prefix_tl {#1} },
2664   prefix .initial:n = ,
2665   prefix .value_required:n = true,
2666   silent-cmd .bool_set:N = \l__spinttent_sprecta_silent_cmd_bool,
2667   silent-cmd .initial:n = false,
2668   silent-cmd .value_required:n = true,
2669   unknown .code:n =
2670     {
2671       \str_set:Nn \l__spinttent_command_name_str { sprecta }
2672       \__spinttent_unknown_keys_cmd:n {#1}
2673     },
2674 }
```

(End of definition for `read-cmd` and others.)

11.33.4 Internal functions for \sprecta

`\@@_sprecta_print:n`

The internal function `\__spinttent_sprecta_print:n` first sets `\l_@@_geo_side_cmd_tl` to `recta` — a fixed literal, since `\sprecta` has no term-selecting key. Only then does it decide, according to mode and `silent-cmd`, whether to send `\l_@@_geo_side_cmd_tl` or an empty group to `\__spinttent_luafun_geo_parse_and`.

In `mathcat` mode, `:silent` is appended to `\l_@@_geo_luaset_intent_str` afterward when needed. It then dispatches on `\l_@@_geo_luaset_is_name_str` to select the visual branch: `\overleftarrow` for the two-point notation, or the raw name with no decorator otherwise.

```
2675 \cs_new_protected:Nn \__spinttent_sprecta_print:n
2676 {
2677   \__spinttent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
```

```

2678 \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2679 {
2680   \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
2681 }
2682 \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
2683 {
2684   \bool_if:NTF \l__spintent_sprecta_silent_cmd_bool
2685   {
2686     \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
2687   }
2688   {
2689     \str_set:Ne \l__spintent_geo_luaset_intent_str { recta- \l__spintent_geo_luaset_inten
2690   }
2691 }
2692 {
2693   \str_set:Ne \l__spintent_geo_luaset_intent_str { _ recta :prefix($pts) }
2694   \bool_if:NT \l__spintent_sprecta_silent_cmd_bool
2695   {
2696     \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
2697   }
2698 }
2699 \__spintent_invisible_sep: % need by MathCAT (bug in lualatex https://github.com/latex3/luamml/issues/26)
2700 \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
2701 {
2702   \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
2703   {
2704     \MathMLintent
2705     {
2706       \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2707       {
2708         \tl_use:N \l__spintent_sprecta_prefix_tl -
2709       }
2710       \l__spintent_geo_luaset_intent_str
2711     }
2712     { \c__spintent_invisible_sep_tl \l__spintent_geo_luaset_print_tl }
2713   }
2714   {
2715     \MathMLintent
2716     {
2717       \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2718       {
2719         \tl_use:N \l__spintent_sprecta_prefix_tl -
2720       }
2721       \l__spintent_geo_luaset_intent_str
2722     }
2723     {
2724       \c__spintent_invisible_sep_tl
2725       \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
2726     }
2727   }
2728 }
2729 {
2730   \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
2731   {
2732     \MathMLintent
2733     {
2734       \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2735       {
2736         \tl_use:N \l__spintent_sprecta_prefix_tl -
2737       }
2738       \l__spintent_geo_luaset_intent_str
2739     }
2740     {
2741       \c__spintent_invisible_sep_tl
2742       \overleftrightharpoon { \l__spintent_geo_luaset_print_tl }
2743     }
2744   }
2745   {
2746     \MathMLintent
2747     {
2748       \tl_if_empty:NF \l__spintent_sprecta_prefix_tl

```

```

2749         {
2750             \tl_use:N \l__spintent_sprecta_prefix_tl -
2751         }
2752         \l__spintent_geo_luaset_intent_str
2753     }
2754     {
2755         \c__spintent_invisible_sep_tl
2756         \overleftrightharrow
2757         {
2758             \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
2759         }
2760     }
2761 }
2762 }
2763 }

```

(End of definition for \@@_sprecta_print:n.)

\@@_sprecta_exec:nn Receives the optional $\langle keys \rangle$ in ‘#1’ and the argument in ‘#2’. Sets the keys via \keys_set:nn, then delegates rendering to __spintent_sprecta_print:n.

```

2764 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
2765 {
2766     \keys_set:nn { spintent / sprecta } { #1 }
2767     \__spintent_sprecta_print:n { #2 }
2768 }

```

(End of definition for \@@_sprecta_exec:nn.)

\sprecta The public command \sprecta enforces math mode, opens a local group, and delegates execution to __spintent_sprecta_exec:nn.

```

2769 \NewDocumentCommand \sprecta { 0{} m }
2770 {
2771     \mode_if_math:TF
2772     {
2773         \group_begin:
2774         \__spintent_sprecta_exec:nn { #1 } { #2 }
2775         \group_end:
2776     }
2777     { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
2778 }

```

(End of definition for \sprecta. This function is documented on page 18.)

11.34 The command \sprayo

The user command \sprayo provides a semantic interface for a ray. The key read-sty selects the verbalized term, rayo or semirrecta, both rendered identically with \overrightarrow.

11.34.1 Definition of variables for \sprayo

\l_@@_sprayo_mode_str The first two follow the same semantics as \l_@@_spPoint_mode_str and \l_@@_spPoint_prefix_tl (§11.32). The variable \l__spintent_sprayo_read_sty_str stores the term chosen by read-sty, and is passed directly to Lua as the concept name.

```

2779 \str_new:N \l__spintent_sprayo_mode_str
2780 \tl_new:N \l__spintent_sprayo_prefix_tl
2781 \str_new:N \l__spintent_sprayo_read_sty_str

```

(End of definition for \l_@@_sprayo_mode_str, \l_@@_sprayo_prefix_tl, and \l_@@_sprayo_read_sty_str.)

11.34.2 Definition of error messages for \sprayo

sprayo-invalid-points Raised when the argument is not a comma-separated pair of uppercase letters.

```

2782 \msg_new:nnnn { spintent } { sprayo-invalid-points }
2783 {
2784     Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'.
2785 }
2786 {
2787     The~argument~must~be~a~comma-separated~list~of~exactly~two~
2788     uppercase~letters~with~optional~subscripts~(\{...\})~or~primes~(').\
2789 }

```

(End of definition for sprayo-invalid-points.)

11.34.3 Definition of keys for \sprayo

We define the *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `read-sty`. The key `read-sty` selects the verbalized term: it defaults to `rayo` and accepts `semirrecta` as the alternative term.

```

2790 \keys_define:nn { spintent / sprayo }
2791 {
2792   read-cmd          .choices:nn = { school , mathcat }
2793   {
2794     \int_case:nn { \l_keys_choice_int }
2795     {
2796       {1} { \str_set:Nn \l__spintent_sprayo_mode_str { school } }
2797       {2} { \str_set:Nn \l__spintent_sprayo_mode_str { mathcat } }
2798     }
2799   },
2800   read-cmd          .initial:n = school,
2801   read-cmd          .value_required:n = true,
2802   read-cmd / unknown .code:n =
2803     \msg_error:nneee { spintent } { unknown-choice }
2804     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2805   prefix            .code:n =
2806     { \__spintent_sanitize_affix:Nn \l__spintent_sprayo_prefix_tl {#1} },
2807   prefix            .initial:n = ,
2808   prefix            .value_required:n = true,
2809   silent-cmd        .bool_set:N = \l__spintent_sprayo_silent_cmd_bool,
2810   silent-cmd        .initial:n = false,
2811   silent-cmd        .value_required:n = true,
2812   read-sty          .choices:nn = { rayo , semirrecta }
2813   {
2814     \int_case:nn { \l_keys_choice_int }
2815     {
2816       {1} { \str_set:Nn \l__spintent_sprayo_read_sty_str { rayo } }
2817       {2} { \str_set:Nn \l__spintent_sprayo_read_sty_str { semirrecta } }
2818     }
2819   },
2820   read-sty          .initial:n = rayo,
2821   read-sty          .value_required:n = true,
2822   read-sty / unknown .code:n =
2823     \msg_error:nneee { spintent } { unknown-choice }
2824     { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
2825   unknown           .code:n =
2826     {
2827       \str_set:Nn \l__spintent_command_name_str { sprayo }
2828       \__spintent_unknown_keys_cmd:n {#1}
2829     },
2830 }

```

(End of definition for `read-cmd` and others.)

11.34.4 Internal functions for \sprayo

`\@@_sprayo_print:n` Resolves the concept name to `\l_@@_sprayo_read_sty_str` or an empty group before calling Lua, following the same pattern as `\sprecta` (§11.33), and uses `\overrightarrow` as the visual decorator in both branches.

```

2831 \cs_new_protected:Nn \__spintent_sprayo_print:n
2832 {
2833   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
2834   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2835   {
2836     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
2837   }
2838   \str_if_eq:VnTF \l__spintent_sprayo_mode_str { school }
2839   {
2840     \bool_if:NTF \l__spintent_sprayo_silent_cmd_bool
2841     {
2842       \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
2843     }
2844     {
2845       \str_set:Ne \l__spintent_geo_luaset_intent_str
2846       {
2847         \l__spintent_sprayo_read_sty_str - \l__spintent_geo_luaset_intent_str
2848       }
2849     }
2850   }

```



```

2851     {
2852       \str_set:N\l__spintent_geo_luaset_intent_str { _ \l__spintent_sprayo_read_sty_str :pref
2853       \bool_if:NT \l__spintent_sprayo_silent_cmd_bool
2854       {
2855         \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
2856       }
2857     }
2858     \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
2859     \str_if_eq:VnTF \l__spintent_sprayo_mode_str { school }
2860     {
2861       \MathMLintent
2862       {
2863         \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
2864         {
2865           \tl_use:N \l__spintent_sprayo_prefix_tl -
2866         }
2867         \l__spintent_geo_luaset_intent_str
2868       }
2869       {
2870         \c__spintent_invisible_sep_tl
2871         \overrightarrow { \l__spintent_geo_luaset_print_tl }
2872       }
2873     }
2874     {
2875       \MathMLintent
2876       {
2877         \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
2878         {
2879           \tl_use:N \l__spintent_sprayo_prefix_tl -
2880         }
2881         \l__spintent_geo_luaset_intent_str
2882       }
2883       {
2884         \c__spintent_invisible_sep_tl
2885         \overrightarrow
2886         {
2887           \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
2888         }
2889       }
2890     }
2891   }

```

(End of definition for `\@@_sprayo_print:n`.)

`\@@_sprayo_exec:nn` Sets the keys via `\keys_set:nn`, then delegates rendering to `\__spintent_sprayo_print:n`.

```

2892 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
2893 {
2894   \keys_set:nn { spintent / sprayo } { #1 }
2895   \__spintent_sprayo_print:n { #2 }
2896 }

```

(End of definition for `\@@_sprayo_exec:nn`.)

`\sprayo` Enforces math mode and delegates to `\__spintent_sprayo_exec:nn`.

```

2897 \NewDocumentCommand \sprayo { 0{ } m }
2898 {
2899   \mode_if_math:TF
2900   {
2901     \group_begin:
2902     \__spintent_sprayo_exec:nn { #1 } { #2 }
2903     \group_end:
2904   }
2905   { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
2906 }

```

(End of definition for `\sprayo`. This function is documented on page 18.)

11.35 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. `\spmside` additionally provides `print-m`, controlling whether `\operatorname{m}` is

typeset before the segment. Both commands are generated by a single generic backend over the short identifiers side and mside, via `\clist_map_inline:nn`, following the pattern already used by `\spmcld/\spmcm`.

11.35.1 Definition of variables for `\spside` and `\spmside`

```
\l_@@_sp\#1_mode_str
\l_@@_sp\#1_prefix_tl
\l_@@_sp\#1_read_sty_str
\l_@@_sp\#1_read_arg_str
```

Each pair of variables is declared once for both side and mside via `\clist_map_inline:nn`. The `_mode_str` and `_prefix_tl` variables follow the same semantics as `\l_@@_spPoint_mode_str` and `\l_@@_spPoint_prefix_tl` (§11.32). The `_read_sty_str` variable stores the value set by `read-sty`; the `_read_arg_str` variable stores the optional override set by `read-arg`. Both are assigned through custom `.choices:nn/.code:n` bodies rather than a direct property, so they require explicit declaration here — unlike the boolean keys defined further below.

```
2907 \clist_map_inline:nn { side , mside }
2908 {
2909   \str_new:c { l__spinttent_sp #1 _mode_str      }
2910   \tl_new:c { l__spinttent_sp #1 _prefix_tl      }
2911   \str_new:c { l__spinttent_sp #1 _read_sty_str  }
2912   \str_new:c { l__spinttent_sp #1 _read_arg_str  }
2913 }
```

(End of definition for `\l_@@_sp\#1_mode_str` and others.)

11.35.2 Definition of error messages for `\spside` and `\spmside`

```
spside-invalid-points
spmside-invalid-points
```

Both messages are generated by the same map. The message text uses `##1/##2` — doubled because it is written one level inside the map’s own `#1` substitution — for the command name and the offending argument, both supplied later when `\msg_error:nnee` is called.

```
2914 \clist_map_inline:nn { side , mside }
2915 {
2916   \msg_new:nnnn { spinttent } { sp #1 -invalid-points }
2917   {
2918     Invalid~point~format~in~##1:~'##2'.
2919   }
2920   {
2921     The~argument~must~be~a~comma-separated~list~of~uppercase~
2922     letters~with~optional~subscripts~(\_{...})~or~primes~(').\
2923   }
2924 }
```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.35.3 Definition of keys for `\spside` and `\spmside`

```
read-cmd
prefix
silent-cmd
read-sty
read-arg
print-m
unknown
```

The keyspaces `spinttent/spside` and `spinttent/spmside` are both generated by the same loop, each with its own independent set of keys pointing at its own `:c`-named variables. The keys `silent-cmd` and `print-m` use the direct `.bool_set:c` property, which `l3keys` declares automatically — no separate `\bool_new:c` call is needed for either. `print-m` defaults to `false`, matching the `\angle` symbol convention documented for `\spmangle` (§11.36).

```
2925 \clist_map_inline:nn { side , mside }
2926 {
2927   \keys_define:nn { spinttent / sp #1 }
2928   {
2929     read-cmd .choices:nn = { school , mathcat }
2930     {
2931       \int_case:nn { \l_keys_choice_int }
2932       {
2933         {1} { \str_set:cn { l__spinttent_sp #1 _mode_str } { school } }
2934         {2} { \str_set:cn { l__spinttent_sp #1 _mode_str } { mathcat } }
2935       }
2936     },
2937     read-cmd .initial:n = school,
2938     read-cmd .value_required:n = true,
2939     read-cmd / unknown .code:n =
2940     \msg_error:nnee { spinttent } { unknown-choice }
2941     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
2942     prefix .code:n =
2943     { \__spinttent_sanitize_affix:cn { l__spinttent_sp #1 _prefix_tl } {##1} },
2944     prefix .initial:n = ,
2945     prefix .value_required:n = true,
2946     silent-cmd .bool_set:c = { l__spinttent_sp #1 _silent_cmd_bool },
2947     silent-cmd .initial:n = false,
2948     silent-cmd .value_required:n = true,
2949     read-sty .choices:nn = { lado , segmento , trazo }
2950     {
```

```

2951         \int_case:nn { \l_keys_choice_int }
2952         {
2953             {1} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { lado      } }
2954             {2} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { segmento } }
2955             {3} { \str_set:cn { \l__spintent_sp #1 _read_sty_str } { trazo      } }
2956         }
2957     },
2958     read-sty          .initial:n = lado,
2959     read-sty          .value_required:n = true,
2960     read-sty / unknown .code:n =
2961         \msg_error:nneee { spintent } { unknown-choice }
2962         { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
2963     read-arg          .code:n =
2964         { \str_set:cn { \l__spintent_sp #1 _read_arg_str } {##1} },
2965     read-arg          .initial:n = ,
2966     read-arg          .value_required:n = true,
2967     print-m           .bool_set:c = { \l__spintent_sp #1 _print_m_bool },
2968     print-m           .initial:n = false,
2969     print-m           .value_required:n = true,
2970     unknown           .code:n =
2971         {
2972             \str_set:Nn \l__spintent_command_name_str { sp #1 }
2973             \l__spintent_unknown_keys_cmd:n {##1}
2974         },
2975     }
2976 }

```

(End of definition for read-cmd and others.)

11.35.4 Internal functions for \spside and \spmside

`\@@_spside_process:nn` The internal function `\__spintent_spside_process:nn` takes the short identifier side/mside in ‘#1’ and the point list in ‘#2’. It first resolves the term into `\l_@@_geo_side_cmd_tl`: `\l__spintent_sp#1_read_arg_str` when non-empty, otherwise `\l__spintent_sp#1_read_sty_str`. When #1 is mside, `medida-del-` is prepended. Only then does it decide, according to mode and `silent-cmd`, whether to send that resolved term or an empty group to `\__spintent_luafun_geo_parse_and_set:nnn`.

In `mathcat` mode, `:silent` is appended to `\l__spintent_geo_luaset_intent_str` afterward when needed. When #1 is mside and `print-m` is true, `\operatorname{m}` is typeset wrapped in `\MathMLintent{<:silent)}` before `\overline`, following the same pause-avoiding pattern already used for `\spmarc` (§11.37).

```

2977 \cs_new_protected:Nn \__spintent_spside_process:nn
2978 {
2979     \tl_set:Nn \l__spintent_geo_side_cmd_tl
2980     {
2981         \str_if_empty:cTF { \l__spintent_sp #1 _read_arg_str }
2982         { \tl_use:c { \l__spintent_sp #1 _read_sty_str } }
2983         { \tl_use:c { \l__spintent_sp #1 _read_arg_str } }
2984     }
2985     \str_if_eq:eeT {#1} { mside }
2986     {
2987         \tl_put_left:Nn \l__spintent_geo_side_cmd_tl { medida-del- }
2988     }
2989     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
2990     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2991     {
2992         \msg_error:nnee { spintent } { sp #1 -invalid-points }
2993         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
2994     }
2995     \str_if_eq:eeTF { \str_use:c { \l__spintent_sp #1 _mode_str } } { school }
2996     {
2997         \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
2998         {
2999             \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
3000         }
3001         {
3002             \str_set:Ne \l__spintent_geo_luaset_intent_str
3003             {
3004                 \l__spintent_geo_side_cmd_tl - \l__spintent_geo_luaset_intent_str
3005             }
3006         }
3007     }
3008     {
3009         \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_side_cmd_tl :prefix($

```

```

3010     \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
3011     {
3012         \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
3013     }
3014 }
3015 \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3016 \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3017 {
3018     \MathMLintent
3019     {
3020         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3021         {
3022             \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3023         }
3024         \l__spintent_geo_luaset_intent_str
3025     }
3026     {
3027         \c__spintent_invisible_sep_tl
3028         \str_if_eq:eeTF { #1 } { side }
3029         { \overline { \l__spintent_geo_luaset_print_tl } }
3030         {
3031             \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3032             {
3033                 \MathMLintent { :silent } { \operatorname { m } } }
3034             }
3035             \l__spintent_geo_luaset_print_tl
3036         }
3037     }
3038 }
3039 {
3040     \MathMLintent
3041     {
3042         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3043         {
3044             \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3045         }
3046         \l__spintent_geo_luaset_intent_str
3047     }
3048     {
3049         \c__spintent_invisible_sep_tl
3050         \str_if_eq:eeTF { #1 } { side }
3051         { \overline { \MathMLarg { pts } } { \l__spintent_geo_luaset_print_tl } } }
3052         {
3053             \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3054             {
3055                 \MathMLintent { :silent } { \operatorname { m } } }
3056             }
3057             \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
3058         }
3059     }
3060 }
3061 }

```

(End of definition for \@@_spside_process:nn.)

\@@_spside_exec:nnn

Takes the short identifier in ‘#1’, the optional $\langle keys \rangle$ in ‘#2’, and the point list in ‘#3’. Sets the keys in the correct keyspace via \keys_set:nn — the keyspace name is plain text substituted from ‘#1’, not a csname, so no :c variant is required here.

```

3062 \cs_new_protected:Nn \__spintent_spside_exec:nnn
3063 {
3064     \keys_set:nn { spintent / sp #1 } { #2 }
3065     \__spintent_spside_process:nn { #1 } { #3 }
3066 }

```

(End of definition for \@@_spside_exec:nnn.)

\spside
\spmside

Both public commands are thin wrappers around __spintent_spside_exec:nnn, passing their own short identifier as the first argument.

```

3067 \NewDocumentCommand \spside { 0{} m }
3068 {
3069     \mode_if_math:TF

```

```

3070     {
3071       \group_begin:
3072       __spintent_spside_exec:nnn { side } { #1 } { #2 }
3073       \group_end:
3074     }
3075     { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3076   }
3077   \NewDocumentCommand \spmside { 0{} m }
3078   {
3079     \mode_if_math:TF
3080     {
3081       \group_begin:
3082       __spintent_spside_exec:nnn { mside } { #1 } { #2 }
3083       \group_end:
3084     }
3085     { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3086   }

```

(End of definition for \spside and \spmside. These functions are documented on page 18.)

11.36 The commands \spangle and \spmangle

The user commands `\spangle` and `\spmangle` — renamed from the earlier `\spangulo`/`\spmangulo` — provide a semantic interface for angles, following the same generic pattern as `\spside`/`\spmside`. The key `print-m` chooses between the `\angle`/`\rightangle` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle`/`\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.36.1 Definition of variables for \spangle and \spmangle

`\l_@@_sp\#1_mode_str` Declared once for both `angle` and `mangle` via `\clist_map_inline:nn`. These follow the same semantics as `\l_@@_spPoint_mode_str` and `\l_@@_spPoint_prefix_tl` (§11.32).

```

3087 \clist_map_inline:nn { angle , mangle }
3088 {
3089   \str_new:c { l__spintent_sp #1 _mode_str }
3090   \tl_new:c { l__spintent_sp #1 _prefix_tl }
3091 }

```

(End of definition for `\l_@@_sp\#1_mode_str` and `\l_@@_sp\#1_prefix_tl`.)

11.36.2 Definition of error messages for \spangle and \spmangle

`spangle-invalid-arg` Both messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside`/`\spmside` (§11.35).

```

3092 \clist_map_inline:nn { angle , mangle }
3093 {
3094   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3095   { Invalid~format~in~##1:~'##2'. }
3096   {
3097     The~argument~must~be~one~of:\\
3098     - three~uppercase~points~separated~by~commas~(e.g.\ 'A,~O,~B');\\
3099     - a~single~uppercase~point~with~optional~subscript~or~prime~
3100     (e.g.\ 'A',~'A_{1}');\\
3101     - an~angle~name~(e.g.\ '\string\alpha',~'1',~'ABC').
3102   }
3103 }

```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.36.3 Definition of keys for \spangle and \spmangle

`read-cmd` The keys `silent-cmd`, `recto` and `print-m` all use the direct `.bool_set:c` property, auto-declared by `l3keys`; none needs a separate `\bool_new:c` call.

```

3104 \clist_map_inline:nn { angle , mangle }
3105 {
3106   \keys_define:nn { spintent / sp #1 }
3107   {
3108     read-cmd .choices:nn = { school , mathcat }
3109     {
3110       \int_case:nn { \l_keys_choice_int }
3111       {
3112         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3113         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
3114       }
3115     }
3116   }

```

```

3115     },
3116     read-cmd          .initial:n = school,
3117     read-cmd          .value_required:n = true,
3118     read-cmd / unknown .code:n =
3119       \msg_error:nneee { spintent } { unknown-choice }
3120       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3121     prefix            .code:n =
3122       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3123     prefix            .initial:n = ,
3124     prefix            .value_required:n = true,
3125     silent-cmd        .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3126     silent-cmd        .initial:n = false,
3127     silent-cmd        .value_required:n = true,
3128     recto             .bool_set:c = { l__spintent_sp #1 _recto_bool },
3129     recto             .initial:n = false,
3130     recto             .value_required:n = true,
3131     print-m           .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3132     print-m           .initial:n = false,
3133     print-m           .value_required:n = true,
3134     unknown           .code:n =
3135       {
3136         \str_set:Nn \__spintent_command_name_str { sp #1 }
3137         \__spintent_unknown_keys_cmd:n {##1}
3138       },
3139   }
3140 }

```

(End of definition for read-cmd and others.)

11.36.4 Internal functions for \spangle and \spmangle

\@@_spangle_sym:n

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightangle` when `print-m` is `true`; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

3141 \cs_new_protected:Nn \__spintent_spangle_sym:n
3142 {
3143   \str_if_eq:eeTF {#1} { mangle }
3144   {
3145     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
3146     {
3147       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3148       { \__spintent_right_angle_sqr_sym: } { \angle }
3149     }
3150     {
3151       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3152       { \measuredrightangle } { \measuredangle }
3153     }
3154   }
3155   {
3156     \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3157     { \__spintent_right_angle_sqr_sym: } { \angle }
3158   }
3159 }

```

(End of definition for \@@_spangle_sym:n.)

\@@_spangle_process:nn

Resolves the base name — `ángulo` or `medida-del-ángulo` — into `\l_@@_geo_side_cmd_tl`, appends `-recto` when `l__spintent_sp#1_recto_bool` is `true`, then applies the same `silent-cmd` resolution as `\__spintent_spside_process:nn` (§11.35) before calling `\__spintent_luafun_geo_angulo_parse_and_set:n`.

```

3160 \cs_new_protected:Nn \__spintent_spangle_process:nn
3161 {
3162   \str_if_eq:eeTF {#1} { mangle }
3163   { \tl_set:Nn \l__spintent_geo_side_cmd_tl { medida-del-ángulo } }
3164   { \tl_set:Nn \l__spintent_geo_side_cmd_tl { ángulo } }
3165   \bool_if:cT { l__spintent_sp #1 _recto_bool }
3166   {
3167     \tl_put_right:Nn \l__spintent_geo_side_cmd_tl { -recto }
3168   }
3169   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
3170   \str_if_eq:VnT \l__spintent_geo_luaseta_error_str { true }
3171   {
3172     \msg_error:nnee { spintent } { sp #1 -invalid-arg }

```

```

3173         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3174     }
3175     \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3176     {
3177         \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3178         {
3179             \str_set:Ne l__spintent_geo_luaset_intent_str { _ l__spintent_geo_luaset_intent_str
3180         }
3181         {
3182             \str_set:Ne l__spintent_geo_luaset_intent_str
3183             {
3184                 l__spintent_geo_side_cmd_tl - l__spintent_geo_luaset_intent_str
3185             }
3186         }
3187     }
3188     {
3189         \str_set:Ne l__spintent_geo_luaset_intent_str { _ l__spintent_geo_side_cmd_tl :prefix($
3190         \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
3191         {
3192             \str_put_right:Nn l__spintent_geo_luaset_intent_str { :silent }
3193         }
3194     }
3195     \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3196     \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3197     {
3198         \MathMLintent
3199         {
3200             \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3201             {
3202                 \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3203             }
3204             l__spintent_geo_luaset_intent_str
3205         }
3206         {
3207             \str_if_eq:eeT {#1} { mangle }
3208             {
3209                 \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3210                 {
3211                     \MathMLintent { :silent } { \operatorname { m } } }
3212                 }
3213             }
3214             \__spintent_spangle_sym:n {#1}
3215             l__spintent_geo_luaset_print_tl
3216         }
3217     }
3218     {
3219         \MathMLintent
3220         {
3221             \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3222             {
3223                 \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3224             }
3225             l__spintent_geo_luaset_intent_str
3226         }
3227         {
3228             \str_if_eq:eeT {#1} { mangle }
3229             {
3230                 \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3231                 {
3232                     \MathMLintent { :silent } { \operatorname { m } } }
3233                 }
3234             }
3235             \__spintent_spangle_sym:n {#1}
3236             \MathMLarg { pts } { l__spintent_geo_luaset_print_tl }
3237         }
3238     }
3239 }

```

(End of definition for \@_spangle_process:nn.)

\@@_spangle_exec:nnn Sets the keys in the correct keyspace via \keys_set:nn, then delegates rendering to __spintent_spangle_process:

```

3240 \cs_new_protected:Nn \__spintent_spangle_exec:nnn

```



```

3241 {
3242   \keys_set:nn { spintent / sp #1 } { #2 }
3243   \__spintent_spangle_process:nn { #1 } { #3 }
3244 }

```

(End of definition for \@@_spangle_exec:nnn.)

`\spangle` Both public commands are thin wrappers around `\__spintent_spangle_exec:nnn`, passing their own short identifier as the first argument.

`\spmangle`

```

3245 \NewDocumentCommand \spangle { 0{} m }
3246 {
3247   \mode_if_math:TF
3248   {
3249     \group_begin:
3250     \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
3251     \group_end:
3252   }
3253   { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
3254 }
3255 \NewDocumentCommand \spmangle { 0{} m }
3256 {
3257   \mode_if_math:TF
3258   {
3259     \group_begin:
3260     \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
3261     \group_end:
3262   }
3263   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
3264 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 20.)

11.37 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` — renamed from `\sparco`/`\spmarco` — provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\overline{AB}$ is the only standard notation for the measure of an arc.

11.37.1 Definition of variables for `\sparc` and `\spmarc`

Declared once for both arc and marc via `\clist_map_inline:nn`. These follow the same semantics as `\l_@@_spPoint_mode_str` and `\l_@@_spPoint_prefix_tl` (§11.32).

`\l_@@_sparc_mode_str`
`\l_@@_spmarc_mode_str`
`\l_@@_sparc_prefix_tl`
`\l_@@_spmarc_prefix_tl`

```

3265 \clist_map_inline:nn { arc , marc }
3266 {
3267   \str_new:c { l__spintent_sp #1 _mode_str }
3268   \tl_new:c { l__spintent_sp #1 _prefix_tl }
3269 }

```

(End of definition for `\l_@@_sparc_mode_str` and others.)

11.37.2 Definition of error messages for `\sparc` and `\spmarc`

Both messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside`/`\spmside` (§11.35).

`sparc-invalid-points`
`spmarc-invalid-points`

```

3270 \clist_map_inline:nn { arc , marc }
3271 {
3272   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3273   {
3274     Invalid~point~format~in~##1:~'##2'.
3275   }
3276   {
3277     The~argument~must~be~a~comma-separated~list~of~uppercase~
3278     letters~with~optional~subscripts~(\_{...})~or~primes~(').\ \
3279   }
3280 }

```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.37.3 Definition of keys for \sparc and \spmarc

The keyspaces `spintenc/sparc` and `spintenc/spmarc` are both generated by the same loop, each with its own independent set of keys pointing at its own `:c`-named variables.

```

read-cmd  \clist_map_inline:nn { arc , marc }
prefix    {
silent-cmd \keys_define:nn { spintenc / sp #1 }
unknown    {
  3281   \keys_define:nn { spintenc / sp #1 }
  3282   {
  3283     \keys_define:nn { spintenc / sp #1 }
  3284     {
  3285       read-cmd .choices:nn = { school , mathcat }
  3286       {
  3287         \int_case:nn { \l_keys_choice_int }
  3288         {
  3289           {1} { \str_set:cn { \l__spintenc_sp #1 _mode_str } { school } }
  3290           {2} { \str_set:cn { \l__spintenc_sp #1 _mode_str } { mathcat } }
  3291         }
  3292       },
  3293       read-cmd .initial:n = school,
  3294       read-cmd .value_required:n = true,
  3295       read-cmd / unknown .code:n =
  3296         \msg_error:nneee { spintenc } { unknown-choice }
  3297         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
  3298       prefix .code:n =
  3299         { \l__spintenc_sanitizet_affix:cn { \l__spintenc_sp #1 _prefix_tl } {##1} },
  3300       prefix .initial:n = ,
  3301       prefix .value_required:n = true,
  3302       silent-cmd .bool_set:c = { \l__spintenc_sp #1 _silent_cmd_bool },
  3303       silent-cmd .initial:n = false,
  3304       silent-cmd .value_required:n = true,
  3305       unknown .code:n =
  3306         {
  3307           \str_set:Nn \l__spintenc_command_name_str { sp #1 }
  3308           \l__spintenc_unknown_keys_cmd:n {##1}
  3309         },
  3310     }
  3311 }

```

(End of definition for `read-cmd` and others.)

11.37.4 Internal functions for \sparc and \spmarc

`\@@_sparc_wide_arc:n` The function `\__spintenc_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of [TeX-SX](#).

```

3312 \cs_new_protected_nopar:Nn \__spintenc_sparc_wide_arc:n
3313 {
3314   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
3315 }

```

(End of definition for `\@@_sparc_wide_arc:n`.)

`\@@_sparc_print_arc:n` The function `\__spintenc_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintenc_sparc_wide_arc:n`.

```

3316 \cs_new_protected:Nn \__spintenc_sparc_print_arc:n
3317 {
3318   \cs_if_exist:NTF \widearc
3319   { \widearc {#1} }
3320   { \__spintenc_sparc_wide_arc:n {#1} }
3321 }

```

(End of definition for `\@@_sparc_print_arc:n`.)

`\@@_sparc_process:nn` Follows the same resolution pattern as `\__spintenc_spside_process:nn` (§11.35), with the base names `arco`/`medida-del-arco`. When `#1` is `marc`, `\operatorname{m}` is always typeset, wrapped in `\MathMLintenc{<:silen` before `\__spintenc_sparc_print_arc:n`.

```

3322 \cs_new_protected:Nn \__spintenc_sparc_process:nn
3323 {
3324   \str_if_eq:eeTF {#1} { marc }
3325   { \tl_set:Nn \l__spintenc_geo_side_cmd_tl { medida-del-arco } }
3326   { \tl_set:Nn \l__spintenc_geo_side_cmd_tl { arco } }
3327   \__spintenc_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3328   \str_if_eq:VnT \l__spintenc_geo_luaerror_str { true }

```

```

3329     {
3330       \msg_error:nnee { spintent } { sp #1 -invalid-points }
3331       { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3332     }
3333   \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3334   {
3335     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3336     {
3337       \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
3338     }
3339     {
3340       \str_set:Ne \l__spintent_geo_luaset_intent_str
3341       {
3342         \l__spintent_geo_side_cmd_tl - \l__spintent_geo_luaset_intent_str
3343       }
3344     }
3345   }
3346   {
3347     \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_side_cmd_tl :prefix($
3348     \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
3349     {
3350       \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
3351     }
3352   }
3353   \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3354   \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3355   {
3356     \MathMLintent
3357     {
3358       \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3359       {
3360         \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3361       }
3362       \l__spintent_geo_luaset_intent_str
3363     }
3364     {
3365       \c__spintent_invisible_sep_tl
3366       \str_if_eq:eeT {#1} { marc }
3367       {
3368         \MathMLintent { :silent } { \operatorname { m } }
3369       }
3370       \__spintent_sparc_print_arc:n { \l__spintent_geo_luaset_print_tl }
3371     }
3372   }
3373   {
3374     \MathMLintent
3375     {
3376       \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3377       { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
3378       \l__spintent_geo_luaset_intent_str
3379     }
3380     {
3381       \c__spintent_invisible_sep_tl
3382       \str_if_eq:eeT {#1} { marc }
3383       {
3384         \MathMLintent { :silent } { \operatorname { m } }
3385       }
3386       \__spintent_sparc_print_arc:n
3387       {
3388         \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
3389       }
3390     }
3391   }
3392 }

```

(End of definition for \@@_sparc_process:nn.)

\@@_sparc_exec:nnn Sets the keys in the correct keyspace via \keys_set:nn, then delegates rendering to __spintent_sparc_process:nn

```

3393 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
3394 {
3395   \keys_set:nn { spintent / sp #1 } { #2 }
3396   \__spintent_sparc_process:nn { #1 } { #3 }

```

```
3397 }
```

(End of definition for `\@@_sparc_exec:nnn`.)

`\sparc` Both public commands are thin wrappers around `\__spintent_sparc_exec:nnn`, passing their own short identifier as the first argument.

`\spmarc`

```
3398 \NewDocumentCommand \sparc { O{} m }
3399 {
3400   \mode_if_math:TF
3401   {
3402     \group_begin:
3403       \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
3404     \group_end:
3405   }
3406   { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
3407 }
3408 \NewDocumentCommand \spmarc { O{} m }
3409 {
3410   \mode_if_math:TF
3411   {
3412     \group_begin:
3413       \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
3414     \group_end:
3415   }
3416   { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
3417 }
```

(End of definition for `\sparc` and `\spmarc`. These functions are documented on page 21.)

11.38 The command `\spPoly`

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\l_@@_geo_luaset_intent_str` (the full `intent`, with the figure name) and `\l_@@_geo_luaset_intent_pts_str` (vertices only) are both returned unconditionally, and `expl3` picks between them according to `print-sym`.

11.38.1 Definition of variables for `\spPoly`

`\l_@@_spPoly_mode_str`
`\l_@@_spPoly_prefix_tl`

These follow the same semantics as `\l_@@_spPoint_mode_str` and `\l_@@_spPoint_prefix_tl` (§11.32).

```
3418 \str_new:N \__spintent_spPoly_mode_str
3419 \tl_new:N \__spintent_spPoly_prefix_tl
```

(End of definition for `\l_@@_spPoly_mode_str` and `\l_@@_spPoly_prefix_tl`.)

11.38.2 Definition of error messages for `\spPoly`

`spPoly-invalid-points`

The “error” `spPoly-invalid-points` is raised when the argument in ‘`#1`’ is not a comma-separated list of at least three uppercase letters.

```
3420 \msg_new:nnnn { spintent } { spPoly-invalid-points }
3421 {
3422   Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'.
3423 }
3424 {
3425   The~argument~must~be~a~comma-separated~list~of~at~least~three~
3426   uppercase~letters~with~optional~subscripts~(\_{...})~or~primes~(').\
3427 }
```

(End of definition for `spPoly-invalid-points`.)

11.38.3 Definition of keys for `\spPoly`

`read-cmd`
`prefix`
`print-sym`
`silent-cmd`
`unknown`

The key `print-sym` uses the direct `.bool_set:N` property, auto-declared by `l3keys`. The key `silent-cmd` is a `\keys` alias for `print-sym=false`.

```
3428 \keys_define:nn { spintent / spPoly }
3429 {
3430   read-cmd .choices:nn = { school , mathcat }
3431   {
3432     \int_case:nn { \l_keys_choice_int }
3433     {
3434       {1} { \str_set:Nn \__spintent_spPoly_mode_str { school } }
3435       {2} { \str_set:Nn \__spintent_spPoly_mode_str { mathcat } }
3436     }
3437   },
3438   read-cmd .initial:n = school,
```

```

3439     read-cmd          .value_required:n = true,
3440     read-cmd / unknown .code:n =
3441       \msg_error:nneee { spintrent } { unknown-choice }
3442       { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3443     prefix            .code:n =
3444       { \__spintrent_sanitizet_affix:Nn \__spintrent_spPoly_prefix_tl {#1} },
3445     prefix            .initial:n = ,
3446     prefix            .value_required:n = true,
3447     print-sym         .bool_set:N = \__spintrent_spPoly_print_sym_bool,
3448     print-sym         .initial:n = true,
3449     print-sym         .value_required:n = true,
3450     silent-cmd        .meta:n = { print-sym = false },
3451     unknown           .code:n =
3452       {
3453         \str_set:Nn \__spintrent_command_name_str { spPoly }
3454         \__spintrent_unknown_keys_cmd:n {#1}
3455       },
3456   }

```

(End of definition for read-cmd and others.)

11.38.4 Internal functions for \spPoly

`\@@_spPoly_sym:` Reads `\l_@@_geo_luaset_poly_sym_str` and typesets the corresponding symbol: `\triangle` for `triangle`, `\mdlgwhtsquare` for `square`.

```

3457 \cs_new_protected:Nn \__spintrent_spPoly_sym:
3458 {
3459   \str_case:Vn \__spintrent_geo_luaset_poly_sym_str
3460   {
3461     { triangle } { \triangle }
3462     { square } { \mdlgwhtsquare }
3463   }
3464 }

```

(End of definition for `\@@_spPoly_sym:`.)

`\@@_spPoly_print:n` When `print-sym` is `true`, uses `\l_@@_geo_luaset_intent_str` and wraps `\__spintrent_spPoly_sym:` in `\MathMLintent{<:silent>}` whenever a symbol is available; when `false`, uses `\l_@@_geo_luaset_intent_pts_str` and typesets no symbol at all.

```

3465 \cs_new_protected:Nn \__spintrent_spPoly_print:n
3466 {
3467   \__spintrent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
3468   \str_if_eq:VnT \__spintrent_geo_luaset_error_str { true }
3469   {
3470     \msg_error:nne { spintrent } { spPoly-invalid-points }
3471     { \exp_not:n {#1} }
3472   }
3473   \str_if_eq:VnT \__spintrent_spPoly_mode_str { mathcat }
3474   {
3475     \str_set:Nx \__spintrent_geo_luaset_intent_str
3476     { _ \__spintrent_geo_luaset_concept_str :prefix($pts) }
3477   }
3478   \__spintrent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3479   \bool_if:NTF \__spintrent_spPoly_print_sym_bool
3480   {
3481     \str_if_eq:VnTF \__spintrent_spPoly_mode_str { school }
3482     {
3483       \MathMLintent
3484       {
3485         \tl_if_empty:NF \__spintrent_spPoly_prefix_tl
3486         { \tl_use:N \__spintrent_spPoly_prefix_tl - }
3487         \__spintrent_geo_luaset_intent_str
3488       }
3489       {
3490         \c__spintrent_invisible_sep_tl
3491         \str_if_empty:NF \__spintrent_geo_luaset_poly_sym_str
3492         {
3493           \MathMLintent { :silent } { \__spintrent_spPoly_sym: }
3494         }
3495         \__spintrent_geo_luaset_print_tl
3496       }
3497     }

```

```

3498     {
3499       \MathMLintent
3500       {
3501         \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
3502         {
3503           \tl_use:N \l__spintent_spPoly_prefix_tl -
3504         }
3505         \l__spintent_geo_luaset_intent_str
3506       }
3507       {
3508         \c__spintent_invisible_sep_tl
3509         \str_if_empty:NF \l__spintent_geo_luaset_poly_sym_str
3510         {
3511           \MathMLintent { :silent } { \__spintent_spPoly_sym: }
3512         }
3513         \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
3514       }
3515     }
3516   }
3517   {
3518     \str_if_eq:VnTF \l__spintent_spPoly_mode_str { school }
3519     {
3520       \MathMLintent
3521       {
3522         \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
3523         { \tl_use:N \l__spintent_spPoly_prefix_tl - }
3524         \l__spintent_geo_luaset_intent_pts_str
3525       }
3526       { \c__spintent_invisible_sep_tl \l__spintent_geo_luaset_print_tl }
3527     }
3528     {
3529       \MathMLintent
3530       {
3531         \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
3532         { \tl_use:N \l__spintent_spPoly_prefix_tl - }
3533         \l__spintent_geo_luaset_intent_pts_str
3534       }
3535       {
3536         \c__spintent_invisible_sep_tl
3537         \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
3538       }
3539     }
3540   }
3541 }

```

(End of definition for \@@_spPoly_print:n.)

\@@_spPoly_exec:nn Sets the keys via \keys_set:nn, then delegates rendering to __spintent_spPoly_print:n.

```

3542 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
3543 {
3544   \keys_set:nn { spintent / spPoly } { #1 }
3545   \__spintent_spPoly_print:n { #2 }
3546 }

```

(End of definition for \@@_spPoly_exec:nn.)

\spPoly Enforces math mode and delegates to __spintent_spPoly_exec:nn.

```

3547 \NewDocumentCommand \spPoly { O{ } m }
3548 {
3549   \mode_if_math:TF
3550   {
3551     \group_begin:
3552     \__spintent_spPoly_exec:nn { #1 } { #2 }
3553     \group_end:
3554   }
3555   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
3556 }

```

(End of definition for \spPoly. This function is documented on page 22.)

11.39 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcm`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l_@@_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintcnt{(:silent)}` — the same mechanism already used by `\spPoly` (§11.38).

11.39.1 Definition of variables for `\spAfig` and `\spPfig`

The `_read_arg_str` variable stores the figure-name override set by `read-arg`, independent of `print-sym`.

```

\l_@@_spAfig_mode_str
\l_@@_spPfig_mode_str
\l_@@_spAfig_print_sym_str
\l_@@_spPfig_print_sym_str
\l_@@_spAfig_prefix_tl
\l_@@_spPfig_prefix_tl
\l_@@_spAfig_read_arg_tl
\l_@@_spPfig_read_arg_tl
3557 \tl_new:N \l__spintenc_geo_sym_word_tl
3558 \tl_new:N \l__spintenc_geo_final_print_tl
3559 \str_new:N \l__spintenc_geo_final_intenc_str
3560 \str_new:N \l__spintenc_geo_mathcat_concept_str
3561 \clist_map_inline:nn { Afig , Pfig }
3562 {
3563   \str_new:c { \l__spintenc_sp #1 _mode_str }
3564   \str_new:c { \l__spintenc_sp #1 _print_sym_str }
3565   \tl_new:c { \l__spintenc_sp #1 _prefix_tl }
3566   \tl_new:c { \l__spintenc_sp #1 _read_arg_tl }
3567 }
```

(End of definition for `\l_@@_spAfig_mode_str` and others.)

11.39.2 Definition of error messages for `\spAfig` and `\spPfig`

Both messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside` (§11.35).

```

spAfig-invalid-points
spPfig-invalid-points
3568 \clist_map_inline:nn { Afig , Pfig }
3569 {
3570   \msg_new:nnnn { spintenc } { sp #1 -invalid-points }
3571   {
3572     Invalid~point~format~in~##1:~'##2'.
3573   }
3574   {
3575     The~argument~must~be~empty,~or~a~comma-separated~list~of~at~
3576     least~three~uppercase~letters~with~optional~subscripts~
3577     (\_{...})~or~primes~(').\
3578   }
3579   \msg_new:nnnn { spintenc } { sp #1 -invalid-arg }
3580   { Invalid~argument~format~in~##1:~'##2'. }
3581   {
3582     The~argument~must~be~either~empty,~a~comma-separated~list~of~
3583     vertices,~a~bare~number~or~letter~(e.g.\ '1',~'n'),~or~a~
3584     table~letter~with~an~optional~subscript~(e.g.\ 'F',~'F_{1}\').
3585   }
3586   \msg_new:nnnn { spintenc } { sp #1 -sym-letra-conflict }
3587   { print-sym~cannot~be~combined~with~a~lettered~argument~in~##1. }
3588   {
3589     When~print-sym~is~not~'none',~the~argument~(if~given)~must~
3590     be~a~bare~number~or~letter~(e.g.\ '1',~'n')~table~letters~
3591     like~'F'~or~'P'~are~not~allowed~in~combination~with~print-sym.
3592   }
3593   \msg_new:nnnn { spintenc } { sp #1 -sym-cardinality-conflict }
3594   { print-sym~does~not~match~the~vertex~count~in~##1. }
3595   {
3596     The~forced~symbol~(triangle/square)~must~match~the~number~of~
3597     vertices~given~use~'auto'~to~infer~it~automatically,~or~'none'~
3598     to~omit~it.
3599   }
3600 }
```

(End of definition for `spAfig-invalid-points` and `spPfig-invalid-points`.)

11.39.3 Definition of keys for `\spAfig` and `\spPfig`

We define the *(keys)* `read-cmd`, `prefix`, `print-sym`, `read-arg`, `read-sub` and `silent-cmd` for both commands.

```

3601 \clist_map_inline:nn { Afig , Pfig }
3602 {
3603   \keys_define:nn { spintenc / sp #1 }
3604   {
3605     read-cmd .choices:nn = { school , mathcat }
```



```

3606     {
3607         \int_case:nn { \l_keys_choice_int }
3608         {
3609             {1} { \str_set:cn { \l__spintent_sp #1 _mode_str } { school } }
3610             {2} { \str_set:cn { \l__spintent_sp #1 _mode_str } { mathcat } }
3611         }
3612     },
3613     read-cmd          .initial:n = school,
3614     read-cmd          .value_required:n = true,
3615     read-cmd / unknown .code:n =
3616         \msg_error:nneee { spintent } { unknown-choice }
3617         { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3618     prefix            .code:n =
3619         { \l__spintent_sanitizize_affix:cn { \l__spintent_sp #1 _prefix_tl } {##1} },
3620     prefix            .initial:n = ,
3621     prefix            .value_required:n = true,
3622     print-sym         .choices:nn = { auto , triangle , square , none }
3623     {
3624         \int_case:nn { \l_keys_choice_int }
3625         {
3626             {1} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { auto } }
3627             {2} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { triangle } }
3628             {3} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { square } }
3629             {4} { \str_set:cn { \l__spintent_sp #1 _print_sym_str } { none } }
3630         }
3631     },
3632     print-sym         .initial:n = auto,
3633     print-sym         .value_required:n = true,
3634     print-sym / unknown .code:n =
3635         \msg_error:nneee { spintent } { unknown-choice }
3636         { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
3637     read-arg          .code:n =
3638         { \l__spintent_sanitizize_affix:cn { \l__spintent_sp #1 _read_arg_tl } {##1} },
3639     read-arg          .initial:n = ,
3640     read-arg          .value_required:n = true,
3641     read-sub          .bool_set:c = { \l__spintent_sp #1 _read_sub_bool },
3642     read-sub          .initial:n = false,
3643     read-sub          .value_required:n = true,
3644     silent-cmd        .bool_set:c = { \l__spintent_sp #1 _silent_cmd_bool },
3645     silent-cmd        .initial:n = false,
3646     silent-cmd        .value_required:n = true,
3647     unknown           .code:n =
3648         {
3649             \str_set:Nn \l__spintent_command_name_str { sp #1 }
3650             \l__spintent_unknown_keys_cmd:n {##1}
3651         },
3652     }
3653 }

```

(End of definition for read-cmd and others.)

11.39.4 Internal functions for \spAfig and \spPfig

\@@_spfig_sym: Reads \l_@@_geo_luaset_poly_sym_str and typesets the corresponding symbol: \triangle for **triangle**, \mdlgwhtsquare for square.

```

3654 \cs_new_protected:Nn \l__spintent_spfig_sym:
3655 {
3656     \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
3657     {
3658         { triangle } { \triangle }
3659         { square } { \mdlgwhtsquare }
3660     }
3661 }

```

(End of definition for \@@_spfig_sym:.)

\@@_spfig_process:nn Sets \l_@@_geo_fig_op_tl to área or perímetro — the word Lua needs to build the **intent** — distinct from the visual symbol $\mathcal{A}/\mathcal{P}$, which is chosen separately when typesetting. When the point list is non-empty, the subscript is built with \c_math_subscript_token rather than a literal $_$, since the latter is not guaranteed to act as a subscript operator inside the token stream built by \MathMLintent.

For the single-argument grammar (§11.39), Lua returns \l_@@_geo_luaset_letra_word_str and \l_@@_geo_luaset_letra_conector_str separately: the connector (de-la, de l, or empty) depends on the

grammatical gender and part of speech of each table letter (F/P are nouns needing an article; B/L are adjectives, taking none), and is inserted before the word only when non-empty.

```

3662 \cs_new_protected:Nn \__spintent_sfig_process:nn
3663 {
3664   \str_if_eq:eeTF {#1} { Afig }
3665   { \tl_set:Nn \__spintent_geo_fig_op_tl { área } }
3666   { \tl_set:Nn \__spintent_geo_fig_op_tl { perímetro } }
3667   \str_set:Ne \__spintent_geo_print_sym_str { \str_use:c { \__spintent_sp #1 _print_sym_str } }
3668   \str_if_eq:eeTF {#1} { Afig }
3669   { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
3670   { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
3671   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3672   {
3673     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3674     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3675   }
3676   \tl_if_empty:cF { \__spintent_sp #1 _read_arg_tl }
3677   {
3678     \str_if_empty:NF \__spintent_geo_luaset_vertices_str
3679     {
3680       \str_set:Ne \__spintent_geo_luaset_intent_str
3681       {
3682         \__spintent_geo_fig_op_tl -del-
3683         \tl_use:c { \__spintent_sp #1 _read_arg_tl } -
3684         \__spintent_geo_luaset_vertices_str
3685       }
3686     }
3687   }
3688   \str_if_eq:VnTF \__spintent_geo_print_sym_str { triangle }
3689   { \tl_set:Nn \__spintent_geo_sym_word_tl { triángulo } }
3690   {
3691     \str_if_eq:VnTF \__spintent_geo_print_sym_str { square }
3692     { \tl_set:Nn \__spintent_geo_sym_word_tl { cuadrado } }
3693     { \tl_clear:N \__spintent_geo_sym_word_tl }
3694   }
3695   \cs_set_protected:Npn \__spintent_sfig_sym_choice:
3696   {
3697     \str_if_eq:VnTF \__spintent_geo_print_sym_str { triangle }
3698     { \triangle } { \mdlgwhtsquare }
3699   }
3700   \tl_if_empty:nTF {#2}
3701   {
3702     \str_if_eq:VnTF \__spintent_geo_print_sym_str { none }
3703     {
3704       \tl_clear:N \__spintent_geo_final_print_tl
3705       \str_set:Ne \__spintent_geo_final_intent_str { \__spintent_geo_fig_op_tl }
3706       \str_set:Ne \__spintent_geo_mathcat_concept_str { \__spintent_geo_fig_op_tl }
3707     }
3708     {
3709       \tl_set:Nn \__spintent_geo_final_print_tl { \__spintent_sfig_sym_choice: }
3710       \str_set:Ne \__spintent_geo_final_intent_str
3711       {
3712         \__spintent_geo_fig_op_tl - del - \__spintent_geo_sym_word_tl
3713       }
3714       \str_set:Ne \__spintent_geo_mathcat_concept_str
3715       {
3716         \__spintent_geo_fig_op_tl - del - \__spintent_geo_sym_word_tl
3717       }
3718     }
3719   }
3720   {
3721     \str_if_eq:VnTF \__spintent_geo_luaset_intent_str { }
3722     {
3723       \str_if_empty:VTF \__spintent_geo_luaset_letra_word_str
3724       {
3725         \str_if_eq:VnTF \__spintent_geo_print_sym_str { none }
3726         {
3727           \tl_set:Nn \__spintent_geo_final_print_tl { \__spintent_geo_luaset_print_tl }
3728           \str_set:Ne \__spintent_geo_final_intent_str
3729           {
3730             \__spintent_geo_fig_op_tl -

```

```

3731         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
3732         \l__spintent_geo_luaset_sub_spoken_str
3733     }
3734 }
3735 {
3736     \tl_set:Nn \l__spintent_geo_final_print_tl
3737     {
3738         \__spintent_spfig_sym_choice:
3739         \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
3740     }
3741     \str_set:Ne \l__spintent_geo_final_intent_str
3742     {
3743         \l__spintent_geo_fig_op_tl - del - \l__spintent_geo_sym_word_tl -
3744         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
3745         \l__spintent_geo_luaset_sub_spoken_str
3746     }
3747 }
3748 }
3749 {
3750     \str_if_eq:VnF \l__spintent_geo_print_sym_str { none }
3751     {
3752         \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
3753         { \c_backslash_str sp #1 }
3754     }
3755     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3756     \str_set:Ne \l__spintent_geo_final_intent_str
3757     {
3758         \l__spintent_geo_fig_op_tl -
3759         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
3760         { \l__spintent_geo_luaset_letra_conector_str - }
3761         \l__spintent_geo_luaset_letra_word_str
3762         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
3763         {
3764             -
3765             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
3766             \l__spintent_geo_luaset_sub_spoken_str
3767         }
3768     }
3769 }
3770 \str_set:Ne \l__spintent_geo_mathcat_concept_str { \l__spintent_geo_fig_op_tl }
3771 }
3772 {
3773     \str_case:VnTF \l__spintent_geo_print_sym_str
3774     {
3775         { auto }
3776         {
3777             \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3778             \str_set:Ne \l__spintent_geo_final_intent_str { \l__spintent_geo_luaset_inten
3779         }
3780         { none }
3781         {
3782             \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3783             \str_set:Ne \l__spintent_geo_final_intent_str { \l__spintent_geo_luaset_inten
3784         }
3785     }
3786     {
3787         \str_if_eq:VVTF \l__spintent_geo_print_sym_str \l__spintent_geo_luaset_poly_sym_s
3788         {
3789             \tl_set:Nn \l__spintent_geo_final_print_tl
3790             {
3791                 \__spintent_spfig_sym_choice:
3792                 \l__spintent_geo_luaset_print_tl
3793             }
3794             \str_set:Ne \l__spintent_geo_final_intent_str { \l__spintent_geo_luaset_inten
3795         }
3796         {
3797             \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
3798             { \c_backslash_str sp #1 }
3799         }
3800     }
3801     \str_set:Ne \l__spintent_geo_mathcat_concept_str { \l__spintent_geo_luaset_concept_str

```

```

3802     }
3803   }
3804   \str_if_eq:eeT { \str_use:c { l__spintrent_sp #1 _mode_str } } { mathcat }
3805   {
3806     \str_set:Ne \l__spintrent_geo_luaset_intent_str { _ \l__spintrent_geo_mathcat_concept_str :
3807   }
3808   \__spintrent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3809   \str_if_eq:eeTF { \str_use:c { l__spintrent_sp #1 _mode_str } } { school }
3810   {
3811     \MathMLintent
3812     {
3813       \tl_if_empty:cF { l__spintrent_sp #1 _prefix_tl }
3814       {
3815         \tl_use:c { l__spintrent_sp #1 _prefix_tl } -
3816       }
3817       \l__spintrent_geo_final_intent_str
3818     }
3819     {
3820       \c__spintrent_invisible_sep_tl
3821       \str_if_eq:eeTF {#1} { Afig } { \__spintrent_mathcal:n { A } } { \__spintrent_mathcal:n
3822       \tl_if_empty:NF \l__spintrent_geo_final_print_tl
3823       {
3824         \c_math_subscript_token { \l__spintrent_geo_final_print_tl }
3825       }
3826     }
3827   }
3828   {
3829     \MathMLintent
3830     {
3831       \tl_if_empty:cF { l__spintrent_sp #1 _prefix_tl }
3832       {
3833         \tl_use:c { l__spintrent_sp #1 _prefix_tl } -
3834       }
3835       \l__spintrent_geo_luaset_intent_str
3836     }
3837     {
3838       \c__spintrent_invisible_sep_tl
3839       \str_if_eq:eeTF {#1} { Afig } { \__spintrent_mathcal:n { A } } { \__spintrent_mathcal:n
3840       \tl_if_empty:NF \l__spintrent_geo_final_print_tl
3841       {
3842         \c_math_subscript_token { \l__spintrent_geo_final_print_tl }
3843       }
3844     }
3845   }
3846 }

```

(End of definition for \@_spfig_process:nn.)

\@@_spfig_exec:nnn Sets the keys in the correct keyspace via \keys_set:nn, then delegates rendering to __spintrent_spfig_process:nn

```

3847 \cs_new_protected:Nn \__spintrent_spfig_exec:nnn
3848 {
3849   \keys_set:nn { spintrent / sp #1 } { #2 }
3850   \__spintrent_spfig_process:nn { #1 } { #3 }
3851 }

```

(End of definition for \@_spfig_exec:nnn.)

\spAfig Converts the -NoValue- marker of an omitted d() argument into an explicit empty group before forwarding, since the Lua parser interprets an empty point list as “no figure — bare symbol only”.

```

3852 \NewDocumentCommand \spAfig { 0{} d() }
3853 {
3854   \mode_if_math:TF
3855   {
3856     \group_begin:
3857     \tl_if_novalue:nTF {#2}
3858     { \__spintrent_spfig_exec:nnn { Afig } { #1 } { } }
3859     { \__spintrent_spfig_exec:nnn { Afig } { #1 } { #2 } }
3860     \group_end:
3861   }
3862   { \msg_error:nnn { spintrent } { need-math-mode } { \spAfig } }
3863 }
3864 \NewDocumentCommand \spPfig { 0{} d() }

```

```

3865 {
3866   \mode_if_math:TF
3867   {
3868     \group_begin:
3869     \tl_if_novalue:nTF {#2}
3870     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
3871     { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
3872     \group_end:
3873   }
3874   { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
3875 }

```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 22.)

11.40 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()` already established by `\spmcld/\spmcm` and `\spAfig/\spPfig`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside/\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.40.1 Definition of variables for `\spcirc`

`\l_@@_geo_luaset_circ_radio_type_str` Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. `\l_@@_geo_luaset_circ_radio_type_str` holds one of *number*, *measure*, *label*, segment, or an empty string when no radius was given. `\l_@@_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

3876 \str_new:N \l_@@_spintent_geo_luaset_circ_radio_type_str
3877 \str_new:N \l_@@_spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\l_@@_geo_luaset_circ_radio_type_str` and `\l_@@_geo_luaset_circ_radio_raw_str`.)

`\l_@@_spcirc_mode_str` `\l_@@_spcirc_prefix_tl` and `\l_@@_spcirc_prefix_tl` follow the same semantics as the corresponding variables of `\spPoint` (§11.32). `\l_@@_spcirc_print_sym_str` stores the value of `print-sym` (usual or *odot*). `\l_@@_spcirc_read_sty_str` stores `read-sty` (*circun* or *circle*). `\l_@@_spcirc_radio_sty_str` stores `radio-sty`, only meaningful when the radius is a segment. `\l_@@_spcirc_sub_index_str` stores `sub-index`. `\l_@@_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to *la* or *el*, matching the grammatical gender of the chosen word.

```

3878 \str_new:N \l_@@_spintent_spcirc_mode_str
3879 \tl_new:N \l_@@_spintent_spcirc_prefix_tl
3880 \str_new:N \l_@@_spintent_spcirc_print_sym_str
3881 \str_new:N \l_@@_spintent_spcirc_read_sty_str
3882 \str_new:N \l_@@_spintent_spcirc_radio_sty_str
3883 \str_new:N \l_@@_spintent_spcirc_sub_index_str
3884 \str_new:N \l_@@_spintent_spcirc_articulo_str
3885 \tl_new:N \l_@@_spintent_spcirc_read_arg_tl

```

(End of definition for `\l_@@_spcirc_mode_str` and others.)

`\l_@@_geo_circ_word_tl` Scratch variables used by `\__spintent_spcirc_print:n` to assemble the chosen word (*circunferencia/círculo*, or the `read-arg` override) and the final literal `intent` string.

```

3886 \tl_new:N \l_@@_spintent_geo_circ_word_tl
3887 \str_new:N \l_@@_spintent_geo_circ_intent_str

```

(End of definition for `\l_@@_geo_circ_word_tl` and `\l_@@_geo_circ_intent_str`.)

11.40.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg`

Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

3888 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
3889 { Invalid~argument~format~in~\token_to_str:N \spcirc:~'#1'. }
3890 {
3891   The~argument~must~be~a~single~point~(the~center),~optionally~
3892   followed~by~a~comma~and~a~radius:~a~pure~number,~a~measure~
3893   (number~+~unit),~a~single~letter,~or~two~concatenated~letters~
3894   (a~segment).
3895 }

```

(End of definition for `spcirc-invalid-arg`.)

11.40.3 Definition of keys for \spcirc

`print-sym` chooses the visual symbol: usual for a plain C, `odot` (default) for `\odot`. `read-sty` chooses the verbalized word and its article: `circun` (default) for «*la circunferencia*», `circle` for «*el círculo*». `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (side, via `\spside`) or its *measure* (mside, default, via `\spmside`). `sub-index` sets a subscript on the symbol; `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig` (§11.39).

```

3896 \keys_define:nn { spintent / spcirc }
3897 {
3898   read-cmd      .choices:nn = { school , mathcat }
3899   {
3900     \int_case:nn { \l_keys_choice_int }
3901     {
3902       {1} { \str_set:Nn \l__spintent_spcirc_mode_str { school } }
3903       {2} { \str_set:Nn \l__spintent_spcirc_mode_str { mathcat } }
3904     }
3905   },
3906   read-cmd      .initial:n = school,
3907   read-cmd      .value_required:n = true,
3908   prefix        .code:n =
3909   { \__spintent_sanitizet_affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
3910   prefix        .initial:n = ,
3911   prefix        .value_required:n = true,
3912   print-sym     .choices:nn = { usual , odot }
3913   {
3914     \int_case:nn { \l_keys_choice_int }
3915     {
3916       {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
3917       {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
3918     }
3919   },
3920   print-sym     .initial:n = odot,
3921   print-sym     .value_required:n = true,
3922   read-sty      .choices:nn = { circun , circle }
3923   {
3924     \int_case:nn { \l_keys_choice_int }
3925     {
3926       {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
3927       {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
3928     }
3929   },
3930   read-sty      .initial:n = circun,
3931   read-sty      .value_required:n = true,
3932   radio-sty     .choices:nn = { side , mside }
3933   {
3934     \int_case:nn { \l_keys_choice_int }
3935     {
3936       {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
3937       {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
3938     }
3939   },
3940   radio-sty     .initial:n = mside,
3941   radio-sty     .value_required:n = true,
3942   sub-index     .code:n =
3943   { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
3944   sub-index     .initial:n = ,
3945   sub-index     .value_required:n = true,
3946   read-sub      .bool_set:N = \l__spintent_spcirc_read_sub_bool,
3947   read-sub      .initial:n = false,
3948   read-sub      .value_required:n = true,
3949   read-arg      .code:n =
3950   { \__spintent_sanitizet_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
3951   read-arg      .initial:n = ,
3952   read-arg      .value_required:n = true,
3953   silent-cmd    .bool_set:N = \l__spintent_spcirc_silent_cmd_bool,
3954   silent-cmd    .initial:n = false,
3955   silent-cmd    .value_required:n = true,
3956   unknown       .code:n =
3957   {
3958     \str_set:Nn \l__spintent_command_name_str { spcirc }

```

```

3959     \__spintent_unknown_keys_cmd:n {#1}
3960   },
3961 }

```

(End of definition for `read-cmd` and others.)

11.40.4 Internal functions for `\spcirc`

`\@@_spcirc_sym:` Typesets `\odot` or the plain letter `C`, according to `print-sym`.

```

3962 \cs_new_protected:Nn \__spintent_spcirc_sym:
3963 {
3964   \str_if_eq:VnTF \l__spintent_spcirc_print_sym_str { odot }
3965   {
3966     \odot
3967   }
3968   { C }
3969 }

```

(End of definition for `\@@_spcirc_sym:`)

`\@@_spcirc_word_and_article:` Sets `\l_@@_geo_circ_word_tl` to the chosen word — either `read-arg`, or `circunferencia/círculo` per `read-sty` — and `\l_@@_spcirc_articulo_str` to the matching article (`la/el`). The article always follows `read-sty`, even when `read-arg` overrides the word itself.

```

3970 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
3971 {
3972   \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
3973   {
3974     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
3975     {
3976       \tl_set:Nn \l__spintent_geo_circ_word_tl { circunferencia }
3977       \str_set:Nn \l__spintent_spcirc_articulo_str { la }
3978     }
3979     {
3980       \tl_set:Nn \l__spintent_geo_circ_word_tl { círculo }
3981       \str_set:Nn \l__spintent_spcirc_articulo_str { el }
3982     }
3983   }
3984   {
3985     \tl_set:Nn \l__spintent_geo_circ_word_tl { \l__spintent_spcirc_read_arg_tl }
3986     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
3987     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
3988     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
3989   }
3990 }

```

(End of definition for `\@@_spcirc_word_and_article:`)

`\@@_spcirc_radio_print:` Dispatches on `\l_@@_geo_luaset_circ_radio_type_str`, reusing the actual public sub-commands rather than reimplementing any parsing: `number` and `measure` delegate to `\spnum/\spunit`; `segment` delegates to `\spside/\spmside` according to `radio-sty` — both already accept the concatenated two-letter form (§11.35) that Lua hands over here. `label` is typeset as a bare letter, with no special intent wrapping. In every case, `\exp_args:No` is required to expand `\l_@@_geo_luaset_circ_radio_raw_str` into the sub-command's argument, since a plain brace group does not expand a `str` variable on its own.

```

3991 \cs_new_protected:Nn \__spintent_spcirc_radio_print:
3992 {
3993   \str_case:Vn \l__spintent_geo_luaset_circ_radio_type_str
3994   {
3995     { number } { \exp_args:No \spnum { \l__spintent_geo_luaset_circ_radio_raw_str } }
3996     { measure } { \exp_args:No \spunit { \l__spintent_geo_luaset_circ_radio_raw_str } }
3997     { label } { \l__spintent_geo_luaset_circ_radio_raw_str }
3998     { segment }
3999     {
4000       \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
4001       { \exp_args:No \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
4002       { \exp_args:No \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
4003     }
4004   }
4005 }

```

(End of definition for `\@@_spcirc_radio_print:`)

`\@@_spcirc_print:n` Calls `\__spintrent_luafun_geo_circ_parse_and_set:n` to resolve the center and classify the radius, then assembles `\l_@@_geo_circ_intent_str` — the word, an optional `sub-index` suffix, `de-centro-en-`, the center, and `-y-radio` when a radius is present. Critically, when a radius is present, the visual is split across *two* separate `\MathMLintent` calls with `\__spintrent_spcirc_radio_print:` — which typesets a sub-command carrying its own independent `intent` — sitting as a *sibling* in between, rather than nested inside either one. Embedding a full sub-command’s own `\MathMLintent` inside another command’s content argument makes MathCAT read only the outer literal string, exactly as observed with `mover/msub` structures (§11.37); as *siblings*, both are read in sequence as one continuous phrase.

```

4006 \cs_new_protected:Nn \__spintrent_spcirc_print:n
4007 {
4008   \__spintrent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
4009   \str_if_eq:VnT \l__spintrent_geo_luaset_error_str { true }
4010   {
4011     \msg_error:nne { spintrent } { spcirc-invalid-arg } { \exp_not:n {#1} }
4012   }
4013   \__spintrent_spcirc_word_and_article:
4014   \str_set:Ne \l__spintrent_geo_circ_intent_str { \l__spintrent_geo_circ_word_tl }
4015   \str_if_empty:NF \l__spintrent_spcirc_sub_index_str
4016   {
4017     \str_put_right:Nn \l__spintrent_geo_circ_intent_str
4018     {
4019       -
4020       \bool_if:NT \l__spintrent_spcirc_read_sub_bool { sub - }
4021       \l__spintrent_spcirc_sub_index_str
4022     }
4023   }
4024   \str_put_right:Ne \l__spintrent_geo_circ_intent_str
4025   {
4026     -de-centro-en-\l__spintrent_geo_luaset_print_tl
4027   }
4028   \str_if_empty:NF \l__spintrent_geo_luaset_circ_radio_type_str
4029   {
4030     \str_put_right:Nn \l__spintrent_geo_circ_intent_str { -y-radio }
4031   }
4032   \__spintrent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
4033   \str_if_empty:NTF \l__spintrent_geo_luaset_circ_radio_type_str
4034   {
4035     \MathMLintent
4036     {
4037       \tl_if_empty:NF \l__spintrent_spcirc_prefix_tl
4038       {
4039         \tl_use:N \l__spintrent_spcirc_prefix_tl -
4040       }
4041       \l__spintrent_geo_circ_intent_str
4042     }
4043     {
4044       \c__spintrent_invisible_sep_tl
4045       \__spintrent_spcirc_sym:
4046       \str_if_empty:NF \l__spintrent_spcirc_sub_index_str
4047       {
4048         \c_math_subscript_token { \l__spintrent_spcirc_sub_index_str }
4049       }
4050       ( \l__spintrent_geo_luaset_print_tl )
4051     }
4052   }
4053   {
4054     \MathMLintent
4055     {
4056       \tl_if_empty:NF \l__spintrent_spcirc_prefix_tl
4057       {
4058         \tl_use:N \l__spintrent_spcirc_prefix_tl -
4059       }
4060       \l__spintrent_geo_circ_intent_str
4061     }
4062     {
4063       \c__spintrent_invisible_sep_tl
4064       \__spintrent_spcirc_sym:
4065       \str_if_empty:NF \l__spintrent_spcirc_sub_index_str
4066       {
4067         \c_math_subscript_token { \l__spintrent_spcirc_sub_index_str }

```

```

4068         }
4069         ( \l__spintent_geo_luaset_print_tl ,
4070         }
4071         \__spintent_invisible_sep:
4072         \__spintent_spcirc_radio_print:
4073         \MathMLintent { :silent } { } }
4074     }
4075 }

```

(End of definition for \@@_spcirc_print:n.)

\@@_spcirc_exec:nn

```

4076 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
4077 {
4078     \keys_set:nn { spintent / spcirc } { #1 }
4079     \__spintent_spcirc_print:n { #2 }
4080 }

```

(End of definition for \@@_spcirc_exec:nn.)

`\spcirc` Follows the same `d()`-argument pattern as `\spmcd/\spmcm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

4081 \NewDocumentCommand \spcirc { 0{} d() }
4082 {
4083     \mode_if_math:TF
4084     {
4085         \group_begin:
4086         \tl_if_novalue:nTF {#2}
4087         { \__spintent_spcirc_exec:nn { #1 } { } }
4088         { \__spintent_spcirc_exec:nn { #1 } { #2 } }
4089         \group_end:
4090     }
4091     { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
4092 }

```

(End of definition for `\spcirc`. This function is documented on page ??.)

11.41 Finish package

Finish package implementation.

```

4093 \file_input_stop:
4094 \endpackage

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

Symbols	
@@ commands:	
\@@_collapse_hyphens:N	<u>120</u>
\l_@@_command_name_str	<u>195</u>
\l_@@_geo_circ_intent_str	<u>109</u> , <u>3886</u>
\l_@@_geo_circ_word_tl	<u>108</u> , <u>3886</u>
\l_@@_geo_fig_op_tl	<u>102</u> , <u>2521</u>
\l_@@_geo_luaset_angulo_case_str	<u>2521</u>
\l_@@_geo_luaset_circ_radio_raw_str	<u>108</u> , <u>3876</u>
\l_@@_geo_luaset_circ_radio_type_str	<u>108</u> , <u>3876</u>
\l_@@_geo_luaset_error_str	<u>81</u> , <u>2521</u>
\l_@@_geo_luaset_intent_pts_str	<u>98</u> , <u>99</u> , <u>2521</u>
\l_@@_geo_luaset_intent_str	<u>81</u> , <u>84</u> , <u>98</u> , <u>99</u> , <u>2521</u>
\l_@@_geo_luaset_is_name_str	<u>84</u> , <u>2521</u>
\l_@@_geo_luaset_letra_conector_str	<u>103</u>
\l_@@_geo_luaset_letra_word_str	<u>103</u>
\l_@@_geo_luaset_poly_sym_str	<u>99</u> , <u>101</u> , <u>102</u> , <u>2521</u>
\l_@@_geo_luaset_print_tl	<u>81</u> , <u>2521</u>
\l_@@_geo_side_cmd_tl	<u>84</u> , <u>90</u> , <u>93</u> , <u>2521</u>
\@@_invisible_sep:	<u>106</u>
\c_@@_invisible_sep_tl	<u>79</u> , <u>106</u>
\@@_luafun_clean_split_and_set:n	<u>65</u> , <u>232</u>
\@@_luafun_spang_parse:n	<u>74</u>
\@@_luafun_sptime_parse:n	<u>59</u>
\@@_luafun_spmQ_scale_int:nn	<u>65</u>
\@@_luafun_spmoney_lookup_data:n	<u>65</u>
\@@_luafun_sptime_parse:n	<u>58</u>
\@@_luafun_spunit_lookup_alias:n	<u>65</u>
\l_@@_luaset_arg_above_tl	<u>251</u>
\l_@@_luaset_arg_below_tl	<u>251</u>
\l_@@_luaset_cal_fam_ok_str	<u>63</u>
\l_@@_luaset_dec_is_all_zeros_str	<u>244</u>
\l_@@_luaset_dec_sep_tl	<u>232</u>
\l_@@_luaset_decimal_period_str	<u>244</u>
\l_@@_luaset_denom_has_number_str	<u>251</u>
\l_@@_luaset_exponent_tl	<u>244</u>
\l_@@_luaset_format_part_dec_str	<u>239</u>
\l_@@_luaset_format_part_int_str	<u>239</u>
\l_@@_luaset_has_exponent_str	<u>244</u>
\l_@@_luaset_has_integer_str	<u>79-81</u> , <u>232</u>
\l_@@_luaset_has_natural_str	<u>232</u>
\l_@@_luaset_has_units_str	<u>251</u>
\l_@@_luaset_millions_str	<u>244</u>
\l_@@_luaset_not_decimal_not_period_str	<u>244</u>
\l_@@_luaset_not_decimal_period_str	<u>244</u>
\l_@@_luaset_only_part_dec_str	<u>239</u>
\l_@@_luaset_only_part_int_str	<u>239</u>
\l_@@_luaset_only_part_period_str	<u>239</u>
\l_@@_luaset_part_dec_tl	<u>232</u>
\l_@@_luaset_part_int_tl	<u>79</u> , <u>232</u>
\l_@@_luaset_part_period_tl	<u>232</u>
\l_@@_luaset_sign_tl	<u>79</u> , <u>232</u>
\l_@@_luaset_status_str	<u>251</u>
\l_@@_luaset_sym_fam_ok_str	<u>63</u>
\@@_mathcal:n	<u>80</u>
\l_@@_mc_args_seq	<u>1924</u>
\@@_mc_process_args:nn	<u>1983</u>
\l_@@_md_out_seq	<u>2071</u>
\@@_print_spunit_hierarchy:	<u>661</u>
\@@_right_angle_sqr:	<u>93</u>
\@@_sanitize_affix:Nn	<u>128</u>
\l_@@_saved_luauml_flag_int	<u>79</u> , <u>2324</u>
\l_@@_setspintent_input_arg_cmd_tl	<u>146</u>
\@@_setspintent_parse_args:nn	<u>166</u>
\@@_setspintent_set_keys:nn	<u>155</u>
\l_@@_smparc_prefix_tl	<u>3265</u>
\l_@@_spAfig_mode_str	<u>3557</u>
\l_@@_spAfig_prefix_tl	<u>3557</u>
\l_@@_spAfig_print_sym_str	<u>3557</u>
\l_@@_spAfig_read_arg_tl	<u>3557</u>
\l_@@_spPfig_print_sym_str	<u>3557</u>
\l_@@_spPfig_mode_str	<u>3557</u>
\l_@@_spPfig_prefix_tl	<u>3557</u>
\l_@@_spPfig_read_arg_tl	<u>3557</u>
\@@_spPoint_exec:nn	<u>2623</u>
\l_@@_spPoint_mode_str	<u>84</u> , <u>86</u> , <u>89</u> , <u>92</u> , <u>95</u> , <u>98</u> , <u>2533</u>
\l_@@_spPoint_prefix_tl	<u>84</u> , <u>86</u> , <u>89</u> , <u>92</u> , <u>95</u> , <u>98</u> , <u>2533</u>
\@@_spPoint_print:n	<u>2571</u>
\@@_spPoly_exec:nn	<u>3542</u>
\l_@@_spPoly_mode_str	<u>3418</u>
\l_@@_spPoly_prefix_tl	<u>3418</u>
\@@_spPoly_print:n	<u>3465</u>
\@@_spPoly_sym:	<u>3457</u>
\l_@@_sp\#1_mode_str	<u>2907</u> , <u>3087</u>
\l_@@_sp\#1_prefix_tl	<u>2907</u> , <u>3087</u>
\l_@@_sp\#1_read_arg_str	<u>2907</u>
\l_@@_sp\#1_read_sty_str	<u>2907</u>
\l_@@_spang_luaset_error_str	<u>2211</u>
\l_@@_spang_luaset_grado_str	<u>2211</u>
\l_@@_spang_luaset_minuto_str	<u>2211</u>
\l_@@_spang_luaset_segundo_str	<u>2211</u>
\@@_spang_process:n	<u>2220</u>
\@@_spangle_exec:nnn	<u>3240</u>
\@@_spangle_process:nn	<u>3160</u>
\@@_spangle_sym:n	<u>3141</u>
\@@_sparc_exec:nnn	<u>3393</u>
\l_@@_sparc_mode_str	<u>3265</u>
\l_@@_sparc_prefix_tl	<u>3265</u>
\@@_sparc_print_arc:n	<u>3316</u>
\@@_sparc_process:nn	<u>3322</u>
\@@_sparc_wide_arc:n	<u>3312</u>
\l_@@_spcirc_articulo_str	<u>106</u> , <u>108</u> , <u>3878</u>
\@@_spcirc_exec:nn	<u>4076</u>
\l_@@_spcirc_mode_str	<u>106</u> , <u>3878</u>
\l_@@_spcirc_prefix_tl	<u>106</u> , <u>3878</u>

\@@_spcirc_print:n	4006	\l_@@_spmoney_luaset_plur_str	784
\l_@@_spcirc_print_sym_str	106, 3878	\l_@@_spmoney_luaset_position_str	784
\@@_spcirc_radio_print:	3991	\l_@@_spmoney_luaset_print_iso_str	784
\l_@@_spcirc_radio_sty_str	106, 3878	\l_@@_spmoney_luaset_sing_str	784
\l_@@_spcirc_read_sty_str	106, 3878	\l_@@_spmoney_luaset_status_str	784
\l_@@_spcirc_sub_index_str	106, 3878	\l_@@_spmoney_luaset_subplur_str	784
\@@_spcirc_sym:	3962	\l_@@_spmoney_luaset_subsing_str	784
\@@_spcirc_word_and_article:	3970	\l_@@_spmoney_luaset_symbol_str	784
\l_@@_spdate_luaset_error_str	1457	\@@_spmoney_normalize_key:n	819
\l_@@_spdate_luaset_output_str	1457	\@@_spmoney_parse_and_route:n	1060
\@@_spdate_process:n	59, 1468	\@@_spmoney_print_dec_sub_unit:	1012
\@@_spdiv_process:nnn	1828	\@@_spmoney_print_int:	901
\@@_spdiv_render_op:nn	1807	\@@_spmoney_print_integer_continue:	1012
\@@_spdiv_render_parens:nn	1816	\@@_spmoney_print_sep_con:	1012
\l_@@_spdsym_prefix_tl	1634	\@@_spmoney_print_symbol:	867
\l_@@_spdsym_read_sym_tl	1634	\@@_spmoney_render_layout:	880
\l_@@_spdsym_suffix_tl	1634	\@@_spmoney_render_number_node:	892
\l_@@_spdsym_symbol_tl	1634	\@@_spmoney_render_only_integer_node:	892
\@@_spfig_exec:nnn	3847	\l_@@_spmoney_res_main_voice_str	794
\@@_spfig_process:nn	3662	\@@_spmoney_set_and_print_sub_units:	965
\@@_spfig_sym:	3654	\@@_spmoney_set_currency:n	819
\@@_spmQ_exec:nnnn	2499	\l_@@_spmoney_sub_units_bool	794
\l_@@_spmQ_join_word_str	79, 2324	\l_@@_spmoney_subnum_int	794
\l_@@_spmQ_math_style_tl	2324	\@@_spmoney_validate_money:	851
\@@_spmQ_print:nnn	2491	\l_@@_spmsym_prefix_tl	1704
\l_@@_spmQ_print_dfrac_bool	80	\l_@@_spmsym_read_sym_tl	1704
\@@_spmQ_print_sacle_int:n	2448	\l_@@_spmsym_suffix_tl	1704
\@@_spmQ_print_scale_int:	2439	\l_@@_spmsym_symbol_tl	1704
\@@_spmQ_print_spnZ_dfrac:nn	2476	\@@_spmUL_process:nnn	1899
\@@_spmQ_set_join_word:	2430	\@@_spmUL_render_op:nn	1878
\@@_spmQ_set_wrap_parens_size:	2375	\@@_spmUL_render_parens:nn	1887
\@@_spmQ_start_wrap_parens:	2410	\l_@@_spnD_luaset_is_truncated_str	2071
\@@_spmQ_stop_wrap_parens:	2410	\l_@@_spnD_luaset_result_tl	2071
\l_@@_spmQ_use_spnZ_bool	80	\l_@@_spnD_prefix_tl	2071
\l_@@_spmarc_mode_str	3265	\l_@@_spnD_print_result_bool	2071
\@@_spmc_process:nnnn	2005	\l_@@_spnD_print_result_num_tl	2071
\l_@@_spmcd_prefix_tl	1924	\l_@@_spnD_silent_cmd_bool	2071
\l_@@_spmcd_print_result_bool	1924	\l_@@_spnM_luaset_is_truncated_str	2071
\l_@@_spmcd_print_upper_bool	1924	\l_@@_spnM_luaset_result_tl	2071
\l_@@_spmcd_read_terse_bool	1924	\l_@@_spnM_prefix_tl	2071
\l_@@_spmcd_suffix_tl	1924	\l_@@_spnM_print_result_bool	2071
\l_@@_spmcm_prefix_tl	1924	\l_@@_spnM_print_result_num_tl	2071
\l_@@_spmcm_print_result_bool	1924	\l_@@_spnM_silent_cmd_bool	2071
\l_@@_spmcm_print_upper_bool	1924	\@@_spnM_spnD_exec:nnnn	2152
\l_@@_spmcm_read_terse_bool	1924	\@@_spnM_spnD_intent:nn	2117
\l_@@_spmcm_spmcd_luaset_error_str	1924	\@@_spnM_spnD_process:nnnnn	2112
\l_@@_spmcm_suffix_tl	1924	\@@_spnM_spnD_result:n	2133
\@@_spmoney_build_tree:	1012	\@@_spnM_spnD_silent:nn	2125
\@@_spmoney_check_sub_units:	965	\l_@@_spnQ_intent_read_sign_str	2324
\l_@@_spmoney_extracted_curr_str	794	\l_@@_spnQ_read_parens_bool	78
\l_@@_spmoney_extracted_num_tl	794	\l_@@_spnQ_wrap_parens_l_tl	78, 2324
\@@_spmoney_intent_int:	901	\l_@@_spnQ_wrap_parens_r_tl	78, 2324
\l_@@_spmoney_intent_money_str	794	\l_@@_spnum_intent_read_dec_sep_str	35, 37, 270
\l_@@_spmoney_intent_sign_str	794	\l_@@_spnum_intent_read_sign_str	35, 270
\l_@@_spmoney_intent_sub_unit_str	794	\l_@@_spnum_notation_sym_tl	270
\l_@@_spmoney_luaset_conde_str	784	\@@_spnum_print_integer:	437
\l_@@_spmoney_luaset_currency_arg_str	784	\@@_spnum_print_num_start:	437

\@@_spnum_print_number_core: 437

\@@_spnum_print_part_dec: 416

\@@_spnum_print_part_period: 416

\@@_spnum_print_period_bar: 345

\l_@@_spnum_read_period_sep_str 35, 270

\l_@@_spnum_read_terse_bool 35, 270

\@@_spnum_render_dec: 328

\@@_spnum_render_decimal_sep: 382

\@@_spnum_render_int: 328

\@@_spnum_render_only_period: 345

\@@_spnum_render_part:n 328

\@@_spnum_render_period_sep: 345

\@@_spnz_process:nn 2298

\@@_spnz_render_parens: 2286

\@@_sprayo_exec:nn 2892

\l_@@_sprayo_mode_str 2779

\l_@@_sprayo_prefix_tl 2779

\@@_sprayo_print:n 2831

\l_@@_sprayo_read_sty_str 87, 2779

\@@_spread_exec:nn 1615

\l_@@_spread_intent_tl 1614

\@@_sprecta_exec:nn 2764

\l_@@_sprecta_mode_str 2638

\l_@@_sprecta_prefix_tl 2638

\@@_sprecta_print:n 2675

\@@_spshort_execute_layout: 1168

\l_@@_spshort_luaset_base_str 1085

\l_@@_spshort_luaset_expanded_str 1085

\l_@@_spshort_luaset_layout_str 1085

\l_@@_spshort_luaset_output_str 1085

\l_@@_spshort_luaset_status_str 1085

\l_@@_spshort_luaset_suffix_str 1085

\@@_spshort_print:nn 1121

\@@_spshort_process_and_render:n 1133

\@@_spshort_render_bypass:n 1144

\@@_spshort_render_error:n 1144

\@@_spshort_render_fallback: 1144

\@@_spshort_render_success: 1144

\@@_spshort_tag_span:nn 1094

\@@_spside_exec:nnn 3062

\@@_spside_process:nn 2977

\l_@@_spsiglo_luaset_arabic_str 1225

\l_@@_spsiglo_luaset_error_str 1225

\l_@@_spsiglo_luaset_roman_min_str ... 1225

\l_@@_spsiglo_print_era_str 1225

\@@_spsiglo_print_math:n 1309

\@@_spsiglo_print_text:n 1309

\@@_spsiglo_render_simple:nn 1260

\@@_spsiglo_render_with_era:nnn 1260

\@@_spsiglo_tag_span:nn 1296

\@@_sptime_build_seconds:nn 1389

\@@_sptime_build_tree: 58, 1407

\l_@@_sptime_luaset_base_str 1375

\l_@@_sptime_luaset_error_str 1375

\l_@@_sptime_luaset_has_seconds_str ... 1375

\l_@@_sptime_luaset_is_fraction_str ... 1375

\l_@@_sptime_luaset_seconds_str 1375

\l_@@_spunit_exp_seq 531

\l_@@_spunit_exponent_tl 531

\@@_spunit_format_canonical: 594

\l_@@_spunit_is_mult_bool 531

\l_@@_spunit_luaset_canonical_str 524

\l_@@_spunit_luaset_compact_exp_str ... 524

\l_@@_spunit_luaset_is_compact_str ... 524

\l_@@_spunit_luaset_is_sexagesimal_str 524

\l_@@_spunit_luaset_lookup_status_str . 524

\l_@@_spunit_luaset_read_str 524

\l_@@_spunit_luaset_status_str 524

\l_@@_spunit_mult_seq 531

\@@_spunit_process_base_exp:n 594

\@@_spunit_process_mult:n 654

\l_@@_spunit_read_cdot_bool 531

\l_@@_spunit_read_slash_str 531

\@@_spunit_safe_exec:n 673

\l_@@_spunit_unit_name_tl 531

\l_@@_spusep_arg_tl 60, 61, 1500

\@@_spusep_convert:n 1562

\l_@@_spusep_left_tl 60, 61, 1500

\l_@@_spusep_right_tl 60, 61, 1500

\@@_spusep_set_parens:nn 1551

\l_@@_spusep_silent_bool 61

\@@_unknown_keys:n 222

\@@_unknown_keys:nn 222

@@ internal commands:

\l_@@_spnum__notation_sym_tl 35

A

\angle 93

C

cifras-sep 275, 560, 833, 1782, 1853, 2265

Document class:

\l3doc 28

clist commands:

\clist_map_inline:nn 89, 92, 95

cmd-key-unknown 195

cmd-key-value-unknown 195

Commands provide by **spintent**:

\setspintent 31

\spmoney 45

\spnum 35

\spsiglo 54, 57

\spunit 40

currency 833

currency-no-subunits 802

D

dolar-math 833

E

exp commands:

\exp_args:No 108

I

inline-numeric-denominator 538

invalid-angle-format 2215

invalid-century-format 1229

invalid-currency 802

invalid-date-format 1459

invalid-time-format 1380

J

join-word	2340
-----------------	------

K

Keys for \spAfig provide by spintent:

prefix	101
print-sym	101
read-arg	101
read-cmd	101
read-sub	101
silent-cmd	101

Keys for \spangle provide by spintent:

recto	92
silent-cmd	92

Keys for \spcirc provide by spintent:

print-sym	106–108
radio-sty	106–108
read-arg	106, 108
read-sty	106–108
read-sub	107
sub-index	106, 107, 109

Keys for \spdiv provide by spintent:

cifras-sep	65
notation-sym	65
read-parens	65
read-period-sep	65
read-sign	65
read-sym	65
set-sym	65
wrap-parens	65

Keys for \spdsym provide by spintent:

prefix	63
read-sym	63
set-sym	63
suffix	63

Keys for \spmangle provide by spintent:

print-m	92
---------------	----

Keys for \spmc d provide by spintent:

prefix	69, 71
read-cmd	69
result-num	71
result	69
sample-arg	69
silent-cmd	69
upper-case	69

Keys for \spmc m provide by spintent:

prefix	69, 71
read-cmd	69
result-num	71
result	69
sample-arg	69
silent-cmd	69
upper-case	69

Keys for \spmoney provide by spintent:

cifras-sep	46
currency	46
dolar-math	46
sub-units	46

Keys for \spmQ provide by spintent:

join-word	77, 79
print-dfrac	77–80
read-parens	77, 78
use-spnZ	77, 80
wrap-parens	77, 78

Keys for \spmside provide by spintent:

print-m	88, 89
---------------	--------

Keys for \spmsym provide by spintent:

not-print	64
prefix	64
read-sym	64
set-sym	64
suffix	64

Keys for \spmul provide by spintent:

cifras-sep	67
notation-sym	67
read-parens	67
read-period-sep	67
read-sign	67
read-sym	67
set-sym	67
wrap-parens	67

Keys for \spnD provide by spintent:

result	71
silent-cmd	71

Keys for \spnM provide by spintent:

result	71
silent-cmd	71

Keys for \spnum provide by spintent:

cifras-sep	34–36
notation-sym	35
read-period-sep	35, 37
read-sign	35

Keys for \spnZ provide by spintent:

cifras-sep	75
read-parens	75, 76
read-sign	75
wrap-parens	75, 76

Keys for \spPfig provide by spintent:

prefix	101
print-sym	101
read-arg	101
read-cmd	101
read-sub	101
silent-cmd	101

Keys for \spPoint provide by spintent:

prefix	82
read-cmd	82
silent-cmd	82

Keys for \spPoly provide by spintent:

print-sym	98
silent-cmd	98

Keys for \sprayo provide by spintent:

prefix	87
read-cmd	87
read-sty	86, 87
silent-cmd	87

Keys for \sprecta provide by spintent:

prefix	84
read-cmd	84
silent-cmd	84

Keys for \spshort provide by spintent:

read-arg	52
small-caps	52

Keys for \spside provide by spintent:

read-arg	88, 89
read-sty	88, 89
silent-cmd	89

Keys for \spsiglo provide by spintent:

print-era	54, 56
-----------------	--------

Keys for \spunit provide by spintent:

cifras-sep	41
------------------	----

notation-sym	41	result-num	2081
print-cdot	41	right-angle-sqr-fallback	70
read-cdot	41	\rightangle	93
read-period-sep	41		
read-slash	41	S	
Keys for \spusep provide by spintent :		sample-arg	1942, 2081
silent	60	seq commands:	
size	60	\seq_set_from_clist:Nn	58
\keys	98	set-sym	1638, 1708, 1782, 1853
keys commands:		\setspintent	3, 31, 183
\keys_set:nn	83, 86, 88, 91, 94, 97, 100, 105	silent	1503
		silent-cmd	1942, 2081, 2543, 2647, 2790, 2925, 3104, 3281, 3428, 3601, 3896
M		size	1503
mathcal-fallback	70	small-caps	1107
\MathMLintent	99, 101, 102, 109	\spAfig	22, 101, 3852
mc-invalid-arguments	1935	spAfig-invalid-points	3568
md-invalid-argument	2107	\spang	21, 74, 2255
\mdlghwtsquare	99, 102	\spangle	20, 92, 3245
\measuredangle	93	spangle-invalid-arg	3092
\measuredrightangle	93	\sparc	21, 95, 3398
Lua module provide by spintent :		sparc-invalid-points	3270
spintent.lua	28, 29, 33–35, 40, 44–46, 51, 52, 54, 56, 57, 59, 68, 71, 74, 76, 81, 82	\spcirc	106, 4081
msg commands:		spcirc-invalid-arg	3888
\msg_error:nnnn	89	\spdate	11, 59, 1483
		\spdiv	13, 65, 1843
N		spdiv-empty-argument	1802
not-print	1708	\spdsym	12, 63, 1685
notation-sym	275, 560, 1782, 1853	\spmangle	20, 92, 3245
		spmangle-invalid-arg	3092
O		\spmarc	21, 95, 3398
\odot	108	spmarc-invalid-points	3270
\overleftrightharpoon	84	\spmcid	15, 68, 2051
		\spmcn	15, 68, 2051
P		\spmoney	6, 45, 1074
Packages :		\spmQ	16, 76, 2511
expl3	33, 35	spmQ-not-integer	2330
lua-unicode-math	28	spmQ-zero-denominator	2330
luamml	30, 77, 79	\spmside	19, 88, 3067
tagpdf	51, 55	spmside-invalid-points	2914
unicode-math	28, 61	\spmsym	12, 64, 1757
prefix	1638, 1708, 1942, 2081, 2543, 2647, 2790, 2925, 3104, 3281, 3428, 3601, 3896	\spmul	13, 67, 1914
print-dfrac	2340	spmul-empty-argument	1873
print-era	1237	\spnD	14, 71, 2191
print-m	2925, 3104	\spnewunit	6, 44, 752
print-sym	3428, 3601, 3896	spnewunit-duplicate	731
		\spnM	14, 71, 2191
R		\spnum	3, 35, 36, 506
radio-sty	3896	spnum-has-units	492
read-arg	1107, 2925, 3601, 3896	spnum-no-digits	492
read-cdot	560	\spnZ	16, 75, 2314
read-cmd	1942, 2543, 2647, 2790, 2925, 3104, 3281, 3428, 3601, 3896	spnz-not-integer	2281
read-parens	1782, 1853, 2265, 2340	\spPfig	23, 101, 3852
read-period-sep	275, 560, 1782, 1853	spPfig-invalid-points	3568
read-sign	275, 1782, 1853, 2265	\spPoint	17, 82, 2628
read-slash	560	spPoint-invalid-point	2535
read-sty	2790, 2925, 3896	\spPoly	22, 98, 3547
read-sub	3601, 3896	spPoly-invalid-points	3420
read-sym	1638, 1708, 1782, 1853		
recto	3104		
result	1942, 2081		

<code>\spray</code>	18, 86, 2897	<code>\str_lowercase:n</code>	58
<code>\sprayo-invalid-points</code>	2782	<code>\str_uppercase:n</code>	58
<code>\spread</code>	11, 62, 1624	<code>sub-index</code>	3896
<code>spread-empty-args</code>	1611	<code>sub-units</code>	833
<code>\sprecta</code>	18, 84, 2769	<code>suffix</code>	1638 , 1708
<code>sprecta-invalid-points</code>	2640		
<code>\spshort</code>	7, 51, 1212	T	
<code>\spside</code>	18, 88, 3067	token commands:	
<code>spside-invalid-points</code>	2914	<code>\c_math_subscript_token</code>	102
<code>\spsiglo</code>	11, 54, 1362	<code>\triangle</code>	99, 102
<code>\sptime</code>	11, 57, 1439		
<code>\spunit</code>	4, 40, 41, 43, 719	U	
<code>spunit-invalid-unit</code>	538	unknown	1503 , 1638 , 1708 , 2340 , 2543 , 2647 , 2790 , 2925 , 3104 , 3281 , 3428 , 3601 , 3896
<code>spunit-no-units</code>	538	unknown-abbreviation	1091
<code>\spunitalias</code>	6, 44, 766	unknown-choice	195
<code>spunitalias-duplicate</code>	731	unknown-command-setup	146
<code>spunitalias-invalid-expr</code>	731	upper-case	1942
<code>\spusep</code>	12, 60, 1600	use-spnZ	2340
<code>spusep-empty-arg</code>	1493		
<code>spusep-invalid-arg</code>	1493		
str commands:		W	
<code>\str_if_eq:nnTF</code>	58	wrap-parens	1782, 1853, 2265, 2340